ZkregPlus: More Efficient Zero Knowledge Argument for Multiple Regular Regex Patterns

Anonymous Author(s)

ABSTRACT

The `zkreg` problem is about providing a a zero knowledge proof for a committed string being or not being a member of a regular expression. We consider how to scale existing techniques for handling the non-membership problem for a large collections of regex specifications (e.g., $38k$ extended logical regex patterns of ClamAV). Naive intersection or negation of regex patterns can easily lead to exponential state space explosion. Instead, we present a disjunctive proof scheme based on the following observation: a "benign" string may only be a "near miss" for very few regex patterns, only for which, the prover pays the computing cost. A second contribution of this paper is the identification of a regex fragment called "Bounded Pattern Regex" which captures 95% of patterns in ClamAV. It yields a more efficient proof scheme both asymptotically and concretely. The prover complexity is $O(n\log(n) + m\log(m)x)$ where n is the input string length, m is the total number of accepting states along the acceptance of the string over an Aho-Corasick automaton, and x is the number of regex operators of those regex patterns that required to be discharged. The verifier complexity is $O(1)$. In practice, m and x are small, e.g., $m = 1024$ and $x = 20$ for all Linux Ubuntu executables with regard to 95% of ClamAV patterns, which incurs only a constant factor of cost of committing to the string being proved.

KEYWORDS

datasets, neural networks, gaze detection, text tagging

1 INTRODUCTION

Proving a committed string belonging (or not) to a regular expression specification has wide applications. For example, it is shown in Zombie [96] and ZkMiddleBox [57] how to prove in zero knowledge an encrypted network packet does not contain addresses in a blocked list. In [83], all Linux executables are proved to be malware free regarding the ClamAV hex-signatures. In [5], committed DNA sequences are shown to contain chromosomes. In [102], zero knowledge proof is applied to redacted emails, twitters, and credit records.

Various techniques have different aims, e.g., striving for expressiveness (providing negation in Zombie [96]), efficiency for large collections of fixed patterns [83], skipping widecard matches for handling very long input strings [5], and general NFA [73].

This paper considers two research problems: (1) can we scale variety of zkreg proof techniques for handling a large *collection* of regex-patterns and prove that a hidden string *is not a member* of any of these patterns? Note that this is a difficult problem, except for the case of fixed patterns using AC-DFA [83], to intersect or negate regex can easily lead to exponential cost; (2) does there exist more efficient weaker regex subset which leads to more efficient techniques? We tackle the above research problems by providing the following contributions. We find our technique very useful, e.g., in policy compliant secure messaging [**?**], where content moderation is needed in certain context, such as preventing child sexual abuse.

Empirical analysis of regex in practice: The analysis forms the basis for efficient ZKP techniques we introduce in this paper. We perform an empirical analysis of the clamav regex and a comprehensive data that that comprises of all executables in a Linux CentOS, malware samples, both benign and malicious emails and SQL database logs (TODO). We observe that for all benign binary executables in Linux, in average they have no more than 4 "near-misses", i.e., containing any of the fixed string patterns extracted from ClamAV regex signatures. Similar observations apply to benign email and SQL database logs, which implies the effectiveness for a disjunctive ZKP scheme which only needs the prover to pay for cost of discharging those "near-miss" regex signatures. We also notice that only a very small portion of signatures have class operators and limited use of Kleene stars. This leads to a "bounded matching Regex" fragment (PM-Reg) we introduce later. *To verify: Using PM-Reg to approximate the general regex patterns has no false positives/negatives on all the data set that we have experimented with.*

Efficient Application of Lookup argument: In [83] it is observed that the generation of transition set and check if they are valid can be separated, i.e., a valid encoding of a transition can be looked up in a pre-processed transition table. We further extend this approach and replace the zk-set component in [83] with a pre-processed lookup argument [42], which leads to prover complexity completely *independent of* the specification size, i.e., depending on the input string length only. Compared with related work such as [57, 73, 96] this approach largely improves the practicality of `zkreg` in the context of non-membership proofs with large automaton specification.

Scale-up to Multiple Patterns: Given the observation that a benign string has only a few "near misses" to large collection of regex patterns. We design a 2-stage zk-proof scheme for disjunctive proof. For each regex signature, we

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–39

https://doi.org/XXXXXXX.XXXXXXX

collect a set of fixed patterns as its "critical fixed patterns", missing any of these patterns means that the input string will not match the given pattern. We then compile an AC-DFA over all such fixed patterns, and feed the input string through the automaton. We collect a much small list of final states that appear along the acceptance path, and use these final states to retrieve the list of signatures that should be proved (and the others be ignored). For signatures of different types (e.g., PM-Reg and general regular expressions), they are compiled into separate circuits and folded as an IVC scheme by leveraging ProtoGalaxy [43]. The verifier has no knowledge that whic patterns are actually being discharged, and via which approach.

Bounded Matching Regex: Based on data analysis we present a *strictly weaker* fragment of regex, however, which is capable of capturing 97% of ClamAV patterns. We develop efficient constraint encoding techniques which encode the distance between regex component and leave the fixed components to be handled by more efficient AC-DFA. All PM-Reg are compiled into one general purpose circuit, while its distance encoding constraints are loaded at run-time, again leveraging lookup arguments.

Cryptographic Constructions: Of independent interest, we present a bounded zero knowledge version of the `cq` lookup argument, which does not rely on the Aurora lemma [14] and results in faster concrete prover performance. We present a modified ProtoGalaxy folding scheme, enhanced with folding techniques for zk-`cq` lookup argument. We implement the schemes mentioned above fully, by instrumenting a PSE-fork of the Halo2 system [95]. We evaluate our scheme over the full ClamAV signature set (which consists of $38k$ regex signatures), and the data sets of emails and SQL database logs. The Halo2 system enhanced with `cq`-lookups and ProtoGalaxy schemes is general purpose, and can be applied to other problems.

2 BACKGROUND

2.1 Research Problem

We[1] consider the problem of *discharging* a *benign* input string from a large collection of regex malware signatures (i.e., the string does not match any of the signatures).

Definition 2.1. Let $R = \{r_1, ..., r_n\}$ be a collection of regex signatures. We *discharge* a string s for R if we show that $\forall i \in [n] : s \notin L(r_i)$.

Constructing an efficient zk-prover, even when $|R| = 1$ is challenging. It is known [85][Chapter 5.4] that a regex r can be translated to NFA size of $3|r|$. To prove a string $s \notin L(r)$, one can discharge it by performing a BFS search over the NFA graph, which results in complexity $O(|r||s|)$, and this zk-prover complexity is achieved in Zombie [96]. The concern in practice is that the concrete prover cost is high. The ClamAV logical signatures have an average length of 300, thus to discharge an $1MB$ file against one regex signature needs over 2^{30} R1CS constraints.

The straw-man approach of discharging regex signatures one by one will not work, as usually the collection of signatures is large. For instance, the ClamAV logical regex signatures has $38k$ patterns. Another approach is to build an automaton from $\bigwedge_{i=1}^{n} \bar{r_i}$. This is not practical as NFA conversion to DFA is needed for regular expression negation, and DFA intersection causes exponential state explosion.

It may seem completely impractical to produce concretely efficient zk-proofs for discharging benign files for malware signature sets used by the industry. However, in this paper, we present technical solutions which allow us to discharge an entire image of binary executables of a Linux operating system, or the entire Enron email dataset for all ClamAV signatures and SNORT signatures. We develop *conservative* over-approximations of regex signatures for discharging a benign input string, and we design efficient encoding of these approximations by leveraging folding and IVC schemes in zk-provers.

Both ClamAV and SNORT ruleset support full fledged PCRE regex patterns. However, not all advanced features of PCRE regex are used "equally", which influence the zk-prover design. Thus, we delve into the technical details of these virus signatures first, and explore the language features (mainly PCRE) that we need to support.

2.2 ClamAV Logical Signatures

As ClamAV signatures subsume the syntax of SNORT, for introducing the regex syntax, we focus on ClamAV only. There are three types of ClamAV signatures: (1) hash signatures, where a SHA256 or MD5 is generated directly from a malicious file; (2) fixed word signatures where Aho-Corasick can be directly applied to discharge them (and the zk-prover in [83] can discharge an entire Linux image efficiently); and (3) logical regex signatures which empowers the full PCRE regex syntax. In ClamAV signature set[2], there are 38839 signatures. To discharge hash signatures, one can simply encode the hash operation of an input string and produce the hash as the secret output witness of the circuit, and then use a lookup argument (e.g., [42, 82, 93, 94]) or zero knowledge non-membership proof to discharge it. The cost is linear on the input string length and can be independent of the hash signature set size. In summary, the $38k$ logical regex signatures are the ones to discharge in this paper. We illustrate its syntax using a factious example to integrate all language features, and for convenience of presentation the patterns in the example are much shorter than the real signatures.

Example 2.2. The vast majority of ClamAV signatures are encoded using 4-bit nibbles, because in many cases, binary (executable) contents are the target. However, ClamAV can be also used to handle HTTP and email contents. In this case, the text information is represented by their encoding (e.g., ASCII). For instance an ASCII string "`ab`" is encoded

[1] **NOTE:** this section contains the complete formal definition of all approximation approaches. Later, needs to split into two sections: one for background/walk-around the general idea, and then another section of algorithms after the preliminary section.

[2] ClamAV 0.103.11.27152, retrieved January 12, 2024

as "6162" for 0x61 is the ASCII value for 'a'. The following is a sample ClamAV logical signature.

```
FakeVirus-012;Engine:51-255,Target:0;0|(1=0&2>2);300:61
↪ 6161??3131*31;626262{2-}323232::i;/^([^a]+bc){1,3}/
```

A logical signature consists of three parts: (1) the meta information (e.g., the virus name, the ClamAV engine version needed, and the `Target` which specifies the file types such as executables and emails that should be checked, and a 0 indicates all files. [3]. (2) the logical relation between sub-signatures. For instance: in Example 2.2, a file is regarded as containing virus `FakeVirus-012` if there is at least one match of sub-signature 0, or sub-signature 2 has more than 2 matches, [4] and sub-signature 1 has no match. Notice that here by specifying counter constraints `1=0`, it is essentially specifying the negation of a regex constraint. (3) sub-signatures. They are separated by semicolons. There are two types of sub-signatures: *hex* and *PCRE* sub-signatures.

A *hex* sub-signature is pattern oriented, and is *weaker* then regex, in the way that repetitions (such as Kleene starts) can be applied to wildcard match only. Here a `?` stands for a wild card of one hex nibbles, and `{4}` stands for 4 bytes (8 nibbles). Note that as a *hex* sub-signature can have wildcard matches, and can express distances between sub-patterns, it is more expressive than the fixed-word signatures in ClamAV that directly encoded using Aho-Corasick automata.

A *PCRE* regex allows full PCRE regex syntax such as character classes such as `[a-z0-9]`, negated char classes such as `[^a]`, macros like `\w`, zero-length assertions such as `$`, and also named back-references and look-arounds such as `(?P=tagname)`. Out of the 37955 logical signatures, there are 559 (1.4%) patterns that carry PCRE regex signatures. Note that look-arounds exceed the capability of regex, and they are approximated. But such cases are rare, only 6 out of 559 PCRE sub-signatures have such features, and they can be approximated without raising false-positives of virus on the data set of all Linux binary executables and Enron emails. [5]

We use a uniform alphabet of hex digits to encode all finite state machines used in this paper. Thus, the PCRE regex sub-signatures are converted to this alphabet. One benefit is that the complete DFA resulted from such signatures needs less space for encoding transitions, because each state has at most 16 out-going transitions. Given the discussion above, we can translate all sub-signatures of `FakeVirus-012` into the following over hex alphabet:

r_0 := `.{600}616161..3131.*31.*`

r_1 := `.*(62|42)(62|42)(62|42).....*323232.*`

r_2 := `.*((00|...|60|62|...|ff)+6263){1,3}.*`

Observe r_0 for sub-signature 0: it first encodes the position requirement `300:`, which requires that the match starts at exactly the 300'th byte from the beginning of the document, into a wildcard of 600 nibbles. Then each wildcard match `?` is translated into a `.`, and lastly a wildcard string `.*` is appended at the end because we need to express the concept the the pattern is *contained* in the target file.

For r_1, the ignore case `::i` requirement is handled by syntax re-writing. It is apparent that a requirement of "sub-signature 1 has no matches" in target string s can be expressed by: $s \in \overline{r_1}$, because the wildcard strings at the beginning and end of r_1. Similarly, negated char class in r_2 is handled by re-writing. Finally, the logical signature can be denoted by the following extended regex pattern (in the sense that intersection and negation are allowed in regex specification):

$$r_0 \vee (\overline{r_1} \wedge r_2 r_2 r_2)$$

If one wishes to discharge a string s of virus `FakeVirus-012`, then it is to prove:

$$s \in \overline{r_0} \wedge (\overline{r_1} \vee \overline{r_2 r_2 r_2})$$

Intuitively, it implies that one has to discharge s of r_1 first, and then discharge either of r_1 or $r_2 r_2 r_2$.

One straw-man approach would be: construct DFA out of the above regular expression, and then the zk-proof for discharging s would be simply linear with $|s|$, given that encoding finite state machine is "free" using pre-processed lookup arguments such as `cq` [42]. Unfortunately, it is known that the worst complexity of constructing DFA out of regex is exponential, and so is negation. We performed an empirical (best-effort) attempt to convert all sub-signatures of ClamAV set to DFA, using a server with 128GB ram (running with 32-threads). A failure is reported when the conversion times out at 100 seconds, or exhausts the 128GB RAM. Out of the 38k logical signatures, there are 16.3k failed to generate DFA.

As an example, observe sub-signature 1 in the following virus signature. Setting the repetitions (replacing the {28}) to 2 results in a minimized DFA of 1622 states, but at 16 it already has 56910 states. The NFA to minimized DFA conversion crashes at repetition 17 by exhausting all $128GB$ RAM resources.

```
Win.Trojan.Zeroaccess-6932503-0;Engine:81-255,Target:1;
↪ 0&1&2;4d535642564d{2}2e444c4c::i;56423521f01f{28}0a
↪ 00{16}00f0300000fffffff080000000100000000000;61d5893c
↪ 82478645bf6059f9d3408757
```

Combining all, In Figure 1 we present the formal syntax of the regex (covering the special logical structure and counter constraints) of ClamAV. This also subsumes the syntax for the PCRE regex used in both ClamAV and Snort. We call it *LC-Regex*. We use Σ to denote an alphabet of 16 hex digits in this paper.

Here a sub-signature is a full-fledged PCRE excepts look-around and back-references, as noted earlier the rare instances are re-written to equivalent standard regex or approximated. We skip all "syntax sugars" such as character classes and

[3] In this work, we apply all signatures to all file types as to check file content type needs expressiveness greater than regex.

[4] We interpret the semantics as each match is not nested in another. For instance, for regex `a+b`, string aabb contains 1 match.

[5] For instance given pattern `<(?P<tag>([a-z]+)).*?>.*?</(?P=tag)>`, we approximate it to: `<(([a-z]+)).*?>.*?</([a-z]+)>` in the sense that any concrete string that match the original regex also matches the over-approximated regex. There are 6 patterns (1 positive look-ahead and 5 negative look-ahead) that use look-around, and we manually convert them to equivalent ones (for positive look-ahead) and approximated ones (for negative look-ahead) for engineering convenience. we re-ran ClamAV over data samples so that no false-positives of virus report are generated.

s	::=		sub-signature
	|	a	character $a \in \Sigma$
	|	.	any
	|	$s*$	
	|	$s+$	repetition
	|	$s_1 s_2$	concatenation
	|	$s_1 \| s_2$	alternation
	|	(s)	grouping
n	::=	$[0-9]+$	number
r	::=		LC-Regex
	|	s	sub-signature
	|	$r_1 \| r_2$	alternation
	|	$r_1 \& r_2$	intersection
	|	$\overline{r}$	negation
	|	$s\ (> \| = \| <)\ n$	counter constraint

Figure 1: Regular Expression with Logical Structures and Counter Constraints (LC-Regex)

repetition operators as they all can be re-written with the basic LC-Regex. For example, PCRE regex /[a-d]/ is re-written as (61|62|63|64).

The intersection and negation is allowed only at the logical layer, where sub-signatures are composed via logical connectors, and counter constraints are defined also at this layer. Counter constraints could be re-written, however, we specifically represent them so that approximation techniques introduced later can leverage the structural information directly. It is apparent that LC-Regex captures the complete regular language.

2.3 Approximations

Our goal is to design an efficient zk-proof scheme that can batch discharge an input string against a large collection of regex patterns. We would like to avoid NFA encoding of regex patterns (to avoid the complexity of $O(|s||r|)$). Due to the difficulty of constructing DFA, we would not construct DFA whenever possible (in fact, we show that for the entire ClamAV rule set, it sufficies to generate DFA for only 6 rules). In this section, we present four approximation approaches that allow us to avoid expensive single NFA discharging approach completely. In terms of the cost for batch discharging, their cost increases in ascending order. Thus, the strategy is to filter all patterns through these approximations one by one. They are: (1) critical pattern (denoted as `CP`), (2) bag of critical pattern sets(`BW`), (3) single thread distance encoding (`STD`), and DFA (`DFA`). We describe the idea of each approximation briefly and present data to support our argument.

2.3.1 Critical Pattern (CP). Consider discharging `51525354` for a subsignature s_1 := `3132{2-}(61626364|41424344)`. A human reader can immediately verify the case without even running the DFA of the pattern, because none of the long pattern words such as `61626364` or `41424344` appears in the target string. These long pattern words are required for any string to match the subsignature. We call them "critical patterns". Formally it is defined as follows:

Definition 2.3. Given a regular expression r, a set of strings $\texttt{CP}(r)$ is called a *critical pattern set* for r if for any string $t \in r$ there exists a string $s \in \texttt{CP}(r)$ s.t. s is a substring in t.

Intuitively, if none of the words in $\texttt{CP}(r)$ appears in a string s, then s can be discharged for r. One should note that a regular expression may have many critical pattern sets. For instance for s_1 mentioned above, both {`61626364,41424344`} and {3132} are valid `CP`, however, {`61626364`} is not, because 313241424344 is a valid match for s_1 and it does not have to contain `61626364`. In practice, we use a heuristic to collect those that are the longest and those appear less frequently in other subsignatures (thus {3132} is not preferred for s_1). Later, we show that `CP`can be constructed from `BS`.

Consider a simple scenario that all regular patterns has only one hex subsignature and no counter constraints. Given a large collection of regular patterns $R = \{r_1, ..., r_n\}$, we collect one critical pattern for each r_i, and construct an Acho-Corasick autaton (AC-DFA) on all these fixed word patterns. Notice that AC-DFA is deterministic, and its construction complexity is superior compared with constructing DFA from regex: it is *linear* in the total length of fixed words. Then given an input string s, we can simply collect the output words of AC-DFA along its acceptance path. Those regular patterns that do not appear (to be associated) with the collected output words do not have to be considered. In this case, we can discharge over 99.48% ClamAV rules for the vast majority of binary executables and email files in Enron data set (in the worst case there are 198 out of 38297 ClamAV rules left to discharge for the next approximation, for one large executable file. For other executables and emails in Enron data set, the number of left-over is much smaller, as their size is smaller and are less likely to contain a lot of critical patterns).

In practice, we need to address more delicacies. For instance, we need to construct two ACDFAs, one for case sensitive and one for case-insensitive. We also need to consider the logical structure of a ClamAV rule which involves multiple subsignatures (and their negation cases). For instance, subsignatures that are negated and counter constraints such as $0 < 2$ do not have critical pattern set (that is, they cannot be discharged via critical patterns). Also, we need to consider the logical structures. For instance, for a logical signature $(r_1|r_2) \wedge (r_3|r_4)$ we only need to pick one out of $\texttt{CP}(r_1) \cup \texttt{CP}(r_2)$ and $\texttt{CP}(r_3) \cup \texttt{CP}(r_4)$.

2.3.2 Bag of Critical Pattern Set (BS). Bag set can be regarded as a straightforward extension of the critical pattern set model. It is a *conjunction* of a set of critial pattern sets. To discharge via `BS` in zk-prover, however, is rather different from the previous approach. We rely on ACDFA to count word occurences and a ternary logic system [81], which allows reasoning about "maybe".

A ternary-value logic system has three values in the system: `T` (True), `F` (False), and `U` (Unknown). We need three

∧	T	F	U
T	T	F	U
U	U	F	U
F	F	F	F

∨	T	F	U
T	T	T	T
U	T	U	U
F	T	F	U

¬	T	F	U
	F	T	U

Figure 2: Truth Table of Ternary-Logic

operators: ¬ (negation), ∧ (and), and ∨ (or), and their truth tables are defined in Figure 2.

Definition 2.4. Given a regular expression subsignature r, a bag of critical pattern set for r,denoted as $\texttt{BS}(r)$, is a set s.t. each of its element is a distinct $\texttt{CP}(r)$.

Intuitively, a word $s \in r$ must satisfy each critical pattern set in $\texttt{BS}(r)$. In another word, if there exists $\vec{u} \in \texttt{BS}(r)$ s.t. for all $t \in \vec{u}$: t is not a substring of s, then $s \notin r$.

Example 2.5. Given a regex $r :=$ `6162(4142|4344)5152`, both {{`6162`},{`4142,4344`},{`5152`}} and {{`6162`}, {`5152`}} are $\texttt{BS}(r)$.

In Figure 3 we present algorithms for generating `BS`, and for discharging a string by `BS`. `gen_bag_sets()` is defined based on the recursive definition of a subsignature. One interesting point is that dropping a critical pattern set from a $\texttt{BS}(r)$ yields a valid but coarser $\texttt{BS}(r)$, as shown in Example 2.5. When handling the wildcard `.` case, the true `BS` is $\{\Sigma\}$, but we return $\emptyset$ to avoid state explosion. Another interesting point is how alternation is handled. For instance, for regex subsignature `ab((12.*34)|(56.*78))cd` the corresponding bag of sets generated are {{`ab`}, {`12,56`}, {`12,78`}, {`34,56`}, {`34,78`}, {`cd`}}, resulting from a cross product operation of $\texttt{BS}(\texttt{12.*34})$ and $\texttt{BS}(\texttt{56.*78})$. In practice, cross-product may result in explosion of terms. We use heuristics to drop terms above a certain pre-set limit (e.g., 2048). Such an approximation is conservative.

Before we discuss how to discharge in batches using `BS`, consider how ACDFA can be leveraged first. Given a large collection of subsignatures $\{r_1, ..., r_n\}$, an ACDFA can be constructed from $\bigcup_{i\in[n]} \texttt{BS}(r_i)$. Then running a string s through the ACDFA, we obtain an acceptance path, from which, one can obtain a hashmap ρ which maps from each accepted word to the *count* of the word in the string. Notice that as all patterns are long (the vast majority has length greater than 10), the ratio of final states in the acceptance path is low, and the size of ρ is much smaller than $|s|$ itself. **In our data set (full ClamAV signatures over 700MB Linux binary executables), $|\rho|$ is only 0.06% of $|s|$, It implies a speed up factor of over 1600× compared with discharging using DFA directly.**

With such information, we can evaluate counter constraints efficiently, and evaluate a regex subsignature is a special case of it. Observe function $\texttt{eval\_cs\_bs}(\rho, r, op, n)$ in Figure 3. The function takes the following input: word frequency map ρ, a counter constraint r *op* n, and it tries to decide if the string represented by ρ can be evaluated by `BS`. It returns a ternary logic value, and when it returns `T` (`F`), it implies

---

1 $\texttt{BS} \leftarrow \texttt{gen\_bag\_sets}(s)$:
Generate `BS` for subsignature s
(1) Denote `gen_bag_sets` as `gbs`.
(2) Return `BS`(s) based on its structure:

$$\begin{cases} a \in \Sigma^+ & \Rightarrow \{\{a\}\} \\ . & \Rightarrow \emptyset \texttt{ \# to avoid state explosion} \\ s_1 s_2 & \Rightarrow \texttt{gbs}(s_1) \cup \texttt{gbs}(s_2) \\ s_1* & \Rightarrow \emptyset \\ s_1+ & \Rightarrow \texttt{gbs}(s_1) \\ s_1 \mid s_2 & \Rightarrow \{\vec{u} \cup \vec{v} \mid \vec{u} \in \texttt{gbs}(s_1) \wedge \vec{v} \in \texttt{gbs}(s_2)\} \end{cases}$$

2 $\texttt{T/U/F} \leftarrow \texttt{eval\_counter\_cs\_bs}(\rho, r, \texttt{op}, n)$:
Evaluate counter constraint by bag of critical pattern sets
(1) Compute $\texttt{occ} = \texttt{min}\left(\{\sum_{w\in\vec{v}} \rho(w) \mid \vec{v} \in \texttt{BS}\}\right)$.
(2) `return` ternary value according to the following chart:

op	T	F	U
¿	–	occ¡=n	occ¿ n
¡	occ¡n	–	occ¿ n-1
= (where n¿0)	–	occ¡n	occ¿ n-1
= (where n=0)	occ = 0	–	occ¿ 0

3 $\texttt{T/U/F} \leftarrow \texttt{eval\_subsig\_bs}(\rho, r)$:
Evaluate subsignature by `BS`
`return` $\texttt{eval\_counter\_cs\_bs}(\rho, r, >, 0)$

4 $\texttt{T/U/F} \leftarrow \texttt{eval\_lc\_bs}(\rho, r)$:
Evaluate LG-regex by `BS`
Here r_i: LC-regex, s_i: subsignature
(1) Denote `eval_lc_bs` by textttdlb.
(2) `return` one of the following based on the structure of r:

$$\begin{cases} s & \Rightarrow \texttt{eval\_subsig\_bs}(\rho, s) \\ s \texttt{ op } n & \Rightarrow \texttt{eval\_counter\_cs\_bs}(\rho, s, \texttt{op}, n) \\ \overline{r_1} & \Rightarrow \neg\texttt{eval\_lc\_bs}(\rho, r_1) \\ r_1 | r_2 & \Rightarrow \texttt{eval\_lc\_bs}(\rho, r_1) \vee \texttt{eval\_lc\_bs}(\rho, r_2) \\ r_1 \& r_2 & \Rightarrow \texttt{eval\_lc\_bs}(\rho, r_1) \wedge \texttt{eval\_lc\_bs}(\rho, r_2) \end{cases}$$

Figure 3: Bag of Critical Pattern Set Algorithms

that the string generating ρ really meets (does not meet) the counter constraint. A `U` indicates an unsure value. Thus the approach is conservative. Then, the ternary value allows composing the evaluation of subsignatures from an LC-regex, as shown in function $\texttt{eval\_lc\_bs}()$.

We now delve into the details of $\texttt{eval\_counter\_cs\_bs}()$ by an example. Given counter constraint r `op` n, it first computes $\texttt{BS}(r)$. For each critical pattern set in $\texttt{BS}(r)$, it uses ρ to compute the sum of occurences of all the words in the critical set pattern, and denote that as the occurrence for that pattern set. Then it iterates through all the pattern sets and get the minimum occurrence (denoted as `occ` in the function). It then use `occ` to decide if the string behind ρ satisfies the constraint.

Example 2.6. Let $r :=$ `aa(11|22)bb`, and let the counter constriant be $r < 2$. One valid $\texttt{BS}(r)$ is {{`aa`}, {`11,22`}, {`bb`}}. Given string $s :=$ `aa112211bbaa`, its word frequency map ρ is: {`aa` → 2, `11` → 2, `22` → 1, `bb` → 1}. Given ρ, the algorithm can compute the total of occurrence for each of the three critical pattern sets. They are 2, 3, and 1, respectively for {`aa`}, {`11,22`}, {`bb`}. Then `occ` is set to minimum of them,

i.e., 1. Following the chart, it decides s satisfies the counter constraint $r < 2$. The intuition is clear: the required pattern `bb` only appeared once, there is no way for the entire string to have more than 2 matches of r.

Fix $r :=$ `aa(11|22)bb`. The following tables lists more examples. A string s is discharged of a regular subsignature r when the output is `F` (a sure false value in ternary logic).

s	counter constraint	occ	output
1122bbaabb	$r > 0$	1	U
11aa	$r > 0$	0	F
aa11bbaa22bb	$r = 2$	2	U
aa22bbbbbbbb	$r < 2$	1	T

2.3.3 Single-Thread Distance Encoding (SDE). We now introduce Single-Thread Distance Encoding (`SDE`) for capturing the distance between pattern words, which `BS` cannot.

Consider $r :=$ `ab(12|34)cd`, one valid `SDE`(r) is `ab.{2,2}cd`. It captures the fact that the distance between `ab` and `cd` should be 2. The motivation for abstracting away alternation and nested structures in regex is based on producing an efficient zk-prover algorithm using ACDFA. We provide a formal definition of `SDE` below.

Definition 2.7. Given a regex subsignature s, we call another regex r a valid Single Thread Distance Encoding of s, written as `SDE`(s), if $L(s) \subseteq L(r)$ and r satisfies the following BNF syntax:

$$r := a \in \Sigma^+ \mid .\{n_1, n_2\} \mid r_1 r_2,$$

where r_1 and r_2 are `SDE` of some regex subsignatures. Here $n_1 \leq n_2$ and we abuse the notation of n_2 to allow ∞, so $.+$ can be represented as $.\{1, \infty\}$.

Each `SDE` regex has a unique simplified `SDE` which consists of alternating wildcard strings and fixed patterns. For instance `.{2,5}ab.{3,3}.{4,5}cd` can be simplified to `.{2,5}ab.{7,8}cd`. The function `simplify`(s) is given in Fig 4. It applies a reduction of merging consecutive fixed words and wildcards and produces an equivalent `SDE`. Note that || stands for fixed word concatenation in its function body.

Similarly to `BS`, we can define `SDE` recursively based on the subsignature structure. This is implemented in function `gen_subsig_sde`(s) in Figure 4. It relies on the `wildcard`(s) function which approximates a a fixed word to wildcards, e.g., from `abc` to `.{3,3}`. Then using it, alternations are abstracted away by keeping track of the length bounds of all components, e.g., `(ab|cdef)` is approximated to `.{2,5}`. It is clear that the approximation is conservative in the sense that any fixed word $w \in L(s)$ also belongs to $L($`SDE`$(s))$.

In function `eval_counter_cs_sde`$(w, s,$ `op`$, n)$ we evaluate whether fixed word w satisfies counter constraint s `op` n, and evaluation of a general subsignature is a special case of it. Then using the ternary logic one can extend to LC-regex.

We present an example to illustrate how the algorithm works. Again, like the `BS` method, the approach relies on ACDFA to generate a map ρ which maps each accepted word to a set of indices where it appears in the target string. When the function is called, ρ instead of the target string is passed for mass processing.

In practice, ρ needs to go through two levels of table projections to optimize zk-prover performance. For the first projection, after the `BS` completes, the zk-prover collects all signatures that need to go to `SDE`, and project ρ to contain entry words that are related to signatures failed by `BS`only. This is a one time projection and the cost is $O(|s|)$. Let there be u signatures to handle in the `SDE` stage, usually $u > 100$ for file size 2^{20}. Given the cost is spread to all signatures, the first projection cost is negligible in average for discharging a signature via `SDE`. The first projection results in a hashmap $\rho^{(1)}$, which is about 5% of $|s|$.[6]

To further cut the cost, for each signature involved (let it be written r_i), $\rho^{(1)}$ is further projected into a $\rho_i^{(2)}$, an individual table for r_i. Layer 2 projection can be processed in batch, as a variation of permutation argument for an inflated table from $\rho^{(1)}$. The maximum inflation rate according to our data, is only 2.17. This implies the total combined cost for layer 2 table projection is $O(|\rho^{(1)}|)$ in total for all signatures.

Then $\rho_i^{(2)}$ is used for the `SDE` process for each signature r_i, note that $\sum_{i=1}^{u} |\rho^{(2)}| \approx |\rho^{(1)}|$. The combined `SDE` cost for all subsignatures is thus $O(\rho^{(1)})$. The concrete data shows that the combined `SDE` witness length in average is only 2.5% (and max 60.8%) of the file size. Thus, in total for u signatures, the zk-prover pays combined cost only a constant factor (of < 2) of file size. Compared with DFA discharging, the speed-up of `SDE` depends on the value of u, which is typically > 100 for file size 2^{20}.

Example 2.8. Let $r =$ `ab(12|32)cd` and $w =$ `ab11cd00ab22cd`. The goal is to evaluate w against counter constraint $r > 2$.

The algorithm first collects bag of words from all related signatures (which includes `ab` and `cd`). Then it computes ρ for w with ACDFA, where $\rho = \{$`ab` $\to \{0, 8\}$, `cd` $\to \{4, 12\}, \}$, e.g., `ab` appeared at indices 0 and 8 in w.

When processing `eval_counter_cs_sde`$(\rho, r, >, 2)$, the algorithm first computes repeated r as the following, to require 3 non-overlapping match of r:

`.*ab(12|32)cd.*ab(12|32)cd.*ab(12|32)cd.*`

Its simplified `SDE` is:

$$\{0,\infty\}\texttt{ab}.\{2,2\}\texttt{cd}.\{0,\infty\}\texttt{ab}.\{2,2\}\texttt{cd}.\{0,\infty\}\texttt{ab}.\{2,2\}\texttt{cd}.\{0,\infty\}$$

Note that we have 6 fixed pattern words in the above `SDE`. Using ρ, the allowed position set are computed, as shown below:

$\varphi_0 = \{0\}$	$\varphi_1 = \{0, 8\}$	$\varphi_2 = \{4, 12\}$	$\varphi_3 = \{8\}$
$\varphi_4 = \{12\}$	$\varphi_5 = \emptyset$	$\varphi_6 = \emptyset$	-

Intuitively, φ_1 reflects the permitted positions of the 1st occurrence of `ab`, φ_2 the positions for the 1st occurrence of `cd`, etc. Each round, the algorithm proceeds from the the last position set, advances with the length of the last word and

[6] The result is obtained at bounding the `SDE` fixed patterns with length at least 4. It is possible to achieve better compression ratio by increasing minimum length, however, reducing effectiveness of `SDE` thus leaving more signatures to the DFA stage.

the wildcard length bound in between, and then intersect with with the set of occurrence indices of the next word.

As shown in the computation, there is no valid position for the third pair of ab and cd. Following the chart, ternary value `F` is returned, which discharges w for counter constraint $r > 2$.

2.3.4 DFA and Individual SDE. When a signature cannot be discharged by `SDE`, it goes to the `DFA` stage where each subsignature is translated to a deterministic finite state machine, then the target string is fed to each DFA and then the results are combined using the logical expression as defined in the signature. Note that this is not an approximation, as long as the DFAs can be constructed for each subsignature within the constraint of time and RAM resources available, which is unfortunately not the case.

In theory, we can discharge a string using NFA (by enumerating all possible state set reached by the input string), as done in Zombie [96]). However, we would like to avoid the cost of $O(|\texttt{NFA}| \times |s|)$. In this case, we will run another round of `SDE`.

Recall that in `SDE` there is a minimum requirement of pattern word length (e.g., 4 nibbles). The reason is that if the pattern word is too short, it blows up the size of the final states along the ACDFA acceptance path. There are very few ClamAV signatures which have short pattern words, meanwhile, its DFA is too costly to construct, e.g., as shown below, where the pattern words are single characters intermixed with variable length wildcards, which fails all aforementioned discharging methods. In addition, its use of named back-references exceeds capability of regex.

```
Js.File.MaliciousHeuristic-6260249-0;Engine:81-255,Targ
↪ et:7;0>1&1>7&2&3;6576616c28;76617220;0&1/w(?P<junk>
↪ [a-z\d]{3,6})s(?P=junk)c(?P=junk)r(?P=junk)i(?P=jun
↪ k)p(?P=junk)t(?P=junk).(?P=junk)s(?P=junk)h(?P=junk
↪ )e(?P=junk)l(?P=junk)l/;
↪ ...
```

In this case, we resolve to *Individual ACDFA SDE* (shortened `ISDE`). We extract the bag of words from the signature again, without the minimum word length. Construct an ACDFA for that subsignature only, and run the `SDE` approach again. In this case, the ACDFA does not "mass process" a lot of signatures, but just one. This approach tends to be more expensive than DFA, and we place it in the last round.

3 PRELIMINARIES

3.1 Notations

We use $\mathbf{P}$ and $\mathbf{V}$ to denote the prover and the verifier. Let $\mathcal{G}$ be a generator of bilinear groups, i.e., $(p, \mathbf{g}_1, \mathbf{g}_2, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda)$, where all groups have order p, and $\mathbf{g}_1$ ($\mathbf{g}_2$) the generator of $\mathbb{G}_1$ ($\mathbb{G}_2$). $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is the bilinear map. Let $\mathbb{F}$ be the scalar field of $\mathbb{G}_1$. $s \xleftarrow{\$} \mathbb{F}$ represents sampling s uniformly from $\mathbb{F}$. We use n and N to denote the size of query and lookup tables. We assume the existence of n-th and N-th roots of unity so FFT is applicable. We follow the notations in Groth16 [56]. For instance, ${\mathbf{g}_1}^a {\mathbf{g}_1}^b$ is written as $[a]_1 + [b]_1$.

---

1 `SDE` $\leftarrow$ `simplify`(s):
produce a simplified version of s where s is a `SDE` for some r
Apply reduction below to s until no more can be applied.

$$\begin{cases} a_1 a_2 \text{ where } a_i \in \Sigma^+ & \Rightarrow & a_1 || a_2 \\ .\{m_1, m_2\}.\{n_1, n_2\} & \Rightarrow & .\{m_1 + n1, m_2 + n2\} \end{cases}$$

2 `SDE` $\leftarrow$ `wildcard`(s):
Assuming s is already a `SDE`, produce its wildcard approximation
(1) Let s' be the result of approximating each fixed word a in s to $.\{|a|, |a|\}$.
(2) `return simplify`(s').

3 `SDE` $\leftarrow$ `gen_subsig_sde`(s):
Generate `SDE` for subsignature s
(1) Denote `gen_subsig_sde` as `gss`.
(2) Compute s_2 based on the structure of s:

$$\begin{cases} a \in \Sigma^+ & \Rightarrow & a \\ . & \Rightarrow & . \\ s_1 s_2 & \Rightarrow & \texttt{gss}(s_1) || \texttt{gss}(s_2) \\ s_1 * & \Rightarrow & .\{0, \infty\} \\ s_1 + & \Rightarrow & \texttt{gss}(s_1) || .\{0, \infty\} \\ s_1 \mid s_2 & \Rightarrow & .\{a, b\} \text{ where} \\ & & .\{u, v\} = \texttt{wildcard}(\texttt{gss}(s_1)) \text{ and} \\ & & .\{x, y\} = \texttt{wildcard}(\texttt{gss}(s_2)) \\ & & a = \texttt{min}(u, x) \text{ and } b = \texttt{max}(v, y) \end{cases}$$

4 `T/U/F` $\leftarrow$ `eval_counter_cs_std`$(\rho, r, \texttt{op}, n)$:
Evaluate counter constraint by SDE
$\rho[u]$ is the set of u's occurrence indices in target string to eval.
(1) Let $n' = \begin{cases} r > n & \Rightarrow & n+1 \\ r = 0 & \Rightarrow & 1 \\ r = n (n > 0) & \Rightarrow & n \\ r < n & \Rightarrow & n \end{cases}$.
(2) Define r' as r repeated n' times intermixed with `.*`.
(3) Compute $e' = \texttt{simplify}(\texttt{SDE}(r'))$. Write e' as the following:

$$.\{a_1, b_1\} w_1 .\{a_2, b_2\} w_2 \ldots .\{a_k, b_k\} w_k .\{a_{k+1}, b_{k+1}\}$$

(4) Define $\varphi_0 = \{0\}$ and $w_0 = \epsilon$. For $i \in [k]$, iteratively compute:

$$\varphi_i = \{[u + a_i + |w_{i-1}|, u + b_i + |w_{i-1}|] \cap \rho(w_i) \mid u \in \varphi_{i-1}\}$$

(5) `return` ternary value according to the following chart:

op	T	F	U
$>$	–	$\varphi_k = \emptyset$	$\varphi_k \neq \emptyset$
$<$	$\varphi_k = \emptyset$	–	$\varphi_k \neq \emptyset$
$=$ (where $n > 0$)	–	$\varphi_k = \emptyset$	$\varphi_k \neq \emptyset$
$=$ (where $n = 0$)	$\varphi_k = \emptyset$	–	$\varphi_k \neq \emptyset$

Figure 4: Single Thread Distance Encoding Algorithms

We use $[n]$ to denote range $[1, n]$. A vector $\mathbf{t} \in \mathbb{F}^n$ is written as $(\mathbf{t}_1, \ldots, \mathbf{t}_n)$. $\mathbf{u} + \mathbf{v}$ denotes $(\mathbf{u}_1 + \mathbf{v}_1, \ldots, \mathbf{u}_n + \mathbf{v}_n)$, and $\alpha\mathbf{u}$ is $(\alpha\mathbf{u}_1, \ldots, \alpha\mathbf{u}_n)$. We use $\mathbf{u}||\mathbf{v}$ to represent concatenation of $\mathbf{u}$ and $\mathbf{u}$. We use ω_n for the n'th root of unity, i.e., $(\omega_n)^n = 1$. We define $\mathbb{H}_{Nn} = \{\omega_n^1, \ldots, \omega_n^n\}$, and the corresponding set of Lagrange base polynomials is denoted as $\{L_{n,i}(X)\}_{i=1}^n$,

where $L_{n,i}(\omega_n^i) = 1$ and $L_{n,i}(\omega_n^j) = 0$ for $j \neq i$. Given a multi-set S, its vanishing polynomial $z_S(X)$ is defined as $z_S(X) = \prod_{s \in S}(X - s)$. It is known that $z_{\mathbb{H}_{N_n}}(X) = X^n - 1$. Given $s \in \mathbb{F}$, its Pedersen commitment is $[s]_1 + r[h]_1$ for a random r, where h is unknown to the prover.

Given $\mathbf{t} \in \mathbb{F}^n$, its encoding polynomial $t(X)$ has degree $n-1$ and it is defined as: $t(X) = \sum_{i=1}^n \mathbf{t}_i L_{n,i}(X)$. We use the univariate zk-VPD scheme [98] to provide zk-vector commitment. Sample $r_t \in \mathbb{F}$, and the masked polynomial for $\mathbf{t}$ is defined as: $\hat{t}(X) = t(X) + r_t z_{\mathbb{H}_{N_n}}(X)$, and the vector commitment to $\mathbf{t}$ is: $\mathsf{C_t} = [\hat{t}(s)]_1$, where s is the trapdoor of a KZG commitment key [60]. Given a public point z, using the univariate port of the zk-VPD scheme, the prover can convince the verifier that a Pedersen commitment C_y hides a value $y = t(z)$ without disclosing y. Note that $\hat{t}(X)$ is also a 1-leaky masked polynomial of $t(X)$ [40, Section 3.6] (also in [30, 49]), in the sense that running standard KZG opening proof for one evaluation of $\hat{t}(X)$ still keeps $\mathbf{t}$ zero knowledge. 2-leaky masking is similarly achieved by defining $\hat{t}(X)$ as $\hat{t}(X) = t(X) + (r_{t_1} X + r_{t_0}) z_{\mathbb{H}_{N_n}}(X)$. In summary, for any vector $\mathbf{u} \in \mathbb{F}^n$, we use $u(X)$, $\hat{u}(X)$, and $\mathsf{C_u}$ to denote its encoding polynomial, masked polynomial, and vector commitment.

We use the notation from [28] to specify zk-protocols. Consider Schnorr's DLOG as an example: $\pi_{\mathsf{DLOG}}(\mathbf{h})\{(x) : \mathbf{h} = [x]_1\}$. Here `DLOG` in π_{DLOG} is a mnemonic. $(\mathbf{h})$ is the public information. The tuple before ":" is the secret known by prover only, i.e., (x). Then the statement inside curly braces states the relation: the prover knows the secret discrete logarithm of $\mathbf{h}$.

3.2 Commitments

We use a variety of commitments in this paper. We first provide a formal definition.

Definition 3.1. Let $\mathbf{v} \in \mathbb{F}^n$ be a vector of field elements. A commitment scheme `com` is a set of algorithms $(\texttt{com.setup}, \texttt{com.prove}, \texttt{com.verify})$, where:

(1) $(\mathsf{pk}, \mathsf{vk}) \leftarrow \texttt{com.setup}(1^\lambda)$. On input security parameter λ generates prover and verifier keys $(\mathsf{pk}, \mathsf{vk})$.
(2) $\texttt{com} \leftarrow \texttt{com.commit}(\mathsf{pk}, \mathbf{v} \in \mathbb{F}^n, r)$. Generates a succinct commitment to vectof $\mathbf{v}$, where r is a hiding factor.
(3) $\pi \leftarrow \texttt{com.prove}(\mathsf{pk}, \mathbf{v}, r, y)$. Generates a proof π which shows that (polynomial built from) $\mathbf{v}$ evaluates to y at r, or $\mathbf{v}[r] = y$ depeding on the context.
(4) $1/0 \leftarrow \texttt{com.verify}(\mathsf{vk}, \texttt{com}, \pi, r, y)$. Verify the above proof.

We require `com` satisfies properties such as *complete hiding*: one cannot infer any information about the vector elements from a commitment, *computational binding*: it is computationally hard to find $\mathbf{v}' \neq \mathbf{v}$ s.t. $\texttt{com.commit}(\mathbf{v}) = \texttt{com.commit}(\mathbf{v}')$.

In this paper, we consider three forms of vector commitments: Pedersen vector commitment (`pedcom`), KZG commitment (`kzgcom`), and Lagrange basis vector commitment (`veccom`). Given a vector $\mathbf{a}$, we write its KZG commitment as $\bar{\mathbf{a}}^{(\mathsf{KZG})}$. Similarly its Pedersen vector commitment is denoted as $\bar{\mathbf{a}}^{(\mathsf{PED})}$ and its Lagrange basis vector commitment $\bar{\mathbf{a}}^{(\mathsf{VEC})}$. We note that using the Quasi-linear NIZK (QA-NIZK) technique [54], as shown in Lego-Snark [34], one can prove the equivanece of these commitments encoding the same vector. The default commitment scheme (as used in Nova/SuperNova) is the Pedersen Vector Commitment. By default $\bar{\mathbf{a}}$ denotes $\bar{\mathbf{a}}^{(\mathsf{PED})}$, if otherwise specified.

3.2.1 *Adding Zero Knowledge to KZG.* Bounded Leaky-Zk

We first discuss the "leaking-zk". The idea is to mask a polynomial with a random poly. It is formally defined as bounded-zk in Lunar [30], but was unofficially used in PlonK [49][7] and then summarized in [40] (section adding zero knowledge). We formally defined it below.

LEMMA 3.2. *Let $\mathbf{f} \in \mathbb{F}^n$, and let $f(X) = \sum_{i=1}^n \mathbf{f}_i \cdot l_i(X)$. Given a fixed $b > 0$, we say that $\hat{f}$ is a b-*bounded-zk extension *of $f(X)$ in the sense that there exists a b-1 degree random polynomial $R(X)$ s.t.*

$$\hat{f}(X) = f(X) + R(X) z_{\mathbb{H}_N}(X)$$

$\hat{f}$ satisfies the following properties:

(1) *For $i \in [n] : \hat{f}(\omega^i) = \mathbf{f}_i$.*
(2) *Given b evaluations of $\hat{f}$ at random points, for any random vector $\mathbf{g} \in \mathbb{F}^n$, there exists a b-1 degree poly U s.t. $\hat{f}(X) = g(X) + U(X) z_{\mathbb{H}_N}(X)$.*

Property (2) essentially states that as long as there are no more than b evaluations, $\hat{f}$ can be generated from any arbitrary vector, thus leaking no information about $\mathbf{f}$. Note that for the 1-leaky case, $R(X)$ is degree 0.

3.2.2 *Complete-Zk KZG (NOT USED TO DEL).*

An alternative way is to applying Pedersen commitment style blinding factor and use Schonorr style zk-protocol to cancel out the blinding factor. This is done in [90] and our own work `zkregex`. The `zkregex` construction is an improved univariate variation of the ZK-VPD construction in [98].

This approach guarantees full zk at a small $O(1)$ cost in proof size and verifier cost. We officially call it $\Sigma_{\mathsf{zk\text{-}kzg}}$ which defines the following operations:

(1) $\mathsf{srs} \leftarrow \mathcal{G}(N, \lambda)$: the trusted set up generates the CRS, which essentially are KZG prover key $\left(([s^i]_1)_{i=1}^N, [z_{\mathbb{H}_n}(s)]_1, [z_{\mathbb{H}_N}(s)]_1\right.$
(2) $\mathsf{C}_f \leftarrow \texttt{Commit}(f, r, \mathsf{srs})$: we have two cases: (1) $f \in \mathbb{F}_{<N}[x]$: generate compute $\mathsf{C}_f = [f(s)]_1 + r_f[z_{\mathbb{H}_n}(s)]_1$; (2) $f \in \mathbb{F}_{<n}[x]$: compute $\mathsf{C}_f = [f(s)]_1 + r_f[z_{\mathbb{H}_N}(s)]_1$.
(3) $(\mathsf{C}_v, \pi) \leftarrow \texttt{Prove}(f, r_f, t, v, r_v, r_\pi, \mathsf{srs})$: generate the valuation proof that f evaluates to some secret value v at point t and C_v commits to v. r_f, r_v, r_π are the blinding openings for the Pedersen commitments to f, v, π respectively.
(4) $1/0 \leftarrow \texttt{Verify}(\mathsf{C}_f, t, \mathsf{C}_v, \pi, \mathsf{srs})$: verify the evaluation proof is valid.

[7] NOTE: It was mentioned in Lunar's introduction in bounded-leaky zk, however, didn't find where it is in [49]. **CHECK LATER.**

We recall the details of $\Sigma_{\text{zk-kzg}}$ in **zkregex**. $\mathbf{P}$ computes $q(X) = (f(X) - v)/(X - t)$ and $\mathsf{C}_q = [q(s)]_1 + r_q[z_{\mathbb{H}_n}(s)]_1$ for random r_q. $\mathbf{P}$ also computes a balancing term B:

$$B = (r_f - r_v)e([1]_1, [z_{\mathbb{H}_n}(s)]_2) - r_q e([z_{\mathbb{H}_n}(s)]_1, [s-t]_2)$$

the proof is tuple $(\mathsf{C}_q, B, \pi_{\mathsf{DLOG}}(B, (e([1]_1, [z_{\mathbb{H}_n}(s)]_2), e([1]_1, [s-t]_2))$ s.t.

$$e(\mathsf{C}_f - \mathsf{C}_v, [1]_2) = e(\mathsf{C}_q, [s-t]_2) + B$$

$\mathbf{V}$ performs the pairing check and the π_{DLOG} check. Note that because C_v is used, the construction is complete-zk.

Remark: we specifically note that $\mathsf{C}_\mathbf{T}$ can be regarded as a standard KZG commitment to $\hat{T}(X)$, as well as a $\Sigma_{\text{zk-kzg}}$ commitment to $T(X)$ at the same time.

4 BASIC CRYPOGRAPHIC CONSTRUCTIONS

In this section we list some basic constructions that we need. Technical details can be found in Appendix.

Given a vector $\mathsf{C}_\mathbf{v}$, and a Pedersen commitment C_u, $\pi_{\mathtt{sum}}(\mathsf{C}_\mathbf{v}, r, \mathsf{C}_u)$ proves that $u = \sum_{i=1}^{N} v_i r^i$. $\pi_{\mathtt{prod}}(\mathsf{C}_\mathbf{u}, \mathsf{C}_\mathbf{v}, \mathsf{C}_\mathbf{w})$ proves that for each i: $\mathbf{w}_i = \mathbf{u}_i \times \mathbf{v}_i$.

$\pi_{\mathtt{perm}}(\mathsf{C}_\mathbf{u}, \mathsf{C}_\mathbf{v})$ to prove that $\mathbf{v}$ is a permutation of $\mathbf{u}$. This can leverage the grand-product used in Plonk system (details later).

$\pi_{\mathtt{set}}(\mathsf{C}_\mathbf{u}, \mathsf{C}_\mathbf{s})$: to prove that vector $\vec{s}$ encodes the the standard set of elements from $\mathbf{u}$. The basic idea is simple: first produce a vector v which is a sorted permutation, run $\pi_{\mathtt{perm}}$ first, and then prove $\mathbf{v}$ is sorted, and then run $\pi_{\mathtt{pack}}$ to produce $\mathbf{s}$.

4.1 Lookup Argument

Let $\mathbf{t} \in \mathbb{F}^n$ and $\mathbf{T} \in \mathbb{F}^N$, a lookup argument for $\mathbf{t} \hat{\subset} \mathbf{T}$ asserts that for each $i \in [n]$: $\mathbf{t}_i \in \mathbf{T}$. Note that both $\mathbf{t}$ and $\mathbf{T}$ may contain duplicates. We need a homomorphic and zero knowledge look-up argument, and a slight zk-enhancement of `cq` [42] satisfies our needs. The basic idea of `cq` is to pre-compute commitments of quotient polynomials in $O(N \log(N))$ time. Then for arbitrary query table $\mathbf{t}$ of size n, the prover work is $O(n \log(n))$, thus independent of N. We need to enhance `cq` so that it is zero knowledge. In our extension, we exploit bounded leaky-zk as in [38, 40], and Schnorr style Σ-protocols as in [90]. We will present its full technical details in the extended version of this paper. Similar constructions for the zk-enhancement of `cq` can be found in two concurrent work: segment lookup [40] and matrix lookup [29]. We denote it as $\pi_{\text{zk\_cq}}$, as shown below:

(1) $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}\ (N, \lambda)$: a trusted set-up given security parameter λ and lookup table size limit N, samples bilinear groups and generates the prover and verifier keys $(\mathsf{pk}, \mathsf{vk})$ for KZG.
(2) $(\mathsf{aux}_\mathbf{T}, \mathsf{C}_\mathbf{T}) \leftarrow \mathtt{Preprocess}(\mathbf{T}, \mathsf{pk})$: Given the lookup table $\mathbf{T}$, it generates $\mathsf{aux}_\mathbf{T}$ (the preprocessed information) and $\mathsf{C}_\mathbf{T}$ (a commitment to $\mathbf{T}$).
(3) $\pi \leftarrow \mathtt{Prove}(\mathsf{pk}, \mathsf{aux}_\mathbf{T}, \mathbf{T}, \mathbf{t})$: produces a zero knowledge proof π for $\mathbf{t} \hat{\subset} \mathbf{T}$.
(4) $1/0 \leftarrow \mathtt{Verify}(\mathsf{vk}, \mathsf{C}_\mathbf{T}, \mathsf{C}_\mathbf{t}, \pi)$: checks that π is valid.
(5) $\pi \leftarrow \mathtt{FoldProve}(\mathsf{pk}, \mathsf{aux}_\mathbf{U}, \mathbf{U}, \mathsf{aux}_\mathbf{V}, \mathbf{V}, \mathbf{u}, \mathbf{v}, \alpha)$: produces a zero knowledge proof π for $\mathbf{u} + \alpha\mathbf{v} \hat{\subset} \mathbf{U} + \alpha\mathbf{V}$.

THEOREM 4.1. *Under the Q-DLOG assumption [42] in the Algebraic Group Model (AGM) and Random Oracle Model (ROM), $\pi_{\text{ZK\_CQ}}$ is perfectly complete, computational knowledge sound, and zero knowledge.*

The proof generally follows the analysis of `cq` [42], and is similar to that of [29, 40]. We delay the details to the extended version.

Based upon $\pi_{\text{ZK\_CQ}}$, we develop $\pi_{\text{RANGE}}(\mathsf{C}_\mathbf{a}, 2^B)$, a batched zk-range proof which asserts that $\mathsf{C}_\mathbf{a}$ is a Pedersen vector commitment to $\mathbf{a} \in \mathbb{F}^n$, and each $\mathbf{a}_i$ in $\mathbf{a}$ is in range $[0, 2^B)$. It has $O(1)$ proof size and $O(n \log(n))$ prover complexity.

4.2 Commit-and-Prove zkSNARK

In our application scenario, we have specialized Σ-protocol (e.g., lookup arguments), and there are boolean and arithmetic logic which can be more conveniently captured by R1CS and handled by general zk-SNARK systems such as Groth16 [56]. We replicate the Commit-and-Prove scheme in LegoSnark [35], and bind a Groth16 system with the zk-cq lookup argument, formally defined as below.

Definition 4.2. A Commit-and-Prove for Lookup SNARK System (CPL-SNARK) is a collection of algorithms that supports the argument for $S = (R = (\mathbf{io}, \mathbf{w} \cup \{r\}, A, B, C), \vec{f}, \{(\mathbf{v}_i, T_i)\}_{i=0}^{k})$, where R is an R1CS instance. $\vec{f} \subseteq \vec{w}$ is a subset of variables to be committed before r of R is determined. For each i, $\mathbf{v}_i$ is a subset of $\mathbf{w}$.

(1) $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathtt{SetupCPL}(\lambda, S)$.
(2) $\pi \leftarrow \mathtt{ProveCPL}(\mathsf{pk}, S)$. It asserts the hidden $\mathbf{w} \cup \{r\}$ is a valid witness of R, and $r = hash(\vec{f})$, and for each $\mathbf{v}_i$, all of its elements is contained in table T_i.
(3) $0/1 \leftarrow \mathtt{VerifyCPL}(\mathsf{vk}, io)$.

We note that the r is used to perform fingerprinting check of two relations. For example given two vectors $\mathbf{a}$ and $\mathbf{b}$ we could have an arithmetic circuit, which given a random challenge r and test $\prod_{i=1}^{n}(\mathbf{a}_i - r) = \prod_{i=1}^{n}(\mathbf{b}_i - r)$. However, this test is only valid when $\mathbf{a}$ and $\mathbf{b}$ is "fixed" before random challenge r is given as the input to the circuit.

TO BE EXPANDED: Given that there are multiple circuits to be called depending on the first stage, and the number of circuits to be deployed ranging between 0 to 64, we have several alternative approaches: (1) aggregation of proofs (2) folding scheme (with or without IVC). Aggregation has $\log(n)$ proof size and folding provides $O(1)$ proof size, and much faster prover performance (as no need to prove each indidivual instance just fold them with a couple of MSMs). The problem is that IVC based folding usually needs to encode folding verifier circuit, which typically requires a cycle of curves, and for preformance issue (e.g., Pasta curves) do not provide pairing, which is needed for the CQ protocol for large tables. In this case, we mix several ingredients together:

(1) We apply folding to all instances of circuits.
(2) We use commit-and-prove scheme to extract out the input and output of circuits, as well as the query table inside circiut, using QA-NIZK.
(3) We use GIPA to aggregate folding.

This eventually yields a $\log(k)$ (given k is the number of instances) proof size, and only one curve needed, for folding scheme (even IVC).

5 AC-DFA BASED PATTERN SELECTION

We first give an intuitive idea. Given a collection of regex patterns $\{r_i\}_{i=1}^m$, the prover extracts critical patterns for each r_i, and build an Aho-Corasick automaton (AC-DFA) for the critical patterns. Each critical pattern is associated with one or more signatures. Given an input string s, the prover runs s through the AC-DFA, and get a Pedersen vector commitment to all the final states (that capture a critical pattern). The set of states is arranged as a concatenation of final and non-final states, thus to verify if a state is in final is simply a range proof.

Given a fixed set of words $\{w_i \in \Sigma^*\}$, an Aho-Corasick automaton can be built in $O(\sum_{i=1^m} |w_i|)$ time with $O(\sum_{i=1^m} |w_i|)$ size. An AC-DFA is deterministic finite automaton (S, s_0, F, T) where each transition $t \in T$ is a tuple (s_1, c, s_2, b) where b indicates whether it is a failure edge. Given any string $s \in \Sigma^*$, it can be accepted in $2|s|$ time. There exists a map ρ which maps each final state to a collection of words that the AC-DFA outputs at that state. An AC-DFA can be further compiled into a standard DFA with back-edges pre-computed away.

Definition 5.1. Given a regex r, we call a string s its critical pattern, written as $s \in \mathcal{CP}(r)$ if for any string $t \in L(r)$, s is a substring of t.

In another word, if a critical pattern does not appear in a string t then t cannot be a match for r.

In practice, for $38k$ PM-Reg ClamAV rules, we collected $38k$ critical patterns which results in a DFA with 2 million states; and all patterns results in $6million$ states for $112k$ patterns, each state has 16 outgoing transitions for hex alphabet (4-bit nibbles).

We will split the proof into two steps: (1) we prove a Pedersen commitment commits to the set set of states along an acceptance path for an input string; (2) We show another Pedersen commitment commits to the set of signatures that should be verified.

5.1 DFA Acceptance Path

Let $u = \log(|\Sigma|)$ and v be the bit-width of states. Given a transition (s, c, t) a transition can be digitized to a field element: $\rho((s, c, t)) = s + 2^v c + 2^{v+u} t$. Given the set of transitions of an AC-DFA, the prover first pre-process τ as look-up table and generate its prover keys and cached quotient information. The set of states is encoded as a vector of consecutive numbers which is a concatenation of final and non-final states. Thus, to check if a state is final is translated into a zk-range proof.

Given an input string s, its acceptance path can be modeled by four vectors: $\mathbf{s}$, $\mathbf{c}$, $\mathbf{t}$. Define τ for each i s.t. $\tau_i = \rho(\mathbf{s}_i, \mathbf{c}_i, \mathbf{d}_i)$.

Given the vector commitments to these five vectors, one can use range-proofs to prove their relation:

(1) Prove $\mathsf{C}_\tau = \mathsf{C}_\mathbf{s} + 2^v \mathsf{C}_\mathbf{c} + 2^{v+u} \mathsf{C}_\mathbf{t}$.
(2) Prove each element in $\mathsf{C}_\mathbf{s}$, $\mathsf{C}_\mathbf{t}$, $\mathsf{C}_\mathbf{d}$ are in range, using batched zk-range proof.
(3) Prove that $\mathbf{s}_0$ is the initial state.

Note that given that this is a standard DFA, $\mathbf{c}$ is actually the input string $\mathbf{s}$, as backedges are computed away. Let $\vec{F}$ be the final states appears in the acceptance path, we then need to show that another Pedersen commitment $\mathsf{C}_{\vec{F}}$ is valid. This is done through the general strategy of CPL-SNARK via fingerprinting techque. We first define some gadgets below.

(1) $\texttt{permutation}(\vec{a}, \vec{b}, r)$. This is achieved by fingerprinting test $\prod_{i=1}^n (\vec{a}_i - r) = \prod_{i=1}^n (\vec{b}_i - r)$. Note that $\vec{a}$ and $\vec{b}$ needs to be committed first before random challenge r is generated to apply Fiat-Shamir.
(2) $\texttt{pack}(\vec{a}, \vec{b}, \vec{c}, r)$. Given $\vec{b}$ is a boolean array, $\mathbf{c}$ is a packed representation of $\vec{a}$, padded by 0. For instance $\vec{a} = (100, 200, 300, 400)$, and $\vec{b} = (1, 0, 1, 0)$, and $\vec{c} = (100, 300)$. This is also accomplished by introducing an array r_i s.t. $r_i = r_{i-1} * r$ iff $b_i = 1$. Then verify $\prod_{i=1}^n \vec{a}_i r_i = \prod_{i=1}^n \vec{c}_i r^i$.
(3) $\texttt{set}(\vec{a}, \vec{b}, r)$, to test $\vec{b}$ is a standard set of $\vec{a}$. The idea is to first produce a sorted list of $\vec{a}$, letting it be $\vec{c}$. Then define a boolean vector $\vec{d}$ s.t. $\vec{d}_i = \vec{c}_i == \vec{c}_{i-1}$. Then run `pack` to remove the duplicates in $\vec{c}$, then $\vec{b}$ is a sorted standard set of $\vec{a}$ with the tail padded with 0.

In summary, we have the construction $\pi_{\texttt{accpath}}(\mathsf{C}_\mathbf{s}, \mathsf{C}_\mathbf{f})$ which asserts that for string s, $\mathsf{C}_\mathbf{f}$ is a vector commitment of the standard set of all final states along its unique acceptance path over the AC-DFA.

Design Idea: since we are using fingerprint checking, we need verifier challenge. So the prover system needs to run in two stages (e.g., Halo2 already natively supports it). Some complexities will arise when we need to fold relations, the verifier challenge needs to be generated once and moved out of IVC prover. We achieve this by aggregating/folding k relations at a time, and let verifier challenge be hash of all first stage witness wires for all instances.

Each of the relation could be modeled as a Halo2 gadget or chip (or rely on Halo2 to optimize the use of selectors), but maybe it's good to have a global cusotmized layouter to layout these relations to a standard Plonkish model (which is later easier to do ProtoGalaxy translation). We discuss the translation below:

$\texttt{permutation}(\vec{a}, \vec{b}, r)$ relation can be encoded with one custom gate with 3 columns, where an extra column serves as the grand product. The r ideally should be referenced as a fixed cell, but it looks not possible in Halo2, and may need an extra column. There might be an extra selector which

enforces the gate. Given there are so many instances, we may create our own numbered selector gate to enforce it.

$\texttt{pack}(\vec{a}, \vec{b}, \vec{c})$ can be implemented similarly. $\texttt{set}(\vec{a}, \vec{b})$ is done in a similar fasion and it needs support of range proof.

Eventually, the finite state automata acceptance logic can be implemented by "invoking" the above logic. However, to save the number of rows in the constraint system, it might be most economical to just encode all constraints together instead of doing multiple chips and copy over the values.

5.2 Selection from Signatures

Now we have the set of accepstating states for critical patterns, and then we need to link them to the set of signatures that need to be inspected.

The trusted set up prepare a public table which maps from each acceptating state f_i to a vector of signature IDs. However these signature IDs needs to be represented compactly and we can represent them using bilinear pairing of $[\prod_{j=1}^{k}(x - s_i)]_1$ for some trap-door x where $\{s_{i,j}\}$ are the related signatures to f_i. This table of information is publically available.

Now the set of related signatures **SIG** can be similarlarly represented as a bilinear pairing commitment $\mathsf{C}_{\mathbf{SIG}}$. To prove that for state f_i's signature is a subset of **SIG**, one can compute the quotient polynomial between the two characteristic polynomials and supply it over G_2.

Then, we can use TIPP to batch prove this relation: given a list of f_i and the corresponding proof π_i. All of them are hiding in TIPP's commitment to vector of group elements. Then we use argument for inner pairing to prove the pairing relation.

Thus, we have the construction for proving $\mathsf{C}_{\mathbf{SIG}}$ represents a *superset* of the signatures related to $\mathsf{C}_{\mathbf{f}}$. Note that the prover may provide a super-set, but it only increases the cost of the prover, and not hurting the soundness of the proof.

5.3 Selection from Signatures (Circuit Version)

We have a set of final states of identifying patterns (average size 6.6k). We have 38k signatures, and the average number of pattens is up to 20. So we are running concurrently 38k lookups (mostly negative) and up to 50 positive. For negative lookups, we perform $2x$ lookups (using the trick of $a < x < b$).

In this case our lookup table is 6.6k, and our query table size is 38k $\times 20 = 760$k. Instead of using CQ, we should use plookup based (and range proof).

Again, given the boolean array indicates selected or not, we can get a small packed factor which indicates which PM-REG signature should be ruled out.

6 CHUNKING DISCHARGING ALGORITHM

In practice, there are large files that have to be split into multiple chunks. In this sense, we have to develop "chunking" version[8] for the regular expression discharging algorithms. We present an intermediate layer of supporting relational database operations, based on which, the chunking algorithms are developed. The relational database operators are bulit upon internal and external lookup arguments. We use the technical details in this section to informally motivate the features that we need in the underlying proof system.

Consider a word s which is chunked into: $s = s^{(1)}||...||s^{(k)}$. The following outlines the general workflow of discharging s against a collection of regex patterns.

(1) The prover first compiles the (AC)-DFA for CP, SED and DFA, respectively. It then runs a planning algorithm to determine the set of signatures to be discharged by each of CP, SED, and DFA. So that later it can show the union of the signatures discharged by SED and DFA is equal to the ones failed by CP. Each algorithm is modeled as a separate gadget circuit, and *conditionally* include them only when they are needed. For instance, for most binary samples, only CP and SED are included, and very rarely DFA would be needed. Also sub-circuits may have verisons with different capacities (e.g., SED circuits with different size of step-queues). This needs the underlying folding/IVC framework to support non-uniform circuits.

(2) At each step of folding, the circuit drives CP, SED, and DFA concurrently. The design for chunking DFA or CP is straigh-forward. Essentially we run a chunk $s^{(j)}$ over the (AC-)DFA. Then we need to pass the last state of the acceptance path as the initial state for the next folding step. For SED algorithm, streaming is much more complex. We maintain the state of the "step queue" for each subsignature, i.e., the list of occurene locations for each step of each step queue. We need to perform forward and backward pruning at each folding stage to minimize the size of the step queue, and between steps, we need to pass the output "step queue" to the next stage. This step queue is huge, and it can only used for one time. *We need techniques more efficient than the general purpose RAM used in zk-VMs for concrete peroformance.* The discharging algorithm needs to check validity of an encoding of DFA transitions. This is accomplished by pre-processed lookup arguments (as DFA's are fixed before proof). It also performs run-time database operations (e.g., table join and queries) for step-queues, this is accomplished by internal lookups in circuits. *This requires the underlying IVC framework to support both internal and external lookup arguments.*

(3) At the last chunk, there is a tabulating circuit which computes the tri-logic evaluation of each subsignature. This is complicated by various cases that subsignatures have counting constraints, and their logical structure. The circuit performs tri-logic operators to

[8] In the extreme case that each chunk is only one character, it is the streaming algorithm.

compose all results, and produce the dishcarged list for SED and DFA, and compare with the "failed-discharged" list by CP. When they are equal, this discharges s of the given regex set.

6.1 Motivating Example of SED Discharging

We start with a motivating example to introduce the chunked SED algorithm. Recall that SED tries to discharge a string s against a regex subsignature r by checking the distance between patterns of r. In Figure 5, we show an example of regex which consists of four patterns ("fgh", "1234", "56", and "78"), and the corresponding pattern information. Each pattern has an expected range of distance from the match of a previous pattern. For instance, the distance between *the end of* "1234" and *the end of* "56" should be 16 hex-nibbles (i.e., 8 characters, because the length of "56" is also counted).

Chunked SED has a pre-compiled AC-DFA that corresponds to all these patterns. Before running the forward-backward pruning algorithm, SED feeds the given string to the ACDFA, gets the list of final states, and then map these final states back to the corresponding pattern words. As an example, the word "fghxx1234..." in Figure 5 generates the "PatLoc" table as shown. For instance, one can see that pattern 2 occurs at locations (9, 13, 43, 53). Note that the first location starts from 1, and each location accounts for one 4-bit nibble.

Then Chunked SED maintains a *step queue* that records the list of locations that correspond to each pattern (step) in the regex subsignature. For instance, given the regex signature and input word in Figure 5, starting with an initial input queue (where location 1 for step 0 is included by default), it results in the output step queue as shown in the figure.

To make the prover more eficient, we run an additional backward elimination process that removes "dead" locations from the step queue at run time. Consider the following example.

Example 6.1. In Figure 5, The forward-propgation process first iteratively starts from each location of each step, identifies the applicable locations for the next step. For instance, given location 7 for step 1 (pattern 7), four locations of the next pattern (2) falls into the range (within min-max distance (16,16): 19, 31, 43, 53. Notice that the algorithm has to propagate based on the "on-going" result. For step 3, it has to start from the newly generated locations (19, 31, 43, 53), and identify the workable locations 59 and 69 for step 3 and so on.

Once the forward process is done, we then run a backward pruning process that removes "dead" locations. Consider step 4, its current minimum location is 79. Given that the distance between step 3 and 4 is 10, this eliminates 59 in step 3, because for new locations of step 4 added in the future, they can only be greater than 79 and for sure there will be no new locations that can be derived from 59. Thus 59 should be removed from the location list of step 3. Similalry, the new "min-loc" for step 3 now becomes 69, and this eliminates 19, 31, 43 from step 2. The list to delete is shown in "ToDel Queue" in Figure 5, which results in the final Output Queue.

For forward propgation, the prover provides its "transcript" of reasoning, i.e., how it derives the locations for the next step, by running a range-query join operation on tables.

Example 6.2. Consider the source location 43 of source step 2 in Figure 5, in the *Forward Proof*. We need to retrieve the locations for pattern 3 from Pat-Loc table which satisfies range-query $[43+16, 43+16]$.

The result are: $(3,0,0), (3,1,59), (3,2,69)$. Here, only the middle entry is the valid result, and $(3,0,0)$ and $(3,2,69)$ are the wrapping entries for validity.

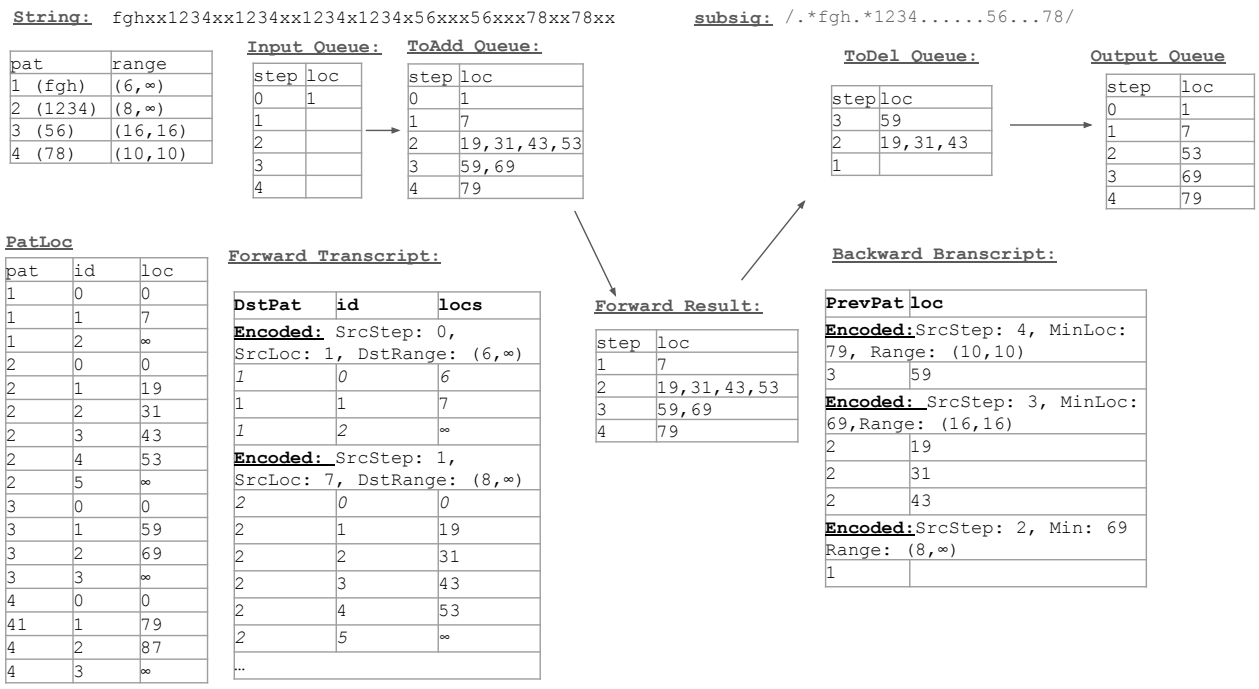
Why would verifiers be convinced that this is exactly the correct result of range query? This is based on the following reasoning. First, $0 < 43+16$ and $69 > 43+16$. Second, the pat-loc table earlier is proved to be sorted. Third, id columns are consecutive, and it prevents a malicious prover to skip valid result. Later, we show that such range queries can be done very efficiently using lookup arguments.

The verifier verifies the validity of forward reason transcript, in addition, it performs the following checks: (1) the resulting step queue is the union of the input and to add; (2) projecting the forward proof at each step (destination) results in the result queue; and (3). projecting the src location of forward proof results in the input step queue. One can see the backward proof (and its validity) works in a similar way.

The algorithm has several interesting properties and challenges. We need to reason about typical relational database operations for structures generated at "run time". They have rather diverse sizes, even for the same type. For instance, a subsig can have very different step-queue size, given different files. A pre-set length could waste many constraints and resources. To handle the flexible length problem, we use a technique similar to RAM modeling (like using a transcript to memory read/write operations), to accomodate data structures of diffrenet length. We now present the details in the next subsection.

6.2 Modeling Relational Operators using Lookup Arguments

Proving database out-sourcing using zk-SNARK systems has been explored in IntegriDB [100] (using pairing based dynamic accumuator and interval trees) and vSQL [99] (using sum-checked based CMT protocols). Compared with these earlier work, in the context of our work, the "databaes" is dynamic and they are generated in circuit (instead of a "fixed" updatable database that exist prior to proof starts). our approach is much simpler by out-sourcing operations to lookup argument, and the number comparison operations can be simply done using lookup (instead of bit-decomposition). Our approach works for read-only query only. It is used as an intermediate layer to support streaming algorithms, which can be built upon classical relational algebra operators: Π (projection), δ (select), $\times$ (cross product), and $\bowtie$ (join), we refer interested readers to [3].



pat	range
1 (fgh)	(6,∞)
2 (1234)	(8,∞)
3 (56)	(16,16)
4 (78)	(10,10)

step	loc
0	1
1	
2	
3	
4	

step	loc
0	1
1	7
2	19,31,43,53
3	59,69
4	79

step	loc
3	59
2	19,31,43
1	

step	loc
0	1
1	7
2	53
3	69
4	79

pat	id	loc
1	0	0
1	1	7
1	2	∞
2	0	0
2	1	19
2	2	31
2	3	43
2	4	53
2	5	∞
3	0	0
3	1	59
3	2	69
3	3	∞
4	0	0
41	1	79
4	2	87
4	3	∞

DstPat	id	locs
Encoded: SrcStep: 0, SrcLoc: 1, DstRange: (6,∞)		
1	*0*	*6*
1	1	7
1	*2*	∞
Encoded: SrcStep: 1, SrcLoc: 7, DstRange: (8,∞)		
2	*0*	*0*
2	1	19
2	2	31
2	3	43
2	4	53
2	*5*	∞
...		

step	loc
1	7
2	19,31,43,53
3	59,69
4	79

PrevPat	loc
Encoded:SrcStep: 4, MinLoc: 79, Range: (10,10)	
3	59
Encoded: SrcStep: 3, MinLoc: 69,Range: (16,16)	
2	19
2	31
2	43
Encoded:SrcStep: 2, Min: 69 Range: (8,∞)	
1	

Figure 5: Streaming Workflow for Foward and Backward Pruning

A relation (table) in a relational database can be viewed as a two-dimensional table, where each row is a fixed-arity tuple, or it can be viewed as a collection of columns. We often write a table $\mathbf{t} = (\mathbf{c}^{(1)}, \mathbf{c}^{(2)}, ..., \mathbf{c}^{(k)})$, where $\mathbf{c}^{(i)}$ is the i'th column. In Figure 6 we show how to use lookup argument to support relational operators.

The support of Π and δ is straight-forward. The trickier one is the support of join operators, it heavily relies on structuring a relation table in a *well-formed* format. Given a table $\mathbf{t} = (\mathbf{key}, \mathbf{id}, \mathbf{val})$, where all "effective" values in **val** are proved earlier to be in range $(0, \texttt{max})$ (via lookup argument). we say $\mathbf{t}$ is well formed if its **key** is sorted, and all of its **val** for the same key are grouped together, and wrapped around with a 0 and `max` dummy entries. We use **id**, which sequentially increaese by 1, to label the values for a same key.

Well-formedness allows us to reason about join operations, e.g., $\mathbf{t}^{(1)} \bowtie_{3=1} \mathbf{t}^{(2)}$, where the $\mathbf{val}^{(1)}$ in $\mathbf{t}^{(1)}$ serves as the foreign key. Natural join is essentially a cross-product followed by a selection. The beautify of zk-proof is that we can directly present the result and prove its validity without having to generate a (usually much larger cross-product) first. Here the key is to show that for each entry in $\mathbf{t}^{(1)}$, we retrieve the corresponding set of entries for the foreign key from $\mathbf{t}^{(2)}$, i.e., one entry is expanded correctly into a set of entries. The proof needs to show that this expansion does not miss real entries or add fake entries, which is captured by look-up argument (see lines (5) and (6)) and the wellformedness (see line (3)). The case of handling where a foreign key value in $\mathbf{t}^{(1)}$ does not exist in $\mathbf{t}^{(2)}$ is handled similarly using multi-set difference, also via lookup-argument (which we omit the details here).

In Figure 7 we present the forward-propgation and backward pruning algorithm for SED. In essence, the data structures involved can be represented as relational database tables, and the proof can be decomposed into relational operations. As shown in Figure 7, the inputs of the algorithm consists of the following data structures:

(1) $\mathbf{S}$ is the set of subsigs IDs.
(2) $\mathbf{R}$ is the pat-loc table, which consits of two columns of pattern IDs and their locations in ACDFA acceptance path.
(3) $\mathbf{Q_i}$ is the input step queue which shows a locations for a pattern at a particular step for a subsig. It stores compactly as a two column table: $(\texttt{key}, \texttt{loc})$, where `key` encodes related information, i.e., $\texttt{key} = \texttt{encode}(\texttt{subsig}, \texttt{step}, \texttt{rg1}, \texttt{rg2})$. We assume all subsequent step queue structures follow the same data scheme.
(4) $\mathbf{\Phi}$ is a collection of relations, e.g., $\mathbf{\Phi}_{\texttt{key,prev\_pat}}$ maps from `key` to its previous pattern id, and we similarly use subscripts to indicate relation columns. By default $\bowtie$ applies to the `key` columns of relations.

The output of 2-phase SED algorithm Figure 7 consists of the following:

1 $\mathbf{a} = \Pi_i(\mathbf{t})$
Let $\mathbf{t} = (\mathbf{c}^{(1)}, ..., \mathbf{c}^{(k)})$, just assert $\mathbf{a} = \mathbf{c}^{(i)}$.

2 $\mathbf{t}' = \delta_c(\mathbf{t})$
(1) Let $\mathbf{t} = (\mathbf{u}^{(1)}, ..., \mathbf{u}^{(k)})$, and $\mathbf{t}' = (\mathbf{v}^{(1)}, ..., \mathbf{v}^{(k)})$,
(2) Prover and verifier jointly computes $\mathbf{c}$ s.t. for any $i \in [|\mathbf{t}|]$: $\mathbf{c}_i = c(\mathbf{u}_i^{(1)}, ..., \mathbf{u}_i^{(k)})$.
(3) Verifier sends challenge r.
(4) Prover and verifier jointly computes $\mathbf{a} = \sum_{i=1}^{k} r^i \cdot \mathbf{u}^{(i)}$, and $\mathbf{b} = (\sum_{i=1}^{k} r^i \cdot \mathbf{u}^{(i)}) \cdot \mathbf{c}$ (where $\mathbf{c}$ serves as a selector).
(5) Prover and verifier runs lookup argument (ignoring 0 entries $\mathbf{a} \hat{\subset} \mathbf{b}$ and $\mathbf{b} \hat{\subset} \mathbf{a}$.

3 $\texttt{wellformed}(\mathbf{t})$ where $\mathbf{t} = (\mathbf{key}, \mathbf{id}, \mathbf{val})$
Assumption: all effective values in range $(0, \texttt{max})$. Define $n = |\mathbf{key}|$.
(1) $\mathbf{key}$ is sorted, i.e., $\forall i \in [n-1]$: $\mathbf{key}_i \leq \mathbf{key}_{i+1}$.
(2) $\mathbf{val}[i] = 0$ if $i = 1$ or $\mathbf{key}[i] \neq \mathbf{key}[i-1]$.
(3) $\mathbf{val}[i] = \texttt{max}$ if $i = n$ or $\mathbf{key}[i+1] \neq \mathbf{key}[i]$.
(4) $\mathbf{id}[i+1] = \mathbf{id}[i] + 1$ if $\mathbf{key}[i+1] = \mathbf{key}[i]$.
Comment: it can be expanded to table with multi-column keys.

4 $\mathbf{t}^{(3)} = \mathbf{t}^{(1)} \bowtie_{3=1} \mathbf{t}^{(2)}$ where $\mathbf{t}^{(i)} = (\mathbf{key}^{(i)}, \mathbf{id}^{(i)}, \mathbf{val}^{(i)})$ for $i \in [2]$ and $\mathbf{t}^{(3)} = (\mathbf{key}^{(3)}, \mathbf{id}^{(3)}, \mathbf{val}^{(3)}, \mathbf{id}^{(4)}, \mathbf{val}^{(5)})$.
(1) Assert $\texttt{wellformed}(\mathbf{t}^{(i)})$ for $i \in [2]$.
(2) Compute $\Delta = \mathbf{t}^{(1)} - \mathbf{t}^{(3)}$ as difference of multi-set. Define $(\mathbf{key}^{(6)}, \mathbf{id}^{(6)}, \mathbf{val}^{(6)}) = ((x, 0, 0), (x, 1, \texttt{max}))_{x \in \Delta}$.
(3) Assert $(\mathbf{val}^{(3)}, \mathbf{id}^{(4)}, \mathbf{val}^{(5)})$ is $\texttt{wellformed}$.
(4) Assert $(\mathbf{key}^{(3)}, \mathbf{val}^{(3)}) \hat{\subset} (\mathbf{key}^{(1)}, \mathbf{val}^{(1)})$
(5) Assert $(\mathbf{key}^{(1)}, \mathbf{val}^{(1)}) \hat{\subset} (\mathbf{key}^{(3)}, \mathbf{val}^{(3)})$.
(6) Assert $(\mathbf{val}^{(3)}, \mathbf{id}^{(4)}, \mathbf{val}^{(5)}) \hat{\subset} (\mathbf{key}^{(2)}||\mathbf{key}^{(6)}, \mathbf{id}^{(2)}||\mathbf{id}^{(6)}, \mathbf{val}^{(2)}||\mathbf{val}^{(6)})$

Figure 6: Relational Operators Based on Lookup Argument

(1) Forward-propgation phase: $(\mathbf{Q}_\texttt{a}, \mathbf{Q}_\texttt{f}, \mathcal{T}_\texttt{f})$, $\mathbf{Q}_\texttt{a}$ is the step-queue to add as the result of propagation. $\mathbf{Q}_\texttt{f}$ is the resulting of merging $\mathbf{Q}_\texttt{a}$ with $\mathbf{Q}_\texttt{i}$. $\mathcal{T}_\texttt{f}$ represents the forward transcript which documents the step by step reasoning from each pattern location to idenitfy the next available location to add. Each tuple $\mathcal{T}_\texttt{f}$ is $(\texttt{key}, \texttt{srcloc}, \texttt{dstloc})$, where $\texttt{key}$ encodes all needed information such as pattern ID and range bounds.
(2) Bacward-pruning phase $(\mathbf{Q}_\texttt{d}, \mathbf{Q}_\texttt{o}, \mathcal{T}_\texttt{b})$: Similarly $\mathbf{Q}_\texttt{d}$ represents the step-queue items to delete, and $\mathbf{Q}_\texttt{o}$ is the final output step queue, which results from deleting $\mathbf{Q}_\texttt{d}$ from $\mathbf{Q}_\texttt{f}$. Transcript $\mathcal{T}_\texttt{b}$ denotes all inferrence steps that leads to the pruning actions.

We specify the $\texttt{two\_pass\_sed}(.)$ function in declarative semantics. One can see that they can be very conveniently modeled using relational operators. For instance, in the forward propagation stage, the critical step is (3) which models $\mathcal{T}_\texttt{f}$ as a sequence of join operations. It first queries $\mathbf{\Phi}$ from the encoded $\texttt{key}$ column to retrieve related information such as the next step and the required range. Then it performs a range-query over the pattern-loc table to generate the result.

1 $((\mathbf{Q}_\texttt{a}, \mathbf{Q}_\texttt{f}\mathcal{T}_\texttt{f}), (\mathbf{Q}_\texttt{d}, \mathbf{Q}_\texttt{o}, \mathcal{T}_\texttt{b})) \leftarrow \texttt{two\_pass\_sed}(\mathbf{Q}_\texttt{i}, \mathbf{S}, \mathbf{R}, \mathbf{\Phi})$:
(1) For each subsignature and for each step propgate step by step and generating $(\mathbf{Q}_\texttt{a}, \mathbf{Q}_\texttt{r}, \mathcal{T}_\texttt{f})$ which satisfies the following:
  (1) $\mathbf{Q}_\texttt{r} = \mathbf{Q}_\texttt{i} \cup \mathbf{Q}_\texttt{a}$.
  (2) $\mathbf{Q}_\texttt{a} = \Pi_{1,3}(\mathcal{T}_\texttt{f})$ (i.e. casting column 1 and 3)
  (3) $\mathcal{T}_\texttt{f} = (\mathbf{Q}_\texttt{s} \bowtie \mathbf{\Phi}_{\texttt{key,nxt\_pat}} \bowtie \mathbf{\Phi}_{\texttt{key,nxt\_rg1,nxt\_rg2}}) \bowtie_c \mathbf{P}$ where the range query condition c is: $\mathbf{P}.\texttt{loc} - \texttt{loc} \in [\texttt{nxt\_rg1}, \texttt{nxt\_rg2}]$.
(2) Given $\mathbf{Q}_\texttt{s}$, retrieve minimum location of each step and propgate backward to eliminate. Generate the supporting data structures $\mathbf{Q}_\texttt{d}, \mathbf{Q}_\texttt{o}, \mathcal{T}_\texttt{b}$ as the following:
  (1) $\mathbf{Q}_\texttt{r} = \mathbf{Q}_\texttt{d} \cup \mathbf{Q}_\texttt{o}$.
  (2) $\mathbf{Q}_\texttt{d} = \Pi_{1,3}(\mathcal{T}_\texttt{b})$.
  (3) First assert that $\mathbf{Q}_\texttt{o}$ is sorted by $\texttt{key}$ (which implies that it's sorted on $(\texttt{subsig}, \texttt{step})$. Define $\mathbf{min}$ as a column same height of $\mathbf{Q}_\texttt{o}$. Each element of $\mathbf{min}$ is to take the first $\texttt{loc}$ for each fresh $\texttt{key}$. Let $\mathbf{Q}'_\texttt{o} = \mathbf{Q}_\texttt{o}||\mathbf{min}$.
  (4) $\mathcal{T}_\texttt{b} = (\mathbf{Q}'_\texttt{o} \bowtie \mathbf{\Phi}_{\texttt{key,prev\_pat}} \bowtie \mathbf{\Phi}_{\texttt{key,rg2}}) \bowtie_{c_2} \mathbf{Q}_\texttt{r}$ where the range query condition c_2 is: $\mathbf{Q}_\texttt{r}.\texttt{loc} + \texttt{rg2} < \texttt{min}$.

Figure 7: Two Pass Chunking SED Algorithm

Similarly, the backward transcripts relies on range-query to prune those locations that cannot reach the $\texttt{min}$ of the next step via range query.

We now show that consecutive application of $\texttt{two\_pass\_sed}(.)$ is equivalent to monolithic SED algorithm. Given a regular expression r, and a word $\mathbf{w}$ we chunk $\mathbf{w}$ into k chunks:= $\mathbf{w}^{(1)}||...\mathbf{w}^{(k)}$. Let $\mathbf{S}$ and $\mathbf{\Phi}$ be the subsigs and database extracted from r. Let $\rho = (\mathbb{F} \times F)^u$ be the ocurrence of all related patterns of r in $\mathbf{w}$. $\texttt{two\_pass\_sed}(.)$ can be applied sequentially in a streaming fasion. We write n applications simply as: $\texttt{two\_pass\_sed}^{(n)}$, where the i'th call is $\texttt{two\_pass\_sed}(\mathbf{Q}_\texttt{o}^{(i-1)}, \mathbf{S}, \mathbf{R}^{(i)}, \mathbf{\Phi})$. Let $\mathbf{R}^{(i)}$ be the two column occurence of patterns fo running through $\mathbf{w}^{(i)}$ the ACDFA of patterns. We have the following result, and the proof is straight-forward, because the step queue of $\texttt{two\_pass\_sed}(.)$ is a always super set of the step queue of $\texttt{eval\_counter\_cs\_std}(.)$.

Theorem 6.3. *Given a regular expression r, and a word $\mathbf{w} = \mathbf{w}^{(1)}||...\mathbf{w}^{(k)}$ be concatenation of k chunks. Let ρ, $\mathbf{S}$, $\mathbf{\Phi}$ derived from r. We have:*

$$\texttt{two\_pass\_sed}(.)^{(k)} = \texttt{eval\_counter\_cs\_std}(\rho, r, \texttt{GT}, 0)$$

.

7 ZKSNARK DESIGN

7.1 Overview

This section introduces $\texttt{FoldPot}$, an efficient folding scheme that is tailored for efficiently verifying the discharging algorithms introduced earlier. $\texttt{FoldPot}$, as its name suggests, mixes the design from several recent folding schemes, including SuperNova [64], [9] Cyclefold [65], Mangrove [77] as well

[9] SuperNova uses a linear folding model as Nova [68]. Here we assume heuristically that there is no known attack over the number of steps

as finger-printing schemes such as Mirage+ [79], and the idea of NARK compiler from ProtoStar [22]. We also assilate the idea of finger-printing techniques, which were practiced in many previous works [63, 79, 83], and adapt it in the folding schemes.

We first introduce the design goals that lead to the design of `FoldPot`. First, we would like to discharge a large collection of files (e.g., all Linux executables). It is known that folding/accumulation provides an overall more efficient solution than proof agregation schemes SNARKPack [52] and GIPA [24] both asympototically (at verifier) and concretely (at prover). Given that discharging of each individual file needs to stack discharging steps non-uniformly, we decide to adopt SuperNova [64] for its support of non-uniform sub-circuits. It is appealing to provide concretely small and constant size proof, especially verifiable on-chain (e.g., ETH), CycleFold [65] provides an efficient decider proof generation over half pairing groups BN254 + Grumpkin, so that $O(1)$ final decider proof can be generated over BN254. Finally, the system relies heavily on finger-printing techniques, and our circuit model adapts the one from Mirage+ [79]. We then integrate the idea of commit-and-fold from Mangrove [77]: commitments of partial segment of witnesses can be added into committed R1CS instance, and the check can be delayed to final decider proof. Leveraging on Mangrove's commit-and-fold scheme, we develop low-cost techniques for passing data between non-uniform circuits.

Our contributions are: (1) A compressing scheme for interactive R1CS model [79] that results in less communication rounds (and hence less recursion overhead); By instantiating the commit-and-fold scheme in Mangrove [77], we are able to produce: (2) An efficient input/output passing scheme between non-uniform step-circuits, which has considerable benefits over general RAM techniques for streaming circuits; (3) A constant-size batching-proof protocol that handles the batch proof problem for non-uniform problem statements.

7.2 Circuit Model

To integrate the finger-printing technique in folding schemes, we mix the I-R1CS model for Algebraic Interactive Protocol (AIP) in [79] and the commited R1CS relation of Nova/SuperNova [68]. The finger-printing technique divides witness into multiple segments, where subsequent segments depend on the Fiat-Shamir public coin randoms that are generated from the hash of previous segments. This technique has been widely deployed in real-world proving systems such as Halo2 [95]. The challenge here in folding is that *the number of segments determines the number of commitments*, which linearly increases the cost of the prover for final decider. In our context, one additional commitments costs at least 5 million R1CS per non-uniform circuit. We therefore seek to reduce the number of segments, and present a Σ-IR1CS model (which is a weaker model than the general I-R1CS) but allows only one segment commitment.

We first motivate the model. Consider the `CP` algorithm, which given an input word s, returns a collection of signatures that cannot be discharged by `CP`. The algorithm walks s through ACDFA, from the acceptance path, extracts a multi-set of acceptance states, compressing it into a unique set, performing a join relation with a table that maps from final states to signatures, and then converting the multi-set into a standard set. The algorithm can be regarded as a sequential composition of a constant number of component interactive protcols that heavily rely on public-coin challenges from the verifier. For instance, to prove two multi-sets $\mathbf{a} \in \mathbb{F}^n$ and $\mathbf{b} \in \mathbb{F}^n$ are permutation of each other, the verifier samples a challenge β and verifies, which is widely used, e.g., in lookup arguments such as plookup [50]:[10]

$$\prod_{i=1}^{n}(\mathbf{a}_i + \beta) = \prod_{i=1}^{n}(\mathbf{b}_i + \beta)$$

Such sequential composition of interactive protocols is nicely captured by the notion of AIP and I-R1CS [79], where an I-RICS instance has a constant number of message exchanges, and a final R1CS check (which is similar to the NARK verifier in ProtoStar [22], with the restriction of degree 2). In [79], the multi-round interaction is handled by a multi-segment variation of Groth16 [56] and commit-and-prove extension of Mirage [63], because verifier can perform the Fiat-Shamir to generate public coin challenges out-of-circuit.

In our context, the circuit has to be folded. Generating a commitment for each round of messages to be exchanged is expensive, because it increases recursion overhead. Instead, we use a restricted I-R1CS (called Σ-I-R1CS) that consists of only two messages from the prover to verifier, one at the beginning, consisting of the concatenation of problem statements and all messages that are independent of public coins, and the last message consists of all intermediate messages that depend on random challenges. We show that a restricted AIP bound by a certain "join" property can always be compiled into a 3-move I-R1CS, and then using the ProtoStar compiler [22], compiled into a NARK. We motivate our construction using an example.

Example 7.1. Assume there is a lookup table $\mathbf{T} \subseteq \mathbb{F}$ which defines a range that excludes 0 (e.g., a set of final states of an ACDFA, which ranges from 1 to $2^k - 1$ for some k). Given $\mathbf{a} \in \mathbf{T}^m$ and $\mathbf{b} \in \mathbf{T}^n$, consider the task of proving that $\mathbf{a}$ is the *set support* of multi-set $\mathbf{b}$, that is: $\mathbf{a}$ includes all

being folded. In [16, Remark 6.3], it was pointed out that stronger assumption on extractor size/time extractor (e.g., additive over prover) leads to polynomial step bound (instead of the constant bound if assuming polynomial extractor for a general SNARK scheme for each step in a recursive SNARK composition scheme). This was shown in recent workk [71]. It gives a contrived example which shows that when blackbox knowledge extractor run time becomes superpolynomial after $\log(\lambda)$ steps, soundness can be broken. However, it also shows that under a modified AGM model (where prover keys are uniformly distributed), Nova/SuperNova is secure given that Pedersen vector commitment is used. Here we use the sequential folding scheme for the problem size we deal with, which incurs maximum 2^{17} steps, though PCD tree based folding/accuulation is possible with further development work.

[10] CHECK TODO: see if this ref is accurate

the elements of $\mathbf{b}$, and $\mathbf{b}$ includes all of $\mathbf{a}$, and there is no duplicate element in $\mathbf{a}$, assuming that all elements of $\mathbf{a}$ and $\mathbf{b}$ belong to $\mathbf{T}$.

We can denote it as a relation $(\mathbf{a}, \mathbf{b}, \mathbf{T}; \mathbf{w}) \in R$, where a four element tuple $(m, n, 2^k - 1, 0)$ defines R's structure (length of each message vector). Here $\mathbf{a}$, $\mathbf{b}$, $\mathbf{T}$ are the problem statement that is visible by both the prover and verifier. $\mathbf{w}$ (separated using ; from the statement) is the secret witness of prover. For this example, $\mathbf{w} = \emptyset$. We call R an *algebraic relation with a fixed structure* as all elements involved in the relation are field elements.[11] We sometimes write $(\mathbf{a}, \mathbf{b}, \mathbf{T}) \in R$ for short when context is clear.

Note that relation R has an interesting property: it is a *natural join* of three relations: R_1, R_1', and R_2. Here, $R_1(\mathbf{x}, \mathbf{y}; \mathbf{w}_1)$ asserts a lookup relation that all elements of $\mathbf{x}$ belongs to $\mathbf{y}$. R_1' is exactly the same as R_1 except changing the config by swapping the size of the first two problem statement elements. $R_2(\mathbf{x}, \mathbf{T}; \mathbf{w}_2)$ asserts that all elements of $\mathbf{x}$ are unique (assuming all $\mathbf{x}_i \in \mathbf{T}$). Apparently $(\mathbf{a}, \mathbf{b}, \mathbf{T}) \in R$ if and only if there exists $\mathbf{w}_1$, $\mathbf{w}_2$, and $\mathbf{w}_3$ s.t. $(\mathbf{a}, \mathbf{b}; \mathbf{w}_1) \in R_1$, $(\mathbf{b}, \mathbf{a}; \mathbf{w}_1) \in R_1$, and $(\mathbf{a}, \mathbf{T}; \mathbf{w}_2) \in R_2$.[12]

For R, each of its sub-relation has a constant move special sound protocol (similar to those defined in ProtoStar [22]). For instance, Consider R_1, it has a 3-move special sound protocol by [58], as shown in Π_1 in Figure 8. Similarly, let Π_1' be the protocol for R_1'. Its basic idea is that if the lookup relation holds, the sum of logarithmic derivative of the elements of two vectors will match. Protocol Π_2 for R_2 is essentially running Π_1 to prove that the difference for each pair of elements of $\mathbf{x}$ is a positive integer, excluding 0 (because $0 \notin \mathbf{T}$), thus proving $\mathbf{x}$ is sorted in ascending order, hence a set without duplicates. One can see that each corresponds to an I-R1CS protocol where a degree 2 function is performed at the end for making checks. It is known that degree 2 functions can be expressed in R1CS.

Sequential composition of these protocols is costly in folding. Instead, we can compile them into Π as shown in Figure 8. Basically, it combines all the first message from all sub-protocols in its first message, keeping the structure of public coin verifier challenges, and then combine all "response" messages from sub-protocols in one last message, and then projecting each individual messages from the combined messages, and run the degree 2 check functions correspondingly. We call this compiled protocol Σ-*I-R1CS* because it has 3 moves.

We can then apply the comiler technique in ProtoStar [22, Section 3.2], to compile a Σ-I-R1CS into a committed protocol by sending commitments of the first and third message, and add additional checks to open them (as shown in Π_{cm} in Figure 8). One might wonder why sending the commitments and then immediately open them up. The reason is that these commitments are included as part of the committed instances to be folded, and the opening check only performed at the final decider proof, *only once* for many instances that are folded.

Let `ro` denotes the hash function to instantiate the random-oracle. We summarize our discussion and the main finding below.

Definition 7.2. An algebraic relation R is an NP relation over $\mathbb{F}$ with a fixed structure. We call $R(\mathbf{x})$ *decomposible by joins* of a constant number of relations $R_1, ...R_k$ if all of the following are satisfied: (1) Each R_i has a special-sound Σ-protocol, and (2) Denote $\mathbf{x}$ as $\mathbf{x}^{(k+1)}$. There exists a join function φ s.t. φ is a DNF (disjunctive normal form) of equality constraints of form $\mathbf{x}_u^{(i)} = \mathbf{x}_v^{(j)}$, where $0 \leq i, j \leq k+1$. For any $(\mathbf{x}, \mathbf{w})$: $(\mathbf{x}; \mathbf{w}) \in R$ if and only if there exists $(\mathbf{x}^{(1)}; \mathbf{w}^{(1)}) \in R_1$, $(\mathbf{x}^{(2)}; \mathbf{w}^{(2)}) \in R_2$, ... $(\mathbf{x}^{(k)}; \mathbf{w}^{(k)}) \in R_k$, and $\varphi(\mathbf{x}^{(1)}, ..., \mathbf{x}^{(k)}, \mathbf{x}) = \texttt{true}$.

We note that there can be more expressive form of expressing the join relation φ, but this form sufficies for our applications. Note that φ is degree 1, hence it can be encoded in R1CS.

Lemma 7.3. *Let R be a decomposable relation joined by k sub-relations with special-sound Σ-protocols. There exists a 3-move special-sound I-R1CS protocol for R.*

Proof. We use $\Pi_s(R)$ to denote the I-R1CS protocol that sequentially composes all k sub-protocols of R, which performs the check of join relation ϕ and the R1CS checks for all of the sub-relations, and returns the conjunction of the Boolean results. $\Pi_s(R)$ is a complete and special-sound protocol for R, and it has $3k$ moves.

We use $\Pi_p(R)$ to represent the Σ-protocol which is constructed by sequentially concatenating the first messages of all subprotocols of R for its first messages, and similarly constructing its second and thrid messages. $\Pi_p(R)$ has 3-moves. Its last R1CS module performs the check of ϕ and the R1CS checks of all sub-protocols. It is apparent that for any transcript $\Pi_s(R)$, there is a one-to-one mapping Ψ_R which uniquely constructs a transcript for $\Pi_p(R)$. Ψ_R can be directly inferred from the structure information of R and its sub-protocols.

It is not hard to see that $\Pi_s(R)$ and $\Pi_p(R)$ are equivalent, in the sense that, for an accepted transcript t of $\Pi_s(R)$, its $\Psi_R(t)$ is also accepted by Π_p; vice versa for the inverse relation Ψ_R^{-1}. An "invalid" transcript (which falsely proves a $(\mathbf{x}; \mathbf{w}) \notin R$ is in R) has negilible probability of passing $\Pi_s(R)$, and also for $\Pi_p(R)$. The soundness error of the two protcols are the same, both linearly degrading with the number of sub-protocols, because the R1CS check modules of the two are equivalent.

When Fiat-Shamir is applied to $\Pi_p(R)$, the soundness loss *independent* of the number of public coin verifier challenges, per [10]. □

We note that our I-R1CS model (even extended to more than 3 messages) is a strictly (restrictive) weaker than the

[11] We note that group elements can always be represented non-natively given a target field by bit-splitting.

[12] In real implementation, we allow $\mathbf{a}$ and $\mathbf{b}$ to contain 0 as dummy filler elements to make the circuits more general purpose at slightly more constriant cost.

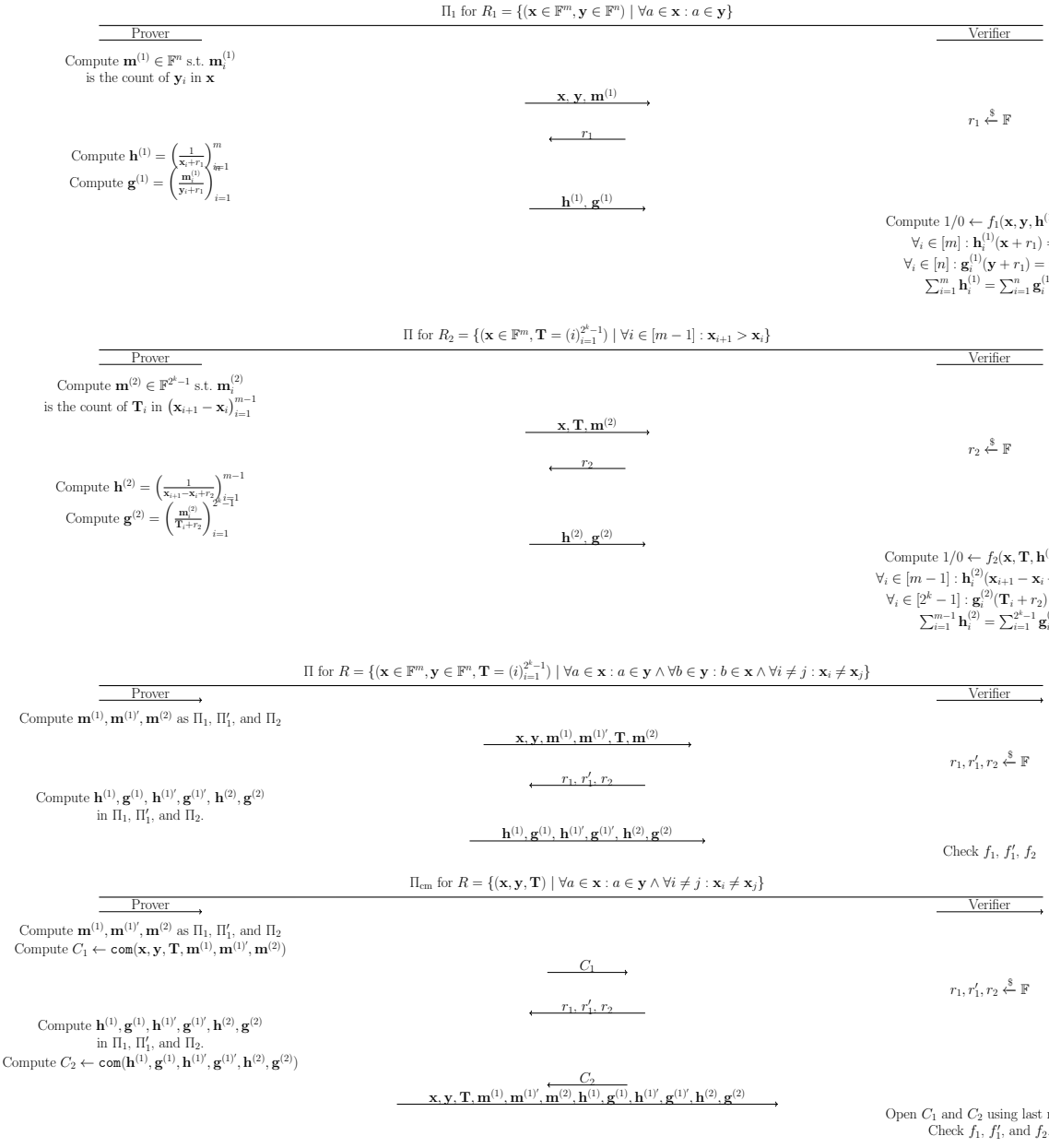


Figure 8: Protocols for Example 7.1

original I-R1CS [79]. The key difference is that, by the restriction of "join" relation, our model explicitly *enforces that the second and third message of sub-protocols do not have dependence on other sub-protocols*, only the claims of sub-protocols might be related. This "restriction" allows the projection between $\Pi_s(R)$ and $\Pi_p(R)$. This model can still capture a wide spectran of application scenarios, e.g., all of circuits in our ZkregPlus system satisfy the condition.

7.2.1 Lookup Table. One drawback of the protocols presented in Figure 8 is that if the lookup table size is much larger than the statement vector, it is concretely very inefficient. We provide two solutions for the problem.

One solution, when the total prover work is small (e.g., very small number of folding steps or in the case of no folding), one can use commit-and-prove approach to take out the commitment of all elements that need to perform lookups, and then outside of the circuit, run an independent zero knolwedge lookup protocol. As the lookup table is fixed for our application scenario, it can be pre-processed. We provide a zero knowledge version of the CQ protocol [42] in Appendix **??**, as an contribution of independent interst. In this case,

the prover only has to pay a concrete cost of the query table size (which is indpepenent of the size of the container table). The protocol in **??** can be replaced by the zk-cq protocol in [29], which has slightly smaller proof size but higher prover cost than our protocol.

The second solution is to adopt the commit-and-fold scheme presented in Mangrove [77]. When the total prover work (e.g., folding 2^{17} circuits of an average circuit size of 2^{16} each, a lookup table of 2^{28} can be distributed into each step, incurring a negligible cost of about 3%). We adopt the Mangrove approach in our implementation for its convenience. We note that the folding lookup scheme in ProtoStar [22] yields similar asympototic and estimated similar concrete performance.

There is one more issue to consider: for the ZkregPlus system, there are many individual look-up tables. Range proofs are all achieved using lookups, and there are several different types of range proofs: ACDFA states are 23 bits, input alphabet is 4-bit, and signature IDs are 16 bits. The `CP` approach needs 2 ACDFAs, one for case insensitive and one for case sensitive. The `BS` and `SDE` combined, needs 2 more ACDFAs. In total, these 4 ACDFAs, have about 160 million transtions, to be encoded in four separate look-up tables. We also need to encode the `BS` and `SDE` discharging instructions separately.

We put all these separate tables into one 2-column look-up table, with first column indicates the sub-table ID, and the second column indicates the value. We reserve 0 as `null` sub-table ID. The logarithmic derivative approach [22, 42, 58] supports multi-column table well.

Putting all ingredients together, we now have the formal definition of Σ-I-R1CS enriched with lookup. It is used to specify the step-circuit in FoldPot, essentially adopting the SuperNova [64] + CycleFold [65]. This definition is adapted from [79, Definition 1].

Definition 7.4. A Σ-I-R1CS is a tuple ϕ defined as below:

$$(N, n, \mathbb{F}; \mathbf{T} \in \mathbb{F}^{N\times 2}, \ell_x, \ell_{m_1}, \ell_{m_2}, \ell_{m_3} \in \mathbb{N}; A, B, C \in \mathbb{F}^{n\times m}; \chi)$$

Here $\mathbf{T}$ is a pre-processed 2-column look-up table. $\chi : \mathbb{N} \to \mathbb{N}$ is a function that maps a variable index to a sub-table ID in $\mathbf{T}$. The protocol defines a 3-round AIP over $\mathbb{F}$. The prover sends the first (including statement) and third message, and the verifier sends the second message of uniformally sampled challenges. Let $\mathbf{x}$ be the statement, and $\{\mathbf{m}^{(i)}\}_{i=1}^{3}$ be the three round of messages. let $m = \ell_x + \ell_{m_1} + \ell_{m_2} + \ell_{m_3}$. The R1CS checker consists of three matrics A, B, C of n constraints. Let $z = \mathbf{x}||\mathbf{m}^{(1)}||\mathbf{m}^{(2)}||\mathbf{m}^{(3)}$. The verifier accepts if $Az \circ Bz = Cz$, and for each $i \in [m]$: if $\chi(i) \neq \texttt{null}$ then $(\chi(i), z_i) \in \mathbf{T}$, i.e., z_i is in sub-table $\chi(i)$.

7.3 Commit-and-Fold

We now briefly review the commit-and-fold scheme proposed in Mangrove [77], which we later instantiate to support inter-circuit data passing and batch processing of non-uniform word problems.

The base line of FoldPot is SuperNova [64] and CycleFold [65], which solves non-uniformity of circuits in folding, and delivers $O(1)$ proof. However, we still need to address the following challenges.

(1) We need to handle large files (e.g., 2^{27} bytes) in "streaming" mode. In this case, step-circuits need to pass large amount of input/output data. We need a concretely more efficient schemes than the general RAM approach (e.g., [79]).
(2) We need to discharge a collection of files in batch, and then provide an individual (2-stage) proof for each file. Given an aggregated word $\mathbf{s}$ which is a concatenation of k segments, i.e., $\mathbf{s} = \mathbf{s}^{(1)}||\mathbf{s}^{(2)}||...||\mathbf{s}^{(k)}$, and the individual commitments to each $\mathbf{s}^{(i)}$, we need to link these segment commitments with the $\mathbf{s}$ in circuit, and prove all of them malware free. Here the challenge is that these word segments can have arbitrary length.

We first present a simplified framework of Mangrove's commit-and-prove, in the context of committed R1CS used by Nova, SuperNova and CycleFold, as all of our circuits are low-degree. Recall the Nova [68] framework: the folding prover is given a running instance $(\mathbf{U}_i; \mathbf{W}_i)$ and an incoming (step) instance $(\mathbf{u}_i; \mathbf{w}_i)$, where $\mathbf{W}_i$ and $\mathbf{w}_i$ are the witnessnes, and fold them into a new instance $(\mathbf{U}_{i+1}; \mathbf{W}_{i+1})$. A folded instance is a tuple $(\overline{\mathbf{W}}, \overline{\mathbf{E}}, u, x)$, where u and x are "public" parameters of relaxed R1CS relation, and $\overline{\mathbf{W}}$ and $\overline{\mathbf{E}}$ are homomorphic commitments (e.g., Pedersen vector commitments) to the witness and cross terms. The folding verifier in the IVC scheme does not have to check the opening of $\overline{\mathbf{W}}$ and $\overline{\mathbf{E}}$ at each step. Instead, the check is *delayed* to the final decider proof, and performed *only once* for all folded instances.

The key observation made by Mangrove [77] is that: the committed instance can have extra commitments on subset of witnesses. The opening of these commitments can be also delayed to the final decider step. Thus, if one performs a one pass aggregation (e.g., using Merkle tree) of these extra commitments, a commit-and-prove scheme like LegoSNARK [34] can be achieved. In our context, the commitment to the subset of witness can be used for generating the Fiat-Shamir randomness and can be verified in-circuit.

In Mangrove, the formalization of commit-and-fold was made using the polynomial relations, which supports high-degree. We re-instate the result of Mangrove in the context of 2-degree R1CS as below. Given a Σ-I-R1CS instance ϕ whose R1CS relation is $A, B, C \in \mathbb{F}^{n\times m}$, where m is the number of variables. Enrich ϕ with ρ, a subset of $[0, m-1]$ sorted in ascending order. Given the fixed size witness vector $\mathbf{W}$, we use $\rho(\mathbf{W})$ to denote the projection of $\mathbf{W}$ over the indices included in ρ. For example, let $\mathbf{W} = (5, 30, 20, 1)$ and $\rho = (0, 2)$, then $\rho(\mathbf{W}) = (5, 20)$. We denote the new Σ-I-R1CS instance as ϕ_ρ. The following defines relaxed and committed R1CS instance. The difference from NOVA is the addition of the projected commitments: $\overline{\rho(\mathbf{W})}$.

Definition 7.5. (extending [68, Definition 12]) A relaxed R1CS instance of ϕ_ρ consists of $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{n\times m}$ as in ϕ, an error vector $\mathbf{E} \in \mathbb{F}^m$, and a scalar $u \in F$, and input/output vector $\mathbf{x} \in \mathbb{F}^\ell$. Let $\mathbf{z} = (\mathbf{W}, \mathbf{x}, u)$, and $(\mathbf{z}, \mathbf{W}^{(\rho)})$ is accepted

if $\mathbf{Az} \circ \mathbf{Bz} = u(\mathbf{Cz}) + \mathbf{E}$ and $\mathbf{W}^{(\rho)} = \rho(\mathbf{W})$. A committed R1CS instance for ϕ_ρ is a tuple $(\overline{\mathbf{W}}, \overline{\mathbf{E}}, u, \mathbf{x}, \overline{\mathbf{W}^{(\rho)}}$.

The Nova/SuperNova folding schemes can be modified correspondingly. When folding $\overline{\mathbf{W}}$, $\overline{\mathbf{W}^{(\rho)}}$ can be folded correspondingly (and in CycleFold, it can be fed to a CycleFold circuit works on Grumpkin curve similarly). At the final decider, the verifier checks the folded $\mathbf{W}^{(\rho)} = \rho(\mathbf{W})$, and they are the opening to the folded commitments. The correctness of the scheme depends on the following lemma, which is apparent.

LEMMA 7.6. *Let $|\rho| = v$. Assume $\overline{\mathbf{W}}$ and $\overline{\mathbf{w}}$ are commitments to some $\mathbf{W} \in \mathbb{F}^n$ and $\mathbf{w} \in \mathbb{F}^n$. Similarly $\overline{\mathbf{U}}$ and $\overline{\mathbf{u}}$ are commitments to $\mathbf{U} \in \mathbb{F}^v$ and $\mathbf{u} \in \mathbb{F}^v$. Let r be a public coin verifier challenge, $\mathbf{W}'$ be the opening to $\overline{\mathbf{W}} + r\overline{\mathbf{w}}$, and $\mathbf{U}'$ the opening to $\overline{\mathbf{U}} + r\overline{\mathbf{u}}$. If $\mathbf{U}' = \rho(\mathbf{W}')$ then the probability that $\mathbf{U} \neq \rho(\mathbf{W})$ or $\mathbf{u} \neq \rho(\mathbf{w})$ is negligible.*

To apply the finger-printing technique in circuit, a binding commitment to the fixed witness subsets have to be generated by the prover by performing a 1st walk before generating the proof, so that the commitment can be used to generate Fiat-Shamir randomness. Following Mangrove, but in a sequential folding scheme, we use a hash-chain.

Let their be k step-circuits involved in a prover job. Let the $\mathbf{W}^{(\rho)} = \mathbf{W}^{(\rho,\mathbf{1})}||\mathbf{W}^{(\rho,\mathbf{2})}||...\mathbf{W}^{(\rho,\mathbf{k})}$. We define $\texttt{hashchain}(\mathbf{W}^{(\rho)}, i)$ recursively as:

$$\begin{cases} \mathsf{hash}(\overline{\mathbf{W}^{(\rho,1)}}) & \text{if } i = 1 \\ \mathsf{hash}(\texttt{hashchain}(\mathbf{W}^{(\rho)}, i-1), \overline{\mathbf{W}^{(\rho,i)}}) & \text{if } i > 1 \end{cases}$$

Now that the hashchain can be computed in circuit by slightly enhancing the Nova framework because at each step $\overline{\mathbf{W}^{(1,i)}}$ is available for random linear combination during folding (and also as input to the CycleFold component on Grumpkin).

7.4 Passing Data Between Circuits

The first application of $\texttt{hashchain}(\overline{\mathbf{W}^{\rho}})$ is to pass data between circuits. Recall that $\mathbf{W}^{\rho}$ serves as a "fixed" memory. Here we require that input and output buffers of circuits are located at fixed positions of $\mathbf{W}^{\rho}$ and the hashchain serves as the seed for generating Fiat-Shamir randoms. We first handle the problem of passing data between k uniform circuits.

For each circuit i, its its fixed chunk $\mathbf{W}^{(\rho,i)}$ has a fixed layout of u elements allocated for input and also u elements for output. We denote these two regions as $\mathbf{I}^{(i)}$ and $\mathbf{O}^{(i)}$. Note that $r = \texttt{hashchain}(\mathbf{W}^{(\rho)})$.

We then let k circuits jointly perform the following computation (in circuit), essentially asserting the equivlance of vector $(\mathbf{I}^{(1)}, ..., \mathbf{I}^{(k-1)})$ and $(\mathbf{O}^{(2)}, ..., \mathbf{I}^{(k)})$

$$\sum_{i=1}^{k-1}\sum_{j=1}^{u} r^{(i-1)*k+j}\mathbf{I}_j^{(i)} = \sum_{i=2}^{k}\sum_{j=1}^{u} r^{(i-2)*k+j}\mathbf{O}_j^{(i)}$$

The above can be computed by accuulating partial results across circuits, and the equality check be performed in the last circuit. The public coin verifier challenge r guarantees that with negligible probability the input and output buffers do not match.

We next extend the above idea to non-uniform circuits in SuperNova. Each circuit (type) has its own configuration (including the size of pre-allocated buffer for its input/output regions). These are *fixed constants* embedded in the circuits. Then we need to keep track of the actual input/output size. Let them be $u_{in,i}$ and $u_{out,i}$ for circuit i, they are located at some fixed positions in In $\mathbf{W}^{(\rho,i)}$. Then we just need to perform the following check using all circuits in folding:

$$\sum_{i=1}^{k-1}\sum_{j=1}^{u_{in,i}} r^{(i-1)*k+j}\mathbf{I}_j^{(i)} = \sum_{i=2}^{k}\sum_{j=1}^{u_{out,i}} r^{(i-2)*k+j}\mathbf{O}_j^{(i)}$$

In addition, each circuit needs to perform in-circuit for: (1) $u_{in,i}$ does not exceed to static input buffer limit; and (2) total size of read from the input buffer does not exceed $u_{in,i}$. Then similar checks apply to $u_{out,i}$ for each circuit. This approach fits our specific application setting, and has better concrete performance (mainly $1\times$ the input and output buffer size), compared with the the general RAM approach (e.g., [79], which costs at least $43\times$ of memory cell accesses).

7.5 Batch Processing with Non-uniform Circuits

We now discuss how to batch process a large collection of non-uniform files via folding. By "non-uniform" we mean that these files have different sizes, and may generate witness of different size, thus relying on non-uniform circuits. In practice, we define one or more circuits for each integer logarithm of file sizes. We first formally define the problem. Let $\mathbf{s} = \mathbf{s}^{(0)}||...||\mathbf{s}^{(k)}$ be a concatenation of k words $\mathbf{s}^{(i)}$, and let $\ell = \left(|\mathbf{s}^{(i)}|\right)_{i=1}^{k}$ be the vector of string lengths. We call $\mathbf{s}^{(i)}$ the i'th *string fragment* and s the *batched string*.

Equation 1 presents the relation, where $\left(k, \overline{\mathbf{s}}, \left(\overline{\mathbf{s}^{(1)}}, ..., \overline{\mathbf{s}^{(k)}}\right)_{i=1}^{k}, \overline{\ell}, R\right)$ are the public statement also visible to the verifier but $\left(\mathbf{s}, \ell, \left(\mathbf{s}^{(i)}\right)_{i=1}^{k}\right)$ are the secret witness, which are separated from the public statement using ";". Here R represents a collection of regex signatures.

$$\left\{ \begin{array}{l} \left(k, \overline{\mathbf{s}}, \left(\overline{\mathbf{s}^{(1)}}, ..., \overline{\mathbf{s}^{(k)}}\right)_{i=1}^{k}, \overline{\ell}, R; \mathbf{s}, \ell, \left(\mathbf{s}^{(i)}\right)_{i=1}^{k}\right) : \\ (1)\forall i \in [k] : \ell_i = |\mathbf{s}^{(\mathbf{i})}| \wedge \\ (2)\mathbf{s} = \mathbf{s}^{(1)}||...||\mathbf{s}^{(k)} \wedge \\ (3)\forall i \in [k] : \forall r \in R : \mathbf{s}^{(i)} \notin L(r) \end{array} \right\} \quad (1)$$

Intuitively, the public statement asserts the following given the commitments to all string fragments and length information: (1) All strings $\mathbf{s}^{(i)}$ are malware free[13], and (2) Each $\overline{\mathbf{s}^{(i)}}$ indeed hides the i'th string fragment of $\mathbf{s}$ as defined by the length vector, which hides inside $\overline{\ell}$. Naturally, the proof

[13] Note that this is however, different from asserting that $\mathbf{s}$ as a single string is malware free. When $\mathbf{s}$ is a concatenation of many files, it is easy to trigger false positive (identified as a malware).

consists of two parts: a global batch proof for all, and an individual proof for each string fragment. Our solution provides $O(1)$ proof size for both.

Note that unlike the folding scheme where Pedersen Vector Commitment is used, here in the problem statement all commitments are *KZG commitments* (where a vector of field elements serve as the co-efficients in the univariate polynomial for KZG). Given a KZG commitment $\overline{\mathbf{x}}$ to a vector $\mathbf{x}$ of degree d, we use $\pi_{\mathtt{kzg}}(\overline{\mathbf{x}}, a, y)$ as the KZG polynomial opening proof for: $y = \sum_{i=0}^{d} \mathbf{x}_i a^i$. It is known that given the Pedersen Vector Commitment and KZG commitment to the same vector, the Quasi-linear NIZK (QA-NIZK) [54] provides linear prover complexity and constant size proof for their equivalence, as demonstrated in LegoSnark [34].

We now address (1) and (2) of the relation and delay (3) to the next subsection. Due to the use of non-uniform circuits, we discharge *at most* one string fragment each circuit.[14] For large string fragment, it may span over multiple circuits. For $\mathbf{s}^{(i)}$ spanning over d circuits, we list these circuits as $\mathtt{C}^{i,1}, ...\mathtt{C}^{i,d}$. Similarly, their fixed witness subset is denoted as $\mathbf{W}^{(\rho,i,1)}, ..., \mathbf{W}^{(\rho,i,d)}$.

The prover first walks over the public statement and defines the $\mathbf{W}^{(\rho,i,j)}$ for each step-circuit $\mathtt{C}^{(i,j)}$. Note that this is "fixed" information that does not depend on any verifier challenge. It can be directly generated from the problem statement.

$$\mathbf{W}^{(\rho,i,j)} = \left(\mathbf{s}^{(i,j)}, \ell^{(i,j)}, \mathbf{r}_i, \mathbf{v}^{(i,j)}\right)$$

Here $\mathbf{s}^{(i,j)}$ is the j'th slice of $\mathbf{s}^{(i)}$ allocated to circuit $\mathtt{C}^{(i,j)}$. $\ell^{(i,j)} = \sum_{k=1}^{j} \mathbf{s}^{(i,k)}$ is the accumulated length of slices up to j. Let $\mathtt{C}^{(i,d)}$ be the last circuit for processing $\mathbf{s}^{(i)}$, then $\ell_{i,d} = \ell_i$, which is checked in circuit. $\mathbf{r}_i$ is a Fiat-Shamir verifier challenge used in the individual proof. It is pre-computed outside circuit, and served as a non-deterministic advice:

$$\mathbf{r}_i = \mathtt{ro}\left(\overline{\mathbf{s}}, i, \overline{\mathbf{s}^{(i)}}, \overline{\ell}\right) \tag{2}$$

Define $\mathbf{v}_i$ as the following, where $\mathbf{s}_u^{(i)}$ is the u'th element of string fragment $\mathbf{s}^{(i)}$:

$$\mathbf{v}_i = \sum_{u=0}^{d} \mathbf{r}_i^u \mathbf{s}_u^{(i)} \tag{3}$$

Apparently, $\mathbf{v}_i$ is an evaluation of $\mathbf{s}^{(i)}$ as the co-efficient vector of a polynomial. $\mathbf{v}_i$ can be be computed by accumulating its value across circuits. This is done by computing $\mathbf{v}^{(i,j)}$ as the following: $\sum_{u=0}^{\ell^{i,j}} \mathbf{r}_i^u \mathbf{s}_u^{(i)}$, and apparently for the last circuit in the sequence, $\mathbf{v}^{(i,d)} = \mathbf{v}^{(i)}$.

The prover then computes the hashchain of the fixed witness subset, and based on it and other KZG commitments in the public statement, compute the Fiat-Shamir challenge c:

$$c = \mathtt{ro}\left(\mathtt{hashchain}\left(\mathbf{W}^{(\rho)}\right), \overline{\mathbf{r}}, \overline{\mathbf{v}}\right)$$

Then the prover computes the rest of the witness fo all circuits, based on challenge c, and after all are folded, the last circuit outputs the following:

(1) $y_v = \sum_{i=1}^{k} \mathbf{v}_i c^i$
(2) $y_r = \sum_{i=1}^{k} \mathbf{r}_i c^i$
(3) $y_s = \sum_{i=1}^{n} \mathbf{s}_i c^i$
(4) $y_\ell = \sum_{i=1}^{k} \ell_i c^i$

Each individual circuits performs the verification and asserts that $\mathbf{v}^{(i,j)}$ are computed inside the circuit correctly using $\mathbf{r}_i$ and $\mathbf{s}^{(i,j)}$, and similarly ℓ_i accumulated correctly using $\ell^{(i,j)}$. Then the global **batch proof** consists of the following. Thus, by asserting the values of the in-circuit y_v, y_r, y_s, y_ℓ and their counter-parts in the KZG opening proof, the verifier is convinced that the shares of $\mathbf{v}, \mathbf{r}, \mathbf{s}, \ell$ are distributed across all circuits in their fixed memory correctly.

(1) $\pi_{\mathtt{snark}}$ is the final decider proof (compressed using Groth16 [56] in CycleFold) for the folded circuits.
(2) $\pi_{\mathtt{kzg}}(\overline{\mathbf{r}}, c, y_c)$, $\pi_{\mathtt{kzg}}(\overline{\mathbf{s}}, c, y_s)$ $\pi_{\mathtt{kzg}}(\overline{\mathbf{v}}, c, y_v)$, and $\pi_{\mathtt{kzg}}(\overline{\ell}, c, y_\ell)$ which can be further aggregated.

We now proceed to each **individual proof**. Given a homomoric commitment $\overline{\mathbf{x}}$ to a vector, $\pi_{\mathtt{vec}}(\overline{\mathbf{x}}, i, y)$ asserts that the i'th element of $\mathbf{x}$ is y. Then for each statement $\left(\overline{\mathbf{s}^{(i)}}, i, \overline{\mathbf{s}}, \overline{\ell}\right)$, i.e., $\overline{\mathbf{s}^{(i)}}$ hides the i'th string segment in $\mathbf{s}$ as defined by the length vector in $\overline{\ell}$, the individual proof consists of the following (in addition to the global batch proof):

(1) Fiat-shamir transform of verifier challenge: $\mathbf{r}_i = \mathtt{ro}\left(\overline{\mathbf{s}}, \overline{\ell}, \overline{\mathbf{s}^{(\mathbf{i})}}, \overline{\mathbf{v}}\right)$.
(2) $\pi_{\mathtt{vec}}(\overline{\mathbf{v}}, i, \mathbf{v}_i)$, $\pi_{\mathtt{vec}}(\overline{\mathbf{r}}, i, \mathbf{r}_i)$
(3) $\pi_{\mathtt{kzg}}(\overline{\mathbf{s}^{(i)}}, \mathbf{r}_i, \mathbf{v}_i)$,

Intuitively, the proof works by performing finger-printing on $\mathbf{s}^{(i)}$ at random point $\mathbf{r}_i$ (producing $\mathbf{v}_i$).[15]

7.6 ==== NOT USED BELOW====

implementation of folding schemes,

In this section, we discuss how to efficiently encode the 5-stage regex discharging algorithms in zkSNARK. Given that there are many small files (e.g., emails), it is not economical to provide a proof with size almost comparable to the original file size. We opt for pairing based constant size proof system such as Groth16 (instead of log-size proof system such as sumcheck based systems). Our choice of proving scheme s is also limited by the choice of open-source folding scheme implementations available at the time the project is implemented. In this case, we choose to adopt Sonobe (an implementation of Nova + Cyclefold folding schemes that work for BN254-Grumpkin curve that allows producing final decider proof in Groth16). ,It is possible to encode our 5-stage discharging algorithms using many other schemes, such as Plonk based for universal prover keys, or Mangrove for PCD foldings schemes with constant memory footprints, and we leave them for future work.

We have several design challenges to overcome. First of all, we would like to take advantage of the folding schemes so that the prover cost can be minimized. Second, in addition to mass processing regex signatures, we can also mass-discharge files,

[14] In another word, no circuit will process multiple string fragments.

[15] In practice, the KZG commitment of $\mathbf{s}^{(i)}$ by padding it with extra random elements, as many systems such as Halo2 [95].

e.g., providing a commitment to a large collection of files (e.g., all binary executables or all Enron emails), provide an $O(1)$ proof for free of malware for all of them, and then provide a membership of an individual file commitment belong to the combined commitment. Third, to achieve the aforementioned scheme, we have to handle vast difference in file sizes from 2^9 to 2^{27}. This concerns handling non-uniform circuits (with different size). Fourthly, we need to handle lookup arguments effectively. Although CQ lookup is handled in ProtoStar, we present a simpler stand-alone zk-CQ protocol which shows similar concrete prover effiicency.

7.7 Consideration of Folding Cost

For the vast majority of folding schemes that follow the Nova folding, it is shown that prover concrete efficiency is 4× of Groth16. However, there is a final decider scheme (in Sonobe) that is expensive to compute, its cost is shown in the following, given n the step-wise circuit size, k the number of steps applied in folding:

$$2^{13.5} + 3n$$

Mainly, the $2^{13.5}$ is a fixed non-native field arithmetic component in the decider and $3n$ is the linear cost on proving the step-wise R1CS relation. Now given that folding itself takes $2n$ group operations per step, the fraction of the decider proof genreation among the total cost is, when k is large enough:

$$\frac{2^{13.5} + 3n}{2nk} \approx \frac{1.5}{k}$$

In our later presentation, we will need to customize the Sonobe final decider proof, which will add several more "expensive" constant size computation. However, given that k is large enough, they will be amortized soon, which does not cause concern.

Also another interesting point is the step-wise circuit size. In the celebrative work of Mangrove, it is shown how any arbitrary Plonk Irelation can be chunked into any arbitrary thread of smaller computation, which allows arbitrary folds of scalability via parallelism, i.e., it allows chunking a large R1CS into many very small steps. In our case, server RAM is not a concern, so that we allow large step-wise circuit size, as long as it does not make k too small.

7.8 Road Map

In this section, we will first discuss how to encode the 5-stage regex discharging algorithm efficiently in R1CS. We then discuss a customized folding scheme to support batch process a large collection files.

7.9 Encoding 5-Stage Regex Discharging Algorithm

7.9.1 Encode DFA (TODO). In practice, an ACDFA can be transformed into a DFA. As DFA acceptance is needed in various parts of the entire algorithm, we discuss them first.

The key idea is that: there is an external and precomputed lookup table of all DFAs needed (notice that there are multiple tables). But we integrate them into one table of two columns `TableID`, `Entry`.

TODOS in details: rough idea

(1) discuss how to encode DFA transition ((2) discuss what kind of information will be stored in external table, e.g., transitions (for different DFAs), the critical patterns needed by each signature).

(3) present an algorithm in R1CS or circuit, which takes in witnesses: (source state, char, dest state) sequence, produces (still secret) output: the set of signatures that should be triggered (those who have their critical patterns appear along the acceptance path)

(4) present a variety of gadgets that proves permutation of vectors, equivalence of vectors and its sorted form, packing a vector to a set etc. In particular, notice that we can use the in-circuit form of the Habock scheme [58] for proving lookup relation between two vectors (when they are small) *in circuit*, this is useful for proving a "set" is reduced from a vector with duplicate items.

(5) Given the gadgets, this motivates the definition of R1CS relation with 2 stages (to allow to generate the random needed by the gadget), and the need to tag some some elements for the use of external lookup argument.

7.10 Customized zkSNARK with Folding Schemes

To support the design goal of batch processing large quantity of files against large collection of regex signatures, we present a customized folding scheme by adapting the Nova [68] + Cyclefold [65] with a Groth16 final decider proof generator (based over the Sonobe [1] framework).

7.10.1 Discussion of Fiat-Shamir. Applying Fiat-Shamir is tricky, as pointed in [15], Fiat-Shamir needs to include statement to defeat adaptive adversaries. There were other attacks on parallel repetition of multi-round protocols, however, it is shown in [10] that for *special sound* interactive protocols the security degrades only linearly with the number of queries to the random oracle, thus, *independent of* the round of an interactive protocol.[16] This fomrs the basis of security of application of Fiat-Shamir in folding schemes such as ProtoStar [22] and Mangrove [77].

7.10.2 Discussion of R1CS Model. We assume the following extended R1CS model: the variables each carry a (an optional) collection of tags which indicates the range proofs it need to satisfy and up to one CQ lookup. The CQ lookup table is a 2-column table with the first column indiates `table_id`, the the 2nd conlumn containing the entry values. Thus, we are allowed to concatenate multiple tables into one 2-column CQ

[16] The intuition is that since the interactive protocol is special sound (allowing efficient knowledge extractor given a transcript tree to extract true witness). If the attacker of Fiat-Shamir has non-negligible probability of success, then the attacker is able to inject too many fake but accepted transcripts, thus making the true knowledge extractor unable to extract true witness efficiently.

table. In particular, we have 4 ACDFA's transition tables (for `CP` and `BS` and each needs two for supporting ignore-case). We also need to encode the `BS` entries for all signatures and `SDE` entries, thus the total CQ lookup table is close to 2^{28} (which is the maximum size allowed by the BN254-Grumpkin curve pair, because it is the largest factor of $|F_r|-1$, needed for FFT). Note that this also restricts circuit size to 2^{28}. (Question: can we find a similar curve pair like BN254-Grumpkin for BLS12-381?)

7.10.3 Discussion on Lookups. When the lookup table is huge, it is not a good idea to embed it in the circuit (because it is part of the circuit size). In ProtoStar [22], a folding scheme is supported for folding the Haböck scheme [58]. The prover pays a cost indpendent of the lookup table size per step of folding, however, this cost is eventually paid in the construction of final decider proof. For our scheme, the total cost paid by the prover is list below, assuming query table size is m and lookup table size is N, for u steps of computation:

$$u \times 3m + |N|$$

If one uses pre-computed quotient cache like [42], this can be improved to (in terms of group operations):

$$u \times 3m + 8 \times \texttt{min}(u \times m, |T|)$$

Here for each folding operation, because there is a need to encode three vectors $\vec{h}$, $\vec{g}$ and (a supppresed form of) of $\vec{m}$ (because $\vec{m}$ is sparse) in the transcripts, when calculating their Pedersen commitment, the prover has to pay $3m$ group operations (in addition to the other witness variables).[17]

In the final proof generation, the checks of three equations as listed in the last step of section 4.3 of ProtoStar has to be represented in R1CS. Given that now the full entries of $\vec{m}$ has to be reasoned about, its size is $\texttt{min}(um, |T|)$. In our setting, we generate Groth16 proof, this is equivalent to 8 times of G1 operations (which is 3 G1 and 1 G2 for each R1CS constraint, where 1 G2 costs 4 times of G1 operation). When $um < |T|$, this amounts to $11 \times um$ group operations in total for the prover.

We later introduce a concretely more efficient scheme than the above. We construct a complete zk protocol for the CQ protocol. Its prover cost is $8m$ G1 operations for each step of computation, thus totaling $8um$. We show that its folding cost is negligible. We commit to each smaller query table at each step, this is a part of the public input. The verifier circuit just performs the random linear compbination of this group element (which costs one group operation over Grumpkin curve, which is about 1000 multiplications). Then in the final decision step, a fixed 10M R1CS is paid for the verification of the group operation, and a fixed m R1CS constraints is paid to verify the fragment of witness corresponds to the query table commitment. In total we pay:

$$8um + 8 \times (2^{23.5} + m)$$

It is not hard to see that when u is reasonably large, it pays off for the constant cost in decider quickly. For instance, set $m = 2^{20}$, when $u > 20$, it already makes even for $3um > 8 \times (2^{23.5} + m)$. When $u = 1000$, our scheme's total cost is about 73% of the ProtoStar scheme.

In Mangrove [77], the cost of $|T|$ is directly distributed into the chunk computation. It might work in our context where the chunk size is big and the number of steps of big, thus the amortized cost compared with step-wise circuit being folded is small. Its total cost is:

$$4um+4T/u*u+(8u+8T/u)*u+8(T/u+3m) \approx 4um+8u^2+12T$$

Details: when folding, the extra witness length has to cover $\vec{h}$, $\vec{g}$, $\vec{m}$ and the part of table T entries distributed (this is because in its secret witness it has to allocate the chunk of corresponding T entries to avoid switch-case logic on big T contents itself in the circuit). Thus they incur extra length: $m + m + T/u + T/u$, and given NOVA cost of 2MSM, it is: $4m + 4T/u$.

In the recursive (verifier) circuit, given public input i, and given the commitment to the T chunk, the circuit has to use a lookup (or membership) argument itself to verify that this is correct (then the folded argument will be verified in the final decision proof). So this needs an O(u) circuit, which translates to 8u group operations. Also to compute the commitment, it needs 8T/u.

For the final decider proof, it needs to verify (1 time) the folded relation for: (1) computing of the commitment of combined T chunk of T/u size; (2) combined relation to compute $\vec{h}$, $\vec{v}$ and $\vec{m}$ (chunked), however, unlike the ProtoStar and our case, it's size m. In total, this incurs one time cost of $8 \times (T/u + 3m)$. In summary: when $um < T$, our zk-cq approach is better. When $T < um$ and $u < m$, the Mangrove approach is better.

7.11 Dynamic and Small In-Circuit Lookups

There are various situations that we need to reason about the range of witness elements, e.g., range of source/destination states and char width (4-bits). They can be processed conveniently with low cost using lookup arguments. There are also cases that "dynamic" lookup arguments are needed when the lookup table itself is not fixed, e.g., for proving one vector is a subset of another. In this case, there are many choices of encoding available for in-circuit lookups: plookup [50] and its variation Halo2 lookup [95], Hab0ck's scheme [58]. In terms of "extra" witness variables they introduced (thus impacting the folding cost of 2MSM, which is based on the number of witness variables), letting m and n be the query and lookup table size, the cost are (**TO VERIFY LATER**): $3(m+n)$, $3m+n$, $2m+n$, respectively. In this case, we implement in-circuit lookup using Haböck scheme [58]. For In-Circuit lookups, they are encoded as part of the step-wise R1CS, thus, does not cause any much extra considering decider cost is amortized by steps.

There is still one problem regarding: small range proofs. Shall we do in-circuit lookups or cq? Given that both small (dynamic) lookup and externdal CQ lookup both exists. If we

[17] Here we ignore the speed-up caused by MSMs for simplicity of discussion.

do range proofs via in-circuit lookup, it addes the folding cost: $2m+n$ (Haböck) per step, thus totaling: $u \times 2 \times (2m+n) \approx 6um$, For CQ lookup, it actually adds "nothing", because we do not add any additional witness, however, we do have to pay the final cost for all of them: $8um$, but given that it's actually 2 column table, we are actually paying $16um$, the in-circuit wins (for range proof).

7.12 Implementation Consideration

There are several choices of encoding schemes, e.g., Circom [75]. However, considering that we are instrumenting the standard NOVA and R1CS model, using Arkworks library [6] and instrumenting the Sonobe library seems a more convenient choice for us.

7.13 Supporting Batch File Discharging

Consider the scenario that there are many files of the same size category (e.g., over $100k$ emails of size 2^{16}). We would to first leverage folding to speed up prover cost, and generate one proof to prove that all of them are free of malware. Also, for each file commitment, we provide a membership proof which shows that it belongs to the commitment to the entire file collection. Thus, for an individual file, its proof consists of 2 parts, and we would like both to be $O(1)$.

We first need a scheme for "commitment of commitments". There are several alternatives available, e.g., AFGHO commitment used in generalized inner product argument [24], which has log size proof. Another potential solution is the bivariate poly commitment used in Pianist [72], however, we do not see a solution provides membership proof better than $O(mn)$ (where m is the number of words and n is the max length of each word).

We use the solution below. The prover precomputes all $[f_i(s)]_1$ as Pedersen vector commitment $\sum_{i=1}^{n} f_{ij}[\lambda_i(s)]_1$. Each such commitment can be written as (x_i, y_i) over $\mathbb{F}_q$. Then the prover can compute similarly commitment to $\vec{x}$ and $\vec{y}$, let them be $\texttt{com}(\vec{x})$ and $\texttt{com}(\vec{y})$. It is known that to "mass-produce" the membership for all x_i belonging to $\texttt{com}(\vec{x})$ is $O(m\log(m))$ (basically the batch KZG proof generation used by CQ [42] using [48]). So, each membership proof is $O(1)$, and the amortized cost for each proof is $O(\log(m))$. Note that the commitment has to be performed over BN254. If it's on Grumpkin, since there is no pairing available, it needs inner product argument (IPA) like argument, and this would need log size proof.

We then need to argue that at each step i, it is indeed using (x_i, y_i), a point in G1. At each step, the step-circuit can compute $\lambda_i(r)$ based on i and r and the Barcentric Langurange poly value $l(r)$. The weight factor w_i of Barcentric can be stored as a small lookup table. Then each step-circuit can compute $\lambda_i(r)$ in $O(1)$ time.

Each step circuit has x_i as secret witness input. It applies and accumulates $x_i\lambda_i(r)$ (passing the partial sum as z_i to next circuit). So the final output contains $\sum_{i=1}^{m} x_i\lambda_i(r)$. Similarly can be done for the y_i series. Then the verifier can check the value of $\sum_{i=1}^{m} x_i\lambda_i(r)$ via a KZG proof. This proves that the (x_i, y_i) at each step is indeed the ones contained in the entire commitment $\texttt{com}(\vec{x})$ and $\texttt{com}(\vec{y})$.

There is one problem though, the field arithmetic of (x_i, y_i) is non-native. So like Sonobe for group element operation, this is ported to the Grumpkin curve: it has a separate R1CS relation, which takes: x_i, y_i, w_i, $l(r)$ and performs the calculation, but the look-up of w_i is performed over BN254. This circuit is much smaller than the group element exponentiations and only costs < 20 R1CS constraints.

Then there is a one time in the decision proof which similarly verifies the non-native R1CS relation over BN254. But much smaller than the group operation cost.

TODO: give concrete cost..

7.14 zk-CQ and Integration

Use zk-CQ only for SINGLE instance (with no folding) lookup argument. And this saves most of the cost and has a good justification, and there is no other solution good than it for small circuit size.

For foldig, simply use Mangrove approach. Here, as mentioned earlier, the segment of table T needs to be stored separately. Then, we need to verify the segment commitment is commensurate with that part of the secret input. This membership proof can be done similarlty to the argument of word membership belong to a large collection. Then we are accumulating as step input and output.

7.15 Handle State In and Output

For very large files, we have to split into multiple steps, especially for side. Then there are state information to pass between steps, however, we want to save the hash operation.

Let's say, we reserve a special segment out of witness as a state output and also another segment as output. Then each step circuit outputs a Commitment to that segment. The next step takes it in, and verifies that its segment of input matches that of the preivous output. This allows to pass LARGE state, and the verification relation can be folded and is ONLY needed to be performed once in the final decision proof. Also the commitment is not restricted, as long as it is homomorphic, e.g., KZG or Pedersen vector commitment, or maybe even coding based or lattice based as shown in LatticeFold.

More formally, let $\vec{w}$ be a vector of witness and let σ be map so that $\sigma(\vec{w})$ is a fixed sub-vector of $\vec{w}$ given a fixed list of indices (e.g., given $\vec{w}$ 100 elements, σ retrieves the fixed list of 1st and 50th elements from it and forms a 2-element sub-vector.) Given the same σ, and let $\vec{w}$ and $\vec{v}$ be two vectors of the same length (or even different length but allowing valid σ). It is not hard to see that, given a verifier sampled challenge r, $\texttt{com}(\vec{w}) + r\texttt{com}(\vec{v})$ encodes a vector $\vec{w} + r\vec{v}$ and $\texttt{com}(\sigma(\vec{w})) + r\texttt{com}(\sigma(\vec{v}))$ encodes $\sigma(\vec{w} + r\vec{v})$.

Using the homomorphic property, in the committed R1CS relation, we also disclose $\texttt{com}(\sigma(\vec{w}))$ (with $\texttt{com}(\vec{w})$. In the final decision, we just need to perform the check of σ once. And there is only one additional group operation in recursion overhead. Using this approach, we can pass large state variables

between states. **TO CONFIRM:** this seems MUCH less expensive than maintaining a RAM memory as in zk-EVM machines, because we do not maintain the RAM for evyer single EVM instruction, but for large chunk of step-wise R1CS.

7.16 Summary on Folding Scheme

In summary we need to modify the Sonobe framework for: (1) additional elements for supporting word colleciton (2) to support state variables.

The development work will: (1) support individual file verification (2) fold large collections of small emails (3) support state variables in large circuits, and non-uniform circuits.

8 PM-REG MODELING

We now discuss how to prove for a pattern r_i in PM-Reg, how does one argue that a committed string s does not contain the patter r_i.

Our starting point will be again an AC-DFA. We collect all patterns from a PM-Reg pattern. Our first gadget would be to provide to Perdersen vector commitments $\mathsf{C_s}$, $\mathsf{C_i}$, which encodes the final state and their index along the acceptance path.

We then need to encode the patten constraints over length. One can see that a PM-Reg is a one level boolean combination of sub-signatures, and each sub-signautre is a sequence of fixed words and patterns like $.*$ and $.\{5,6\}$. Given a $(state, pos)$ sequence, one can assert that if it matches a sub-signature by checking arithmetic constraints. We illustrate the concept using an example.

Example 8.1. Let `abc` and `def` and `gh` be three patterns and their corresponding states be s_1, s_2, and s_3. Let a sub-signature $r :=$ `abc`.$\{3,5\}$`def`.$\{4,6\}$`gh`. Then given a string $s :=$ `ccabc1234def123gh`. Running through the ACDFA which got state pairs $(s_1, 4), (s_2, 9), (s_3, 14)$, one can see that s does not satisfy the regular sub-signature, because it does not satisfy the constraints that in between s_2 and s_3 there are 7 to 9 characters.

Formally, the algorithm works as the follows. Given a subsignature it extracts the corresponding state sequence, and after each state, there is a range, e.g., $[3, 5]$ which determines the position of the next matching state. At each step, the algorithm receives a set of positions for the corresponding state (e.g., initially $\{5\}$ for state s_1), then it leads to allows ranges $[6, 8]$ for the appearance of second state s_2 in the sequence. The algorithm then looks through the state position pairs and finds the allowed ones. Thus, for the 2nd step, it is $(s_9, 9)$. Then for the third state s_3, the allowed range is $[15, 17]$. Then, the $(s_3, 14)$ does not fall into the range, thus proving that $s \in L(r)$.

So now the problem boils down to: given a sequence of indices, e.g., $[1, 3, 5]$, we want to generate its subset will falls into a set of ranges e.g., $[2, 4], [5, 7]$ (which generates $[3, 5]$). One could actually mark up the sorted array of these two lists appropriately:

r_1	1234557
r_b	0100100
r_e	0001001
r_m	0111111
r_s	1010010
r_o	0010010

Here r_1 represents the sorted merged list of numbers, r_b (boolean) indicates the the beginning dices, and r_e stands for the ending indices of ranges. Note that r_b and r_e can be checked by finger printing techniques (if it's consistent with a subsignature pattern). Then r_m marks all elements that are in some ranges, this can be accomplished by checking formula that $r_m[i] = 1$ if $r_e[i] \neq 1$ and its previous element is 1 or $r_b[i-1] = 1$.

r_s represents the index of elements in input sequence. r_o represents the output , which is a bit-wise conjunction between r_s and r_s.

Altogether the prover complexity is linear over the sum of the number of states and patterns. Given that the average of final states is $6.6k$, the concrete cost is $10\times$, i.e., $66k$. Given max 50 patterns, this is 3 million constraints max and for average 4 patterns, this is only $240k$.

8.1 Boolean Combination of Subsignatures

A regex pattern is a boolean combination of sub-signatures, in the CNF form. Then each conjunction can be represented by a bit pattern, e.g., assuming 5 subsignatures in max, it consists of 10 bits. These bit-patterns (converted to field elements) are stored in a lookup table. At prover time, the prover feeds these bit-patterns for a PM-REG pattern in a separate advice column, and they are verified using lookup argument. One field element can store 10 20-bit patterns and this is sufficient for our application.

Now given a CNF pattern $\mathbf{a}$ and the corresponding subsignature evaluation $\mathbf{b}$, the evaluation could be defined as the result $c = \prod_{i\in[10]}(\mathbf{a}_i = 0 \wedge 1) \vee (\mathbf{a}_i = 1 \wedge \mathbf{b}_i)$.

8.2 Question:

Do a global optimizer or using gadgets in halo2?

9 IMPLEMENTATION AND EVALUATION

9.1 Implementation Details

System Implementation The `ZkregPlus` system contains three parts: (1) a parser of ClamAV regex signatures, (2) the FoldPot framework which provides folding and zkSNARK, and (3) a collection of circuits that implements streamed CP, SED, and DFA discharging algorithms. The system is implemented in Rust, consisting of around *to check (30k)* lines of code, and its source code is publicaly available on GitHub. [18]

[18] http://github.com/xxxx

Sub-Lookup Table	Size
(1) range table1 $[0, 16]$	16
(2) range table2 $[0, 2^{23}]$	16
(3) AC-DFA for CP (regular)	2000
(4) AC-DFA for CP (ignore case)	2000
(5) AC-DFA for SED (regular)	2000
(6) AC-DFA for SED (ignore case)	2000
(7) 16 (hard) DFAs	2000
(8) signature DB (e.g., subsig-step-pattern relations)	2000
(9) tri-logic computation table	2000

Table 1: Breakdown of 2-column Lookup Table (TO FILL DETAILS)

The regex parser supports ClamAV's own hex signature format, and as well as full PCRE regex specificaton. We rely on the `regex_syntax` crate of Rust to parse PCRE regex into a high-level intermediate representation, from where, either AC-DFAs are constructed for sub-signatures, or DFA built using the `Rustomaton` crate[19], or SED related information are generated. The signature related information (e.g., transition table) are encoded into a two-column global lookup table. Table 1 shows the break-down of the lookup table which has 2^{28} entries.

The FoldPot framework is built over the Sonobe framework [1], extending its Nova + Cyclefold framework to SuperNova [64] and then we implement other components such as QA-NIZK [54], the Sigma-IR1CS protocol, scheme for distributing Logup lookup [58] checks into shares as suggested in Mangrove [77], and the cycle pairing framework as shown in Section A.1.1. We use Bn254 and Grumpkin curves [51] (s.t., Grumpkin's scalar field is the base field of Bn254), and instantiate hash functions using Poseidon [55]. Most crypto primitives and polynomial related functions are built over the Arkworks library [6]. We further developed an additional layer of relational database operations to support CP, SED, and DFA discharging algorithms, which heavily rely on lookups. We also made some minor improvements of the arkworks r1cs framework to speed up R1CS synthesis from constraint systems, so that folding can be speeded up. [20]

Experimental Setup All of our experiments are run on a Google Cloud Computing (GCP) xxyyzz model server with xxyyzz cores and XXYYZZ GB of RAM. The folding stages needs XXXXYY GB, and decider consumes all XXYYZZGB of RAM.

Evaluation Questions There are two basic essential research questions to answer:

- $\mathbf{Q_1}$: How effective is the approximation technique? In particular, What is the false positives generated by approximations?
- $\mathbf{Q_2}$: What is the performance of the proposed approach in terms of prover speed and proof size?

[19] https://crates.io/crates/rustomaton

[20] replacing the BTreeMap etc.

9.2 Evaluation of Approximation

To answer $\mathbf{Q_1}$, we evaluate the approximation algorithms using the real data set of entire ClamAV logical signatures. The set is comprised of 38876 signatures.

Fourteen signatures (14) are dropped from the original 38889 signatures, because after applying approximation they report false positives on some Linux executables or Enron emails which are benign. By "benign" we mean: when running clamav using the specific rule set, ClamAV does not report positive on matching virus signatures. The breakdown of these false positives are listed below. (i) Over approximation of enlarged target field. We do not handle the `Target` meta-data, which specifies to which categories of executables that the signature should apply to. In our approximation, we just ignore the target field and apply it all all. `Win.Trojan.FakeSkype` was targeted at Win32 PE, and it raises false positive `vmlinuz` (Linux kernel image). (ii) Over approximation program entry point position constraint ignored. For example, in `Win.Trojan.Generic-43; ...;EP+0:5250-85133c0;...`, the position constraint needs to locate executable file entry point. This exceeds regex ability, we approximate it with `EP+0:` with wildcard string `.*` before the patterns. However, such patterns are too short and causing false positives. (iii) Over approximation of multiline mode. We use multiline mode for all signatures, i.e., new line character is matched by wildcard dot. In this case, one signature (which works in single line mode) is removed for causing false positive due to multi-line approximation. (iv) Both DFA and SED are expensive. This applies to `Js.File.MaliciousHeuristic-6260249-0` where its signature is a concatenation of patterns "`...s(?P=junk)c(?P=junk)r(?P=junk)i(?P=junk)...`", where for SED, the pattern words are too short and it is too expensive to construct its DFA.

Among all remaining 38875 (99.96%) of the 38889 signatures, the vast majority have hex subsignatures and 559 of them have PCRE subsignatures. Six (6) of these PCRE signatures use features such as named back-references and look-arounds, and they are approximated.

We use two sets of target data set (to discharge free of malware). The first is a set of all binary executable files in a Linux CentOS (2702 files total 752 MB) and Enron Email Data Set (517401 files totally 2.74 GB). Running ClamAv to discharge all binary executables takes about 10 seconds, but much longer (4 hours) on Enron Data set (mainly due to file reading cost).

Given the cost of the proposed schemes, we discharge an input file following the order of `CP`, `BS`, `SDE`, `DFA`, and `ISDE`. Once a signature is discharged, it does not go to the next stage. At the last stage a count 0 means that a file is completely discharged of all virus signatures (thus malware free). **CHECK:** We summarize the asymptotic complexity of these methods. Let u be the number of signatures to discharge at a stage, their prover complexity are: $O(|s|)$, $O(|s|)$, $O(|s|)$, $O(u \times |s|)$, $O(u \times |s|)$, respectively. Note that for the first 3 stages, the prover complexity is *independent* of the number

ClamAV Logical Signatures (38886) TO UPDATE!!!				
Centos Binary Executables (2702 samples)				
log(file)	files	After CP ⇒	SDE ⇒	DFA ⇒
7	72	(87,90,72)	(0,0,0)	(0,0,0)
8	0	(0,0,0)	(0,0,0)	(0,0,0)
9	0	(0,0,0)	(0,0,0)	(0,0,0)
10	2	(96,101,2)	(0,0,0)	(0,0,0)
11	13	(96,103,13)	(0,0,0)	(0,0,0)
12	16	(98,105,16)	(0,0,0)	(0,0,0)
13	105	(98,105,105)	(0,0,0)	(0,0,0)
14	727	(105,119,727)	(0,1,2)	(0,0,0)
15	478	(104,162,478)	(0,2,1)	(0,0,0)
16	442	(107,163,442)	(0,2,70)	(0,0,0)
17	273	(116,164,273)	(0,2,6)	(0,0,0)
18	163	(122,183,163)	(0,2,7)	(0,0,0)
19	158	(131,191,158)	(0,2,60)	(0,0,0)
20	67	(145,201,67)	(1,3,55)	(0,0,0)
21	153	(185,202,153)	(1,3,146)	(0,0,0)
22	14	(150,173,14)	(1,3,11)	(0,0,0)
23	6	(175,235,6)	(3,4,6)	(0,0,0)
24	9	(224,277,9)	(2,4,9)	(0,1,1)
25	3	(226,270,3)	(3,4,3)	(0,1,2)
26	0	(0,0,0)	(0,0,0)	(0,0,0)
27	1	(285,285,1)	(4,4,1)	(0,0,0)
Enron Email Data (517401 samples)				
9	14025	(87,147,14025)	(0,0,0)	(0,0,0)
10	141762	(88,148,141762)	(0,0,0)	(0,0,0)
11	175664	(90,156,175664)	(0,0,0)	(0,0,0)
12	113795	(91,160,113795)	(0,0,0)	(0,0,0)
13	47882	(93,166,47882)	(0,0,0)	(0,0,0)
14	17421	(96,172,17421)	(0,0,0)	(0,0,0)
15	4681	(98,162,4681)	(0,0,0)	(0,0,0)
16	1331	(101,154,1331)	(0,0,0)	(0,0,0)
17	644	(104,164,644)	(0,0,0)	(0,0,0)
18	175	(110,177,175)	(0,0,0)	(0,0,0)
19	12	(113,119,12)	(0,0,0)	(0,0,0)
20	3	(107,108,3)	(0,0,0)	(0,0,0)
21	6	(105,110,6)	(0,0,0)	(0,0,0)

Figure 9: Preliminary Data

Column 1: the log of file size. Column 2: the number of file samples. For the rest of columns, each tuple (avg, max, count) represents the average and max number of signatures that can not be discharged by the approach. The count represents the number of files that need to go to next discharging stage.

of signatures to discharge. In our real data set, the u for the last two stages is less than 2.

In Figure 9 we display the effectiveness of the proposed 3-stage discharging scheme, over the binary executable data set and Enron data set. The Enron set performs similarly, while it relies more on SDE because Enron dataset are mostly alpha-numerical and there are many more "false positives" on patterns if no checks on structures are performed.

Take the 2^{20} file size category in CentOS dataset as an example:

log(file)	files	After CP ⇒	SDE ⇒	DFA ⇒
20	67	(143,198,67)	(1,2,54)	(0,0,0)

There are 67 files in the category of $1MB$ size. The first CP stages screens the vast majority (99.5%) of signatures and in average there are 143 (max 198) out of $38k$ signatures left. The next BS stages discharged in average 39 signatures, and the SDE discharged all of the rest except 1 (in average) and 2 (max) for 54 files. None of the signatures for $1MB$ files would have to go through to the last ISDE stage. For the entire CentOS dataset, in the worst case, we only have to discharge 7 signatures via DFA for one file, however, this happens very rarely. For both the CentOS and Enron dataset, there are only two signatures[21] that have to to through ISDE for 5 (out of 2702) binary executable files and 6 (out of 517401) Enron email files. In summary, given that for the last two stages the number of signatures is less than 2, the total prover cost is just a constant factor (≤ 5) of the input string size, i.e.,

$$5 \times |s|.$$

This is a speed-up of 7.7×10^6 compared with the state of the art ([73, 96])[22], which use NFA discharging each signature separately, where the average $|\texttt{NFA}| > 1000$ (given average length of ClamAV signature is 300):

$$38878 \times |s| \times |\texttt{NFA}|.$$

The speed-up comes from two sources. First, we "mass-process" large collections of signatures using ACDFA in the first three stages. Second, we rely on pre-processed lookup arguments (whose prover complexity is independent of the lookup/container table size) in proving DFA acceptance, thus eliminating the size of automata in the prover cost. Several different types of look-up arguments in used in our zk-prover scheme, which is demonstrated in details in Section 4.

9.3 Overall System Performance

We then try to answer the question what is the system performance?

(1) micro-bench mark: group and field operations (how many CPUs). (2) data size: all 2700 binary samples, chunked into 128k. i) Show the variety of configs for circuit setup (like different size for how many dfas, and the size of step-pattern stack). ii) Show an example of a sample circuit cost breakdown. cost of lkups, cost of circuit itself, and cost of support SED. (3) folding speed (4) decider step cost. i) show the breakdown of the decider circuit. ii) report the time and space consumption. iii) report the proof size breakdown.

9.4 Micro Benchmarks

9.5 Learn from Other Papers

(1) ZkMiddleBox: eval questions: baseline perf and optimization; case study, cost for real-world case. http firewalling, domain blocklisting. no comparative work.

(2) Zombie: eval questions: (1) overheads added by config of Zombie; (2) how does batching improve throughput? (3) costs of different components? (4) how effective are zombie regex?A

(3) Reef: overal performance (compilation, solving, proving, ver)A comparaive performance, different appraoches (dfa, dfa + recursion, safa + lookup),

[21] Js.File.MaliciousHeurstic-6260249-0 and Html.Exploit.CVE_2017_0069 -6013012-3

[22] In the case of Reef [5] it works very well for the non-membership proof case where the patterns start at fixed positions. However, if there is a .* preceding regex patterns, our technique works better. It is not known if Reef's alternating automata can be extended easily to mass process large collection of signatures.

10 RELATED WORK

10.1 Alternative Approaches for Encoding PM-Reg

We consider several alternative approaches for encoding PM-Reg. The first choice is to use a general proof system with the support of lookup-arguments.

HyperPlonk [36] improves Plonk by replacing the univariate polynomial quotient check with multi-variate polynomial commitment and evaluation proof using sumchecks. This increases the proof size to $\log(n)$, however, removes dependence on FFT (thus improving parallelism) and allows it to support high degree customized gates. However, like PlonkUp [80], it implements the Plookup protocl [50] which has prover complexity depending on the lookup table size. Another restriction of [50] and [80] is that the lookup table size cannot exceed the circuit size, which in our case is a much larger value.

10.2 Folding and Proof Recursion

Detailed comments: **Accumulation:** Halo [19] uses cyclic curves (but without pairing) to achieve low cost proof recursion (however it's recursive overhead is $\log(n)$). PCD [23]: It presentes the accumulator scheme to realize IVC (incremental verification). The basic idea is that an accumulator is an object of aggregated statement and proof, whose size does not increase with the number of (statement, proof) that are aggregated. A new accumulator can be generated from an old accumulator, and it recursively proves that all prior instances are correct. A final decider (which has the same verifier cost of verifying a single circuit) decides the validity of all n copies of the circuit. Compared with Nova (Folding) [68], it does not use an extended R1CS system, but does use a similar error term to cancel out the cross terms when accumulation is applied. It commits to $A \cdot z$, $B \cdot z$, $C\dot{z}$ and has a slightly higher concrete cost than Nova. ProtoStar [22] further extends the accmulation scheme, it accumulates/folds a running accumulator with a NARK (not necessarily) proof. It accomplishes the non-uniform circuit as SuperNova, and it provides better verifier circuit complexity using Hab22 [58] technique ($O(1)$ verifier complexity other than $O(|t|\log(|T|)$ of HyperNova. Note that its verifier circuit has only one more hash operation for the lookup (and others error terms computation have the same complexity of others, and embedded with other constraints). **However, notice that the cost of $|T|$ has to be paid in the last decider!** (It looks like if it sacrifices the no trusted-up, it can still be independent of table size). Note that due to extra long vectors of $\vec{m}$, these constraints cannot be incorporated into the standard columns of plonk system. For an extra of $6|t|$ group operations is needed on the prover side for each step of recursion of lookup tables, in addition to the standard cost of encoding columns. Origami [91] concurrently proposes a similar folding skeme as ProtoStar using error terms for AIR, and it applies to plookup (actually Halo2 lookup) by error terms of polynomial maps. Note that plookup table has to be the same as number of constraints (cannot handle large table). Its passing of randoms input is subsumed by commit-and-prove feature of halo2 (multi-stages) anyway. DeepThought [20] further instantiate GKR protocol over the ProtoStar framework. It makes two contributions: (1) GKR allows to commit to input and ouput wires only (thus not committing to all wires); The paper modifies GKR to bivariate to lower its cost in the ProtoStar framework (reducing rounds) (2) adapting the Hab22 technique, it can prove a *dynamic* memory table read/write operations (by permuting read/write records and also add addr/index into Hab22 formula), thus achieving constant recursion cost per mem op, just like lookup table in ProtoStar. *Altnertive solution: maybe permutation of memory transcript is good enough. Then for initial memory read and final memory write in a sorted list they all can be performed using CQ protocol anyway.* ProtoGalaxy [43] further cuts the recursion cost of extra 3 group operations from ProtoStar by the following trick: it folds multiple instances, and it uses a Lagrange polynomial to encode the error terms. So instead of folding instances using a random factor, it allows verifier to use a new random for the Lag based poly encoding all instances. This cuts the group operations (replacing with a log number of field operations) in the "marginal cost" (but note that linear combination of group elements of claims still have to be performed in the verifier circuit). In a very recent work [21], the accumulation scheme is achieved by symmetric crypto pritmitices. The basic idea is to replace the Pedersen commitment of witness vectors by a Merkle tree of an RS-code of witness. Then the homomorphic group operation is replaced by a protocol of verify code validity. This moves away from pub-key assumptions. For decider, its complexity is $n/\log(n)$, also we can avoid non-native field arithmetics and cycle curve, if we encode it in BN254? [27] furtuer improves [21] by preserving distance of RS-code. Recently Mira [13] extends ProtoStar by allowing group elements in the Special-Sound Protocol and simulate the running of pairing in SSP, and shows better performance than Groth16 proof packing (*However, not sure how the discrete logarithm of Groth16 proofs were obtained? Authors never responded*).

Folding Nova [68]: it presents the *folding* technique which folds two relaxed R1CS instance into one. The folding prover at each step pays two MSMs, the folding verifier pays two group scalar multiplications. When applied to IVC, the verifier circuit is embedded in the IVC proof recursively. The proof size is linear, but using a cyclic curve with sumcheck, the proof size can be compressed to log of circuit size using a variation of Spartan. Sangria [**?**] implements the same idea of NOVA by folding PLONK relations. Note that it is subsumed by ProtoStar. In [76]: a rigorous proof is given for the correctness of Nova when implemented over a cycle of curves and correcting one vulnerability in its implementation. SuperNova [64] further improves Nova to handle non-uniform R1CS instances, which allows the prover to pay cost only on instructions that is being executed. Given ℓ circuits, the idea is to maintain maintain ℓ instances of R1CS instances, but when applying folding, only fold the actual circuit being executed. Note that the IVC proof size is still linear to the total

of the ℓ circuit size, also when proving the circuit the prover has to maintain the memory for all ℓ circuits. HyperNova [66] further extends the folding idea to CCS (a combination of R1CS, Plonky, and AIR specification) which supports high digree gates. The idea is to model the system using multi-linear polynomials whose evaluation can be specified using sumcheck techniques, which results in verifier circuit of log size of field ops plus one scalar multiplication. The interesting part of this is that the more expressive the spec system, the easier the folding? The IVC proof size is still linear, and needs a cyclic curve for succinct proof. CycleFold [65] further improves the Nova scheme by not running a dual R1CS system over a cycle of curves, but simply outsourcing a few non-native field and group operations to a second cycle (e.g, BN254/Grumpkin). That second curve is folded back in the first curve. **Decider cost:** `sonobe` implements the NOVA + cyclefold. The final decider has to handle non-native field operations over BN254 (it first uses KZG commitment to pass values between two curves, but needs to evaluate non-native field operations of the poly over BN254, and then it eneds to encode the R1CS group multiplication circuit (on Grumpin) over BN254 as well. The two operations result in over 11 million constraints for $0.5million$ single-step circuit (hence the overhead of non-native is big, about 10 million constraints). In Goblin Plonk[23] non-native field operations are encoded as high-degree (over 30 columns) custom-gates (zk-VM machine), so the prover's number of constraints is cut down (but it does not cut down decider cost). Also it looks like it is not on one-curve (two proofs? but need to verify). Reverie [89] further improves CycleFold approach by mixing it with the ProtoGalaxy folding scheme (multiple instances) and gives detailed estimate of the cost. Its goal is to reduce the inter-prover communication, and it is estimated to cut the cost to around 4000 field elements (around 128kb), which is used for peer-2-peer network application scenarios (Etherum Consensus). In Mangrove [77], the folding and accumulation scheme is further extended to tree structure PCD. It divides a plonkish system into chunks (and using product of grand product to merge the proof for copy constraints). Thus it achieves very small memory print for provers. It includes an extension of Hab22 protocol in its chunking mechanism. However, it requires the lookup table size not exceeding the entire circuit size so that the lookup table (fragments) can fit in the memory of prover. Note that its Merkle tree structure of all commitments allows to provide Commit-and-Prove (since original Nova and other schemes fold witness commitments, hence not easy to take them out). Recently there are directions in further reducing the recursive overhead by removing the non-native arithmetics caused by the homomorphic commitment. One related needs to mention for Mangrove is Gemini [18], which also accomplishes space-saving snark via streaming polynomial commitment and inner product argument prover algorithms (thus achieving space efficiency). Since Gemini is still a general purpose working for a large circuit, Mangrove exhibits better conrete performance via folding compared with Gemini (about one magnitude). In SCRIBE [12], streaming is also applied to sumcheck based prover, where it is shown a magnitude improvement of speed improvement, and log size RAM access needed (mainly for a RAM memory for pre-computing evlauation of multi-variate Lagrange basis polynomials). It looks like an interesting direction to cut the RAM usage for our decider stage. In LatticeFold [17], the homomorphic commitment of witness is replaced by Ajtai's commitment. It enjoys the benefits of small field and post-quantum security. The main efforts aims at reducing the increasing norm of such commitments, and the depth of folding is limited to constant round, thus the general PCD tree structure should be used. NeutronNova [67] further applies folding to the sumcheck protocol via Lagrange interpolation. In Neo [78], an alternative scheme based on Ajtai's commitment. It uses sum-check protocol in verifier circuit to prove that elements are within norm bound and enjoys a claimed 10x speed over HyperNova.

Applications of Folding: Kaizen showed customized sumcheck based protocol, when combined with folding can prove large scale DNN training efficiently [2].

Comments: The currently known best folding cost for Hab22 based lookup argument is ProtoStar and its derivatives such as ProtoGalaxy and Mangrove. The folding cost seems minimal, but in the final decider the cost is *linear* to the lookup table. Even if the lookup table size is the same of total queries, using a SNARK e.g. Plonk will incur more prover cost than the commit-and-prove approach (e.g., 22MSM for Plonk). It is possible to mix ProtoStar decider SNARK with the right half of the CQ protocol, but essentially it still resolves to the commit-and-prove approach.

Comments: We extended Sonobe [1], which instantiates CycleFold + Nova, exploiting half-pairing Bn254/Grumpkin curves. Our extension extends it to SuperNova and similarly added a 24k constraints for pairing operation at Grumpkin. MicroNova [101] similarly uses half-pairing approach in the context of sum-check based provers (Spartan [86], which results in slightly higher proof size (using HyperKZG to reduce multi-linear polynomial commitments to batched KZG). MicroNova adopts a variety of techniques to reduce decider size, e..g, reducing one MLE evaluation over E2 by spliting it into multiple sub-matrix evaluation. We could adopt the same technique by spliting the pairing operation into multiple identical cycles, thus reducing decider size, which remain as a future direction.

Fiat-Shamir Recently it is shown that when circuit has white-box access to the hash function used for Fiat-Shamir transformations, there exists diagonization attacks [7, 61], where a proof-of-work remedy is given. The attack aims at *general purpose proof system* such as the GKR protocol which allows adversary prover to submit *any* circuit, where the adversary can provide a proof for false statement. In our design, the circuit does have white-box access to hash. However, our circuit is specific (not general purpose), and we take a security heurstic assumption that the circuit is set up by a trusted party (so that the prover does not have the ability to create circuit). To further mitigate the attack, we

[23] https://hackmd.io/@aztec-network/B19AA8812

use two *different paddings* when the real hash() is used in circuit, and when hash() is used for Fiat-Shamir. This reduces the likelihood that a malicoius adversary use some gates in the circuit to simulate the Fiat-Shamir transformation hashes. However, in the furture work, it is interesting to apply another layer of formal verification to prove that a circuit's gates cannot be leveraged to build the Fiat-Shamir transformation hash.

10.3 Recusion Depth

For knowledge extraction property of our FoldPot system, we have to use the same security heuristics adopted in the lines of Nova and SuperNova: assuming a predetermined the concrete and *constant* recursion depth. Although recently there is a concrete attack against coding based IVC's constant depth [26], there is no known attacks agains Nova series IVC schemes. In [37] it is shown that IVC based on straignline SNARKs enjoys unbounded recursion depth, but the analysis does not apply to our work. However in [**?**] Campenelli et al. show that if we seek binding property only instead of knowledge extraction, any snark (maybe with poly size extractor) suffice to support IVC with polynomial size. This fits our system goal very well to prove that the files behind a commitment conforms to (do not contain matches) of a collection of public regex patterns. We take this route to prove the soundness of our work. Another route (for recursion depth) problem is to structure the IVC as a tree-like PCD scheme, as suggested in [39].

Very recently, there are models such as open-and-sign oracle model [**?**] which proves the security of Hypernova (basic idea modeling oracle access in signed oracle model so that oracle function call can be modeled in circuit, AGM provides straight-line extractor to allow poly recursion depth). However, the model needs hardware instantiation. In [32, 33, 37] IVC is shown to enjoy unbounded recursion depth, but it works only for SNARK based IVC [16].

10.4 Lookups

Many past lookups .... including CQ. See survey [59].

Latest one: **Lasso** [88] which uses sumcheck protocol and fast multilinear poly commitment to large matrixes with sparse non-zero entries. It allows to represent (not materialize) a huge table (e.g., $[0, 2^{128}]$) by projecting a large table to multiple small tables (decomposable), essentially it's actually modeling a multi-linear polynomial functions (where smaller functions are modeled using small look-up tables). This fits very well with modeling truth table of all CPU instructions, and is applied in [9]. Note that the prover cost is still dependent on the container table: for Sona: $O(N)$ field operation snad $O(\sqrt(N))$ group operations. But verifier has logarithm complexity. It is shown to be at least 3 times faster than plookups. Recently in μ-seek [31], the `cq` protocol is modified with multi-linear polynomial commitment to natively support super-efficient lookups with zk-SNARKs that use multilinear commitments; a commit-and-prove construction is also proved by connection univariate and multi-variate polynomial commitments. The verifier complexity is logarithmic.

10.5 Regex Result

It is well known [85][Chapter 5.4] that a regex can be translated to a graph of $|3m|$ where m is the regex length. Then to decide if a string of size n belongs to the NFA can be implemented by a BFS search, thus resulting in $O(mn)$ complexity.

10.6 Other Regex Efforts

1. TO READ: ZK-email https://github.com/zkemail/zk-regex: According to Reef it converts to DFA and the constraint size is proportional to DFA size.

2. TO READ: halo2-regex: https://github.com/zkemail/halo2-regex. It is using look-ups, but table is inside circuit (cannot exceed circuit size). Also do not handle large collections of regex.

10.7 zk-EVM

Compared with general purpose zero knowledge virtual machines such as zk-EVM [69], our approach is lighter weight.

In Polygon zk-EVM [69], a general purpose microprocessor is supported. Lookups (plookup) is used link state machine traces (lookup up result produced in another state machine trace). A RAM machine state is maintained as a Merkle tree. In some sense, retrieving bag and ternary logic computation resembles executing micro-instructions on a zk-EVM, however, as our approach is customized, and it certainly is more expensive executing it on a general purpose zk-EVM microprocessor. In Cairo [53], a general purpose algebraic RISC architecture is supported. It simulates read-and-write memory using a memory trace expressed as a read-only dictionary structure (essentially involving range proofs for proving ascending order of memory traces and permutation arguments for relating logs and memory traces). Apparently such a RAM model is more expensive than our simple mode of passing data between "big" step-circuits. Also decoding general machine instructions is more expensive than a customized model tailored for the specific application domain. We do want to point out that, using zk-EVM can nicely solve the problem of decoding an executable file and finding its entry point, which is approximated by our work. In this case, how to integrate zk-EVM and customized zk-SNARK circuits would be an interesting follow-up work. Recently Nexus [74] further exploits parallelizable Hypernova [66] and distributed computing to maximize the computing power of provers. However, customized protocol still shows performance advantage for specific problems. For instance, its whitepaper reports that Nexus VM costs about $30k$ constraints per step, and implementing SHA256 usin the general purpose VM is still about 10^3 slower than a specialized circuit.

10.8 RAM

We use a global commitment and leverage cyclefold to pass states between steps in folding schemes. An alternative way

is to use a global RAM as in many variaations of zk-EVM. General RAM is very expensive, e.g., using routing network, RSA accumulator, Merkle trees and lookups. See [79] for a summary of related work. Our model is similar to the "permanent memory" category in [79] where there are two commitments to a region of memory and the prover needs to prove that a program, given the initial memory setting, leads to the final memory state as committed in the second commitment. In the latest work, which much improved the earlier work on memory transcript validation using Bézout's identity, ordered permutation leveraging [58], its complexity is $3N + 41A$ (where N is the capacity of RAM and A is the number of read/write accesses). In our case $A \approx N$, the general RAM approach is much more expensive, because there is a large co-efficient over A.[24]. In StackProofs [44], a similar approach to the Mangrove commit-and-fold is presented, for reasoning about global memory, which is based on [58], and it has similar asymptotic compleixty and we estimate it has similar concrete performance. In Nebula [8], commit-and-fold is also used to support offline RAM verification, improving the state of art of Mangrove by allowing RAM check in IVC proofs (avoiding generating public verifier challenge over commitments of all witness at all steps), it also provides an alternative switchboard approach to SuperNova's approach in supporting non-uniform circuits, although enjoying the same asymptotic complexity. Coral [4] was built over Nebular for IVC proof of Context-Free Grammar.

In HEKATON [84], a parallel proof scheme is proposed (note: not folding). It shares a similar observation with Mangrove that a long product for finger-printing can be split among multiple circuits. The key idea is to split a circuit into multiple sub-circuits. The *shared wires* between sub-circuits are modeled as a global ROM. The memory trace of ROM access is modeled using finger-printing (and commit and prove snark systems). It improves the concrete performance of ROM (compared with Mirage+) to 13× the size of ROM. Notice that however, our approach (in our context) for sharing incurs only 1× of constraints for I/O memory buffer, because we do not have to reason about timestamp and trace consistency. *SIDENOTE to remove later: compared with Mangrove the HEKATON approach pays the extra cost for inter-circuit I/O, if the I/O buffer is large, it affects concrete performance.*

In Twist and Shout [87], the memory checking for read/write memory is further improved to only $4T$ and $6T$ commitments for small and large memory (where T is the trace size), by leveraging one-hot encoding of addresses and incremental updates for taking advantage of localicty. The technique does not have a direct application into our framework given its sumcheck technique results in log-size proof.

In the very recent work [41] memory checking is accomplished based on CQ lookup. The idea is that a base table is prepreprocessed using CQ, and m operations over a deviated table with hamming distance δ can be achieved in complexity $O((m+\delta)\log(m+\delta))$. Then preprocessed CQ keys are generated periodically, achieving amortized memory access complexity of $O(m\log(m)+\sqrt{mN})$ per batch. Concretely, for 2^{20} RAM, 10^3 access costs about 10^2 seconds (thus 10 access per second). This cannot satisfy our demand of efficiency.

One recent work zkPi [70] depends on random circuit approach Mirage[63], which also shows the power of the random circuit. (Some of the interesting point, the theoretical complexity for non-random circuit to prove permutation is nlog(n), but rand circ is O(n), and the analysis of collision for soundness, e.g., $n^3/|F|$).

[24] Interestingly, the use of Bézout's identity and the use of CP-Snark in [79] is very similar to our earlier work [83]

10.9 Related in Field

Compare with zk-MiddleBox, Zombie [96] and Zk-regex [73] and Reef [5]. Coral [4] (there are several CFG work).

A related line is the TLS Oracle problem where given an TLS session (encrypted), one proves the property of the data (e.g., correct generation of authentication key etc.). Traditionally this was done via MPC e.g., [46, 92, 97] and recently done using zkSNARK in Origo [47]. Our work can be used to prove TLS data with rich set of properties, such as redacted emails, or compliance with security policies without revealing encrypted data, efficiently.

Coral [4]: The key idea Coral (to support zk-proof about CFG) is built upon Nebula [8] an off-line meomry checking technique that works with IVC. Its basic idea is to store CFG grammar in public ROM, and parse tree in private ROM, and stacks in private RAM. Given the "global" commitment to all memory traces (very similar to our first pass of commitment in first pass, adopting Mangrove [77]) approach, its finger-printing hash greatly reduced the cost. The difference in our scheme is that: we heavily rely on external lookup tables (as our database of regex is so large, and encoding them in ROM is not efficient). Compared with Coral, our benefits is that the signature set in our data is three magnitudes larger than Coral. Coral's benefit is the expressive power in handling CFG.

11 CONCLUSION

Acknowledgement: We thank Dr. Ariel Gabizon for introducing and discussing varieties of lookup argument and plonkish systems for this work. We thank Dr. Ahbiram Kothapalli for explaining supernova system and the discussion over the depth bound of supernova.

REFERENCES

[1] 0xPARC and PSE. 2024. sonobe framework. *available at* https://github.com/privacy-scaling-explorations/sonobe.
[2] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. 2024. Zero-Knowledge Proofs of Training for Deep Neural Networks. In *CCS*. 4316–4330. https://doi.org/10.1145/3658644.3670316
[3] S. Abiteboul, R. Hull, and V. Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
[4] Sebastian Angel, Sofía Celi, Elizabeth Margolin, Pratyush Mishra, Martin Sander, and Jess Woods. 2025. Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs. *Cryptology ePrint Archive* (2025).
[5] S. Angel, E. Ioannidis, E. Margolin, S. Setty, and J. Woods. 2023. Reef: Fast Succinct Non-Interactive Zero-Knowledge Regex Proofs. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1886.
[6] arkworks contributors. 2022. `arkworks` zkSNARK ecosystem. https://arkworks.rs
[7] Gal Arnon and Eylon Yogev. 2025. Towards a white-box secure Fiat-Shamir transformation. In *Annual International Cryptology Conference*. Springer, 27–56.
[8] Arasu Arun and Srinath Setty. 2024. Nebula: Efficient read-write memory and switchboard circuits for folding schemes. Cryptology ePrint Archive, Paper 2024/1605. https://eprint.iacr.org/2024/1605
[9] A. Arun, S. Setty, and J. Thaler. 2023. Jolt: Snarks for virtual machines via lookups. In *EUROCRYPT*. 3–33.
[10] T. Atteta, S. Fehr, and M. Klooß. 2022. Fiat-Shamir Transformation of Multi-Round Interactive Protocols. In *TCC*. 113–142.
[11] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. 2003. *Constructing Elliptic Curves with Prescribed Embedding Degrees*. Springer Berlin Heidelberg, 257–267. https://doi.org/10.1007/3-540-36413-7_19
[12] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, Karan Newatia, and Steve Wang. 2024. Scribe: Low-memory SNARKs via Read-Write Streaming. Cryptology ePrint Archive, Paper 2024/1970. https://eprint.iacr.org/2024/1970
[13] Josh Beal and Ben Fisch. 2024. Mira: Efficient Folding for Pairing-based Arguments. Cryptology ePrint Archive, Paper 2024/2025. https://eprint.iacr.org/2024/2025
[14] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *EUROCRYPT*. 103–128.
[15] D. Bernhard, O. Pereira, and B. Warinschi. 2012. How not to prove yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios. In *ASIACRYPT*. 626–643.
[16] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. 2013. Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data. In *STOC*. 112–120.
[17] D. Boneh and B. Chen. 2024. LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2024/257.pdf.
[18] J. Bootle, A. Chiesa, Y. Hu, and M. Orru. 2022. Gemini: Elastic SNARKs for Diverse Environments. In *Eurocrypt*. 427–457.
[19] S. Bowe, J. Grigg, and D. Hopwood. 2019. Recursive Proof Composition Without a Trusted Setup. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2019/1021.
[20] B.ünz and J. Chen. 2024. Proofs for Deep Thought: Accumulation for Large Memories and Deterministic Computations. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2024/325.
[21] B.ünz, P. Mishra, W. Nguyen, and W. Wang. 2024. Accumulation without Homomorphism. *available at* https://eprint.iacr.org/2024/474.pdf.
[22] B. Bünz and B. Chen. 2023. Protostar: Generic Efficient Accumulation/Folding for Special-sound Protocols. IACR Cryptol. ePrint Arch..
[23] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. 2020. Proof-Carrying Data from Accumulation Schemes. IACR. ePrint Arch 2020/499.
[24] B. Bünz, M. Maller, P. Mishara, N. Tyagi, and P. Vesely. 2021. Proofs for Inner Pairing Products and Applications. In *ASIACRYT*. 65–97.
[25] Vitalik Buterin. 2018. On-chain scaling to potentially 500 tx/sec through mass tx validation (2018).

[26] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. 2024. Accumulation without Homomorphism. Cryptology ePrint Archive, Paper 2024/474. https://eprint.iacr.org/2024/474

[27] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. 2024. Arc: Accumulation for Reed–Solomon Codes. Cryptology ePrint Archive, Paper 2024/1731. https://eprint.iacr.org/2024/1731

[28] J. Camenisch and M. Stadler. 1997. Efficient Group Signature Schemes for Large Groups. In *CRYPTO (LNCS 1296)*. 410–424.

[29] M. Campanelli, A. Faonio, D. Fiore, T. Li, and H. Lipmaa. 2023. Lookup Arguments: Improvements, Extensions and Applications to Zero-Knowledge Decision Trees. https://hal.science/hal-04234948/document.

[30] M. Campanelli, A. Faonio, D. Fiore, A. Querol, and H. Rodriguez. 2021. Lunar: A Toolbox for More Efficient Universal and Updatable zkSNARKs and Commit-and-Prove Extensions. In *ASIACRYPT*. 3–33.

[31] M. Campanelli, D. Fiore, and R. Gennaro. 2024. Natively Compatible Super-Efficient Lookup Arguments and How to Apply Them. https://eprint.iacr.org/2024/1058.pdf.

[32] Matteo Campanelli, Dario Fiore, and Mahak Pancholi. 2025. On Composing AGM-Secure Functionalities with Cryptographic Proofs: Applications to Unbounded-Depth IVC and More. *Cryptology ePrint Archive* (2025).

[33] Matteo Campanelli, Dario Fiore, and Mahak Pancholi. 2025. When Can We Incrementally Prove Computations of Arbitrary Depth? *Cryptology ePrint Archive* (2025).

[34] M. Campanelli, D. Fiore, and A. Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. In *CCS*. 2075–2092.

[35] M. Campanelli, D. Fiore, and A. Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. IACR Cryptol. ePrint Arch. 2019:142.

[36] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *EUROCRYPT*. 499–530.

[37] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. 2024. Security Bounds for Proof-Carrying Data from Straightline Extractors. In *Theory of Cryptography Conference*. Springer, 464–496.

[38] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. P. Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS.. In *EUROCRYPT*. 738–768.

[39] Alessandro Chiesa and Eran Tromer. 2010. Proof-Carrying Data and Hearsay Arguments from Signature Cards.. In *ICS*, Vol. 10. 310–331.

[40] A. Choudhuri, S. Garg, A. Goel, S. Sekar, and R. Sinha. 2023. SublonK: Sublinear Prover Plonk. https://eprint.iacr.org/2023/902.

[41] Moumita Dutta, Chaya Ganesh, Sikhar Patranabis, Shubh Prakash, and Nitin Singh. 2024. Batching-Efficient RAM using Updatable Lookup Arguments. In *CCS*. 4077–4091. https://doi.org/10.1145/3658644.3670356

[42] L. Eagen, D. Fiore, and A. Gabizon. 2022. cq: Cached quotients for fast lookups. IACR Cryptol. ePrint Arch..

[43] L. Eagen and A. Gabizon. 2023. ProtoGalaxy: Efficient ProtoStar-Style Folding of Multiple Instances. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1106.

[44] L. Eagen, A. Gabizon, M. Sefranek, P. Towa, and Z. Williamson. 2024. stackproofs: Private proofs of stack and contract execution using ProtoGalaxy. https://eprint.iacr.org/2024/1281.pdf.

[45] Liam Eagen, Ariel Gabizon, Marek Sefranek, Patrick Towa, and Zachary J. Williamson. 2024. Stackproofs: Private proofs of stack and contract execution using Protogalaxy. Cryptology ePrint Archive, Paper 2024/1281. https://eprint.iacr.org/2024/1281

[46] J. Ernstberger, J. Ernstberger, A. Finkenzeller, and S. Steinhorst. 2024. Janus: Fast Privacy-Preserving Data Provenance for TLS 1.3. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2024/447.

[47] J. Ernstberger, J. Lauinger, Y. Wu, A. Gervais, and S. Steinhorst. 2024. ORIGO: Proving Provenance of Sensitive Data with Constant Communication. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1377.

[48] D. Feist and D. Khovratovich. 2023. Fast Amortized KZG Proofs. IACR Cryptol. ePrint Arch..

[49] A. Gabizon, Z. Williamson, and O. Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2019/953.

[50] A. Gabizon and Z. J. Williamson. 2020. Plookup: A simplified polynomial protocol for lookup tables. IACR Cryptol. ePrint Arch..

[51] Ariel Gabizon, Zachary J. Williamson, and Tom Walton-Pocock. 2021. *Zatec Yellow Paper*. Technical Report. Aztec Lab. https://hackmd.io/@aztec-network/ByzgNxBfd

[52] N. Gailly, M. Maller, and A. Nitulescu. 2022. SnarkPack: Practical SNARK Aggregation. In *FC*. 203–229.

[53] L. Goldberg, S. Papini, and M. Riabzev. 2021. Cairo - a Turing-complete START-friendly CPU architecture. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2021/1063.pdf.

[54] A. González, A. Hevia, and C. Ràfols. 2015. QA-NIZK Arguments in Asymmetric Groups: New Tools and New Constructions. In *ASIACRYPT*. 605–629.

[55] L. Grassi, D. Kales, D. Khovratovich, A. Roy, C. Rechberger, and M. Schofnegger. 2021. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In *USENIX Security*.

[56] J. Groth. 2016. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*. 305–326.

[57] P. Grubbs, A. Arun, Y. Zhang, J. Bonneau, and M. Walfish. 2022. Zero-Knowledge Middleboxes. In *USENIX Security Seposium 2022*. 4255–4272.

[58] U. Habock. 2022. Multivariate lookups based on logarithmic derivatives. IACR Cryptol. ePrint Arch..

[59] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. 2025. SoK: Lookup Table Arguments. *Cryptology ePrint Archive* (2025).

[60] A. Kate, G. M. Zaverucha, and I. Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *ASIACRYPT*. 177–194.

[61] Dmitry Khovratovich, Ron D Rothblum, and Lev Soukhanov. 2025. How to prove false statements: Practical attacks on fiat-shamir. In *Annual International Cryptology Conference*. Springer, 3–26.

[62] A. Kosba, Z. Chao, A. Miller, Y. Qian, T. H. Chan, C. Papamanthou, R. Pass, and a. shelat. 2015. *C*OO: A Framework for Building Compososable Zero-Knowledge Proofs. *Cryptology ePrint Archive. 2015/1093*.

[63] A. Kosba, D. Papadopoulos, C. Papamanthou, and D. Song. 2020. MIRAGE: Succinct Arguments for Randomized Algorithms with Applications to Universal zk-SNARKs. In *USENIX Security*.

[64] A. Kothapalli and S. Setty. 2022. SuperNova: Proving universal machine executions without universal circuits. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2022/1758.

[65] A. Kothapalli and S. Setty. 2023. CycleFold: Folding-scheme-based Recursive Arguments Over a Cycle of Elliptic Curves. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1192.

[66] A. Kothapalli and S. Setty. 2024. HyperNova: Recursive arguments for customizable constraint systems. CRYPTO. 345–379.

[67] Abhiram Kothapalli and Srinath Setty. 2024. NeutronNova: Folding everything that reduces to zero-check. Cryptology ePrint Archive, Paper 2024/1606. https://eprint.iacr.org/2024/1606

[68] A. Kothapalli, S. Setty, and I. Tzialla. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes.. In *CRYPTO*. 359–388.

[69] Polygon Lab. 2024. Polygon zk-EVM. *available at* https://polygon.technology/polygon-zkevm.

[70] Evan Laufer, Alex Ozdemir, and Dan Boneh. 2024. zkPi: Proving Lean Theorems in Zero-Knowledge. In *CCS*. ACM, 4301–4315. https://doi.org/10.1145/3658644.3670322

[71] H. Lee and J. H. Seo. 2024. On the Security of Nova Recursive Proof System. https://eprint.iacr.org/2024/232.pdf.

[72] T. Liu, T. Xie, J. Zhang, D. Song, and Y. Zhang. 2023. Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1271.pdf.

[73] N. Luo, C. Weng, J. Singh, G. Tan, R. Piskac, and M. Raykova. 2023. Privacy-Preserving Regular Expression Matching using Nondeterministic Finite Automata. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/643.pdf.

[74] D. Marin, M. Abdalla, P. Govereau, J. Groth, S. Judson, K. Sosnin, G. Policharla, and Y. Zhang. 2024. Nexus 1.0: Enabling Verifiable Computation. https://nexus-xyz.github.io/assets/nexus_whitepaper.pdf.

[75] M. Munoz, M. Isabel, J. Munoz-Tapia, A. Rubio, and J. Baylina. 2022. CIRCOM: A Robust and Scalable Language for Building Complex Zero-Knowledge Circuits. *IEEE Trans. on Dependable and Secure Computing* 20, 6 (2022), 4733–4751.

[76] W. Nguyen, D. Boneh, and S. Setty. 2023. Revisiting the Nova Proof System on a Cycle of Curves. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/969.

[77] W. Nguyen, T. Datta, B. Chen, N. Tyagi, and D. Boneh. 2024. Mangrove: A Scalable Framework for Folding-based SNARKs. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2024/416.

[78] Wilson Nguyen and Srinath Setty. 2025. Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Paper 2025/294. https://eprint.iacr.org/2025/294

[79] A. Ozdemir, E. Laufer, and D. Boneh. 2024. Volative and Persistent Memory for zkSNARKs via Algebraic Interactive Proofs. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2024/979.

[80] L Pearson, J. Fitzgerald, and H. Masip. 2022. PlonKup: Reconciling Plonk with plookup. IACR Cryptol. ePrint Arch. 2022/086.

[81] C. Peirce. 1909. Logic: autograph manuscript notebook. *available at* hollisarchives.lib.harvard.edu/repositories/24/digital_objects/63983.

[82] J. Posen and A. Kattis. 2022. CaulkPlus: Table-independent lookup arguments. IACR Cryptol. ePrint Arch..

[83] Michael Raymond, Gillian Evers, Jan Ponti, Diya Krishnan, and Xiang Fu. 2025. Efficient Zero Knowledge for Regular Language. In *Security and Privacy in Communication Networks (SecureCOM'2023)*. Springer Nature Switzerland, 369–394. https://doi.org/10.1007/978-3-031-64948-6_19

[84] M. Rosenberg, T. Mopuri, H. Hafezi, I. Miers, and P. Mishra. 2024. HEKATON: Horizontally-Scalable zkSNARKs via Proof Aggregation. https://eprint.iacr.org/2024/1208.pdf.

[85] R. Sedgewick and K. Wayne. 2011. *Algorithms 4e*. Addison-Wesley Professional.

[86] S. Setty. 2020. Spartan: Efficient and General-Purpose zkSNARKs without Trusted Setup. In *CRYPTO*. 704–737.

[87] Srinath Setty and Justin Thaler. 2025. Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105. https://eprint.iacr.org/2025/105

[88] S. Setty, J. Thaler, and R. Wahby. 2023. Unlocking the lookup singularity with Lasso. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1216.

[89] L. Soukhanov. 2023. Reverie: an end-to-end accumulation scheme from Cyclefold. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1888.

[90] S. Srinivasan, I. Karantaidou, F. Baldimtsi, and C. Papamanthou. 2022. Batching, Aggregation, and Zero-Knowledge Proofs in Bilinear Accumulators. In *CCS*. 2719–2733.

[91] A. Vark and Y. X. Zhang. 2024. Folding Custom Gates with Verifier Input. https://arxiv.org/abs/2401.11364.

[92] X. Xie, K. Yang, X. Wang, and Y. Yu. 2024. Lightweight Authentication of Web Data via Garble-Then-Prove. In *USENIX Security*.

[93] A. Zapico, V. Buterin, D. Khovratovich, M. Maller, A. Nitulescu, and M. Simkin. 2022. Caulk: Lookup Arguments in Sublinear Time. In *CCS*. 3121–3134.

[94] A. Zapico, A. Gabizon, D. Khovratovich, M. Maller, and C. Ràfols. 2022. Baloo: Nearly Optimal Lookup Arguments. IACR Cryptol. ePrint Arch..

[95] zcash. 2022. The Halo2 Book. https://zcash.github.io/halo2/concepts.html.

[96] C. Zhang, Z. DeStefano, A. Arun, J. Bonneau, P. Grubbs, and M. Walfish. 2023. Zombie: Middleboxes that Don't Snoop. IACR Cryptol. ePrint Arch. https://eprint.iacr.org/2023/1022.

[97] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels. 2020. DECO: Liberating Web Data using Decentralized Oracles for TLS. In *CCS*. 1919–1938.

[98] Y. Zhang, D. Genkin, J. Katz, D. Papadopoulos, and C. Papamanthou. 2017. A Zero-Knowledge Version of vSQL. IACR Cryptol. ePrint Arch. 2017:1146.

[99] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 863–880.

[100] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for outsourced databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1480–1491.

[101] Jiaxing Zhao, Srinath Setty, and Weidong Cui. 2024. MicroNova: Folding-based arguments with efficient (on-chain) verification. Cryptology ePrint Archive, Paper 2024/2099. https://eprint.iacr.org/2024/2099

[102] zkemail team. 2023. ZK Email. https://github.com/zkemail.

A FOLDPOT SNARK

The starting point of our framework is a combination of SuperNova [64] and CycleFold [65], and our implementation is based on deriving Sonobe [1], which implements Nova [68] plus CycleFold [65]. SuperNova supports non-uniform circuits, and cyclefold allows us to take advantage of half pairing groups like BN254/Grumpkin.

Our contribution in the framework is the introduction of a cycle pairing component in producing compressed decider proof. We observe that non-native arithmetics such as Pairing operations for BN254 can also be folded, by generalizing CycleFold. This allows us to use Quazi-linear NIZK (QA-NIZK) [34, 54] to encode the decider proof for many non-uniform circuits as pairing operations in circuit.

A.1 Folding Scheme: SuperNOVA + CycleFold

Let $\mathbb{G}_1$ and $\mathbb{G}_3$ be two curve groups (e.g., that of BN254 [11] and Grumpkin [51]) s.t. the scalar field of $\mathbb{G}_1$ is the base field of $\mathbb{G}_3$ and the base field of $\mathbb{G}_1$ is the scalar field of $\mathbb{G}_3$. We also assume bilinear pairing is supported on the curve for $\mathbb{G}_1$ (e.g., BN254), i.e. there exists $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerating bilinear pairing function. We use additive group notations from Groth16 [56]. We let $\mathbb{F}_r$ and $\mathbb{F}_q$ be the scalar and base prime field of $\mathbb{G}_1$ (i.e., base and scalar field of $\mathbb{G}_3$). For example, let $a, b \in \mathbb{F}_r$, then $[ab]_T = e([a]_1, [b]_2)$ denotes that the pairing of $[a]_1 \in \mathbb{G}_1$ and $[b]_2 \in \mathbb{G}_2$ results in $[ab]_T \in \mathbb{G}_T$.

A.1.1 CyclePair Gadget. We first briefly review CycleFold, based on which we develop CyclePair. The key observation of CycleFold is that: curve operations in $\mathbb{G}_1$ can be represented *natively* over $\mathbb{G}_3$, because $\mathbb{G}_1$'s point in the affine form, can be represented as two scalar field elements of $\mathbb{G}_3$. Let $k = 2\lambda$ where λ is the security parameter. Formally, the CycleFold relation is denoted as $\mathbf{R}_{\texttt{fold}}$ over $\mathbb{F}_q$, i.e., $\mathbf{x}$ (i/o) and $\mathbf{w}$ (witness) are vectors of $\mathbb{F}_q$ elements:

$$(\mathbf{x}; \mathbf{w}) \in \mathbf{R}_{\texttt{fold}} \text{ iff. } \begin{cases} \mathbf{x} = ([a]_1, [b]_1, [c]_1, r) \\ [c]_1 = [a]_1 + r[b]_1 \\ r \in \{0,1\}^k \end{cases}$$

In folding schemes like Nova [68], the committed instance of $\mathbf{R}_{\texttt{fold}}$ can be represented as an R1CS over $\mathbb{F}_r$. Thus in CycleFold, the system can fold the committed instances for both the main circuit and the cyclefold circuit *natively* over $\mathbb{G}_3$ and $\mathbb{G}_1$ respectively at reduced cost. In the following, we write $([a]_1, [b]_1, [c]_1, r) \in \mathbf{R}_{\texttt{fold}}$ for short (saving the notation of witness). Similarly, we can use $\mathbf{R}_{\texttt{pair}}$ to denote the *cycle pair* relation over $\mathbb{F}_q$:

$$(\mathbf{x}; \mathbf{w}) \in \mathbf{R}_{\texttt{pair}} \text{ iff. } \begin{cases} \mathbf{x} = ([c]_T, [a]_1, [b]_2, [d]_T) \\ [d]_T = [c]_T + e([a]_1, [b]_2) \end{cases}$$

Here we have one point in $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively and two points in $\mathbb{G}_T$, all can be represented as elements in $\mathbb{F}_q$ and the relation encodes one pairing and one $\mathbb{G}_T$ operation (natively). In practice it costs $12k$ R1CS constraints over $\mathbb{F}_q$.

A.1.2 Mixed R1CS. The use of cyclepair gadget allows encoding a mixed R1CS relation over two fields. We first give a motivating example for the use of such mixed R1CS.

In the decider proof for SuperNova, each non-uniform circuit has a commitment $\overline{\mathbf{W}^{(i)}}$ to its witness $\mathbf{W}^{(i)}$. The prover needs to demonstrate the knowledge of them, and each circuit has a separate Pedersen vector commitment key.[25] It is possible to batch-prove all $\mathbf{W}^{(i)}$ using QA-NIZK. However, we would like to perform the QA-NIZK verification in-circuit to further compress proof size, given that there is a large number of circuits. This would require *encoding bilinear pairing operations in circuit*, if performed non-natively over $\mathbb{G}_1$, will be very costly. We show how to accomplish it with mixed R1CS relation and $\mathbf{R}_{\texttt{pair}}$.

We know that any field element $x \in \mathbb{F}_q$ can be represented as a vectof of field elements in $\mathbb{F}_r$.[26] We denote it as $x_{\mapsto \mathbb{F}_r}$, and naturally apply it to a vector of $\mathbb{F}_r$ elements, and similarly to $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ elements (as they are can be represented as vectors of $\mathbb{F}_q$ elements).

Definition A.1. A *mixed R1CS relation* over $\mathbb{F}_r$ and $\mathbb{F}_q$ is a tuple $(\mathbf{x}^{(1)} \in \mathbb{F}_r^{m_1}, \mathbf{x}^{(2)} \in \mathbb{F}_r^{m_2}, \mathbf{w}^{(1)} \in \mathbb{F}_r^{n_1}, \mathbf{w}^{(2)} \in \mathbb{F}_q^{n_2}, R_1, R_2)$ for some $m_1, m_2, n_1, n_2 \in \mathbb{N}$ where R_1 and R_2 are two R1CS relations over $\mathbb{F}_r$ and $\mathbb{F}_q$ respectively, and $(\mathbf{x}^{(1)} || \mathbf{x}^{(2)}_{\mapsto \mathbb{F}_r}; \mathbf{w}^{(1)}) \in R_1$ and $(\mathbf{x}^{(2)}; \mathbf{w}^{(2)}) \in R_2$.

In this paper, we restrict R_2 to be $\mathbf{R}_{\texttt{pair}}$, but it can be generalized to any R1CS relation over $\mathbb{F}_q$. In the following, we show $\mathbf{R}_{\texttt{qanizk\_ver}}$, an instance of mixed R1CS relation for capturing the verification of QA-NIZK proofs.

Definition A.2. $\mathbf{R}_{\texttt{qanizk\_ver}}$ is a mixed R1CS relation:

$$\begin{pmatrix} \left(h^{(a)}, h^{(b)}, h^{(a')}, h^{(b')}\right), \\ ([c]_T, [a]_1, [b]_2], [d]_T), \\ R_1, \mathbf{R}_{\texttt{pair}} \end{pmatrix}$$

such that: $([c]_T, [a]_1, [b]_2, [d]_T) \in \mathbf{R}_{\texttt{pair}}$ and R_1 defines the following relation over $\mathbb{F}_r$:

$$h^{(a')} = \mathsf{hash}(h^{(a)}, [a]_{1 \mapsto \mathbb{F}_r}) \wedge h^{(b')} = \mathsf{hash}(h^{(b)}, [b]_{1 \mapsto \mathbb{F}_r})$$

One can chain many instances $\mathbf{R}_{\texttt{qanizk\_ver}}$ as step circuits to verify QA-NIZK relation below:

$$\sum_{i=1}^{n} e([\mathbf{x}_i]_1, [\mathbf{C}_i]_2) = e(\pi_{\texttt{nizk}}, [a]_2) \tag{4}$$

$$\wedge \quad h^{(x)} = \texttt{hashchain}(\mathbf{x}) \tag{5}$$

$$\wedge \quad h^{(C)} = \texttt{hashchain}(\mathbf{C}) \tag{6}$$

Here: QA-NIZK proof $\pi_{\texttt{nizk}}$ is a $\mathbb{G}_1$ element, its verification key $([\mathbf{C}_i]_2, [a]_2)$ is a vector of $\mathbb{G}_2$ elements. The I/O of the folded relation are the two hashchains ($h^{(x)}$ and $h^{(C)}$), and

[25] We note that the authors of SuperNova mentions in a private communication that when circuits have the same size, they do can share the same Pedersen vector commitment keys. Here, when the circuit size are different, using separate Pedersen commitment keys for individual circuits avoids the extra proof to justify the degree bound of commitments.

[26] In practice, to support the R1CS representation non-natively, we set the limb size to 55, inheriting the setting in Sonobe [1], so that the lazy evaluation of non-native representation can be applied [62].

they can be further hided in circuit. When applying to the decider proof of SuperNova, the $\mathbf{x}$ are the $\mathbb{G}_1$ points contained in each committed R1CS instance.

A.1.3 SuperNova [64] + CycleFold [65] with CyclePair. We now present a slightly refactored SuperNova [64] + CycleFold [65] framework to support cyclepair, which also incorporates the support for finger-printing technique of fixed memory segement introduced in Section 7.3. We use the $\mathsf{NIFS.P}$ and $\mathsf{NIFS.V}$ functions from Nova [68] directly, which folds the concrete and committed R1CS instances. We first introduce a slightly modified definition of committed and relaxed R1CS instance.

Definition A.3. (extending [68, Definition 12]) Given finite field $\mathbb{F}$, the structure of a *Fixed-memory R1CS* (F-R1CS) relation R is denoted by $R = (n, m, t, \mathbf{A}, \mathbf{B}, \mathbf{C}, \rho)$ where $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{n\times n}$, each with $\Omega(n)$ non-zero entries, and m is the public input length. ρ is 1-to-1 map (projection) of size t s.t. $\forall i \in [t] : \rho(i) \in [0, n-1]$. Given $\mathbf{W} \in \mathbb{F}^n$, the projected word $\rho(\mathbf{W})$, also denoted as $\mathbf{W}^{(\rho)}$ has length t and satisfies: $\forall i \in [t] : \mathbf{W}^{(\rho)}{}_i = \mathbf{W}_{\rho(i)}$.

We say $(\mathbf{x}; \mathbf{w}) \in R$ if and only if $(\mathbf{A}\cdot\mathbf{Z})\circ(\mathbf{B}\cdot\mathbf{Z}) = \mathbf{C}\cdot\mathbf{Z}$ where $\mathbf{x} \in \mathbb{F}^m$ and $\mathbf{W} \in \mathbb{F}^{n-m-1}$ and $\mathbf{Z} = (\mathbf{W}, \mathbf{x}, 1)$. A committed R1CS instance for R_ρ is a tuple $(\overline{\mathbf{E}}, u, \overline{\mathbf{W}}, \mathbf{x}, \overline{\mathbf{W}^{(\rho)}})$ where $\overline{\mathbf{W}^{(\rho)}}$, $\overline{\mathbf{W}}$, $\overline{\mathbf{E}}$ are Pedersen vector commitment of $\mathbf{W}^{(\rho)}$, $\mathbf{W}$ and $\mathbf{E}$, s.t. $(\mathbf{A}\cdot\mathbf{Z})\circ(\mathbf{B}\cdot\mathbf{Z}) = u(\mathbf{C}\cdot\mathbf{Z})+\mathbf{E}$ where $\mathbf{Z} = (\mathbf{W}, \mathbf{x}, u)$.

To support the finger-printing techniques, the main circuit (over $\mathbb{F}_r$) is in F-R1CS, and the cyclefold and cyclepair circuits are the standard R1CS as in SuperNova. Their committed instsances are the ones given in SuperNova, i.e., without the $\overline{\mathbf{W}^{(\rho)}}$ component.

We present the augmented circuit F'_j in Figure 10, with slight refactoring of the algorithm in [64, Construction 1]. Let ℓ be the number of circuits. Each F'_j takes a collection of non-deterministic hints from the prover, and mainly performs two operations: (1) computes $\mathsf{z}_{i+1} \leftarrow \mathsf{F}_j(\mathsf{z}_i, \omega_i)$, and (2) generates U_{i+1} by folding U_i (current folded instance) and u_i (incoming instance). Compared with the standard SuperNova, there are three relations that are being folded: the main circuit F_j over $\mathbb{F}_r$, and the cyclefold and cyclepair relations over $\mathbb{F}_q$. To support IVC, the circuit outputs three $\mathbb{F}_r$ elements $(\mathsf{x}^{(\mathtt{main})}, \mathsf{x}^{(\mathtt{fold})}, \mathsf{x}^{(\mathtt{pair})})$, which are the hash of the committed instances of these three folded relations. Function $\mathsf{gen\_cf\_x}()$ generates the input for the cycle fold instance (the random combination of the $\overline{\mathbf{W}}$, $\overline{\mathbf{E}}$, $\overline{\mathbf{W}^{(\rho)}}$ for folding). Notice that all outputs are converted to limbs of $\mathbb{F}_r$.

In Figure 11, we then provide a refactored IVC framework presented [64, Construction 1], by adding cyclepair. The $\mathsf{ProveIVC}()$ function prepares the input for F'_j, and generates the next IVC proof which consists of the pairs of folded and incoming R1CS instances. Here $\mathsf{NIFS.P}'$ is a wrapper of $\mathsf{NIFS.P}$, which given a committed instance pair $(\mathsf{U}_i, \mathsf{W}_i)$ for a relation R, an input $\mathbf{x}$ for R, finds the corresponding witness for $\mathbf{x}$ and generates an incoming instance $(\mathsf{u}_i, \mathsf{w}_i)$ and fold it with $(\mathsf{U}_i, \mathsf{W}_i)$. We know that for $\mathbf{R}_{\mathtt{pair}}$ and $\mathbf{R}_{\mathtt{fold}}$ relations, it takes linear time to compute the witness for a given input.

$\mathbf{1}$ $\left(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}\right) \leftarrow \mathsf{gen\_cf\_x}(\mathsf{vk}, \mathsf{U}, \mathsf{u}, \mathsf{U}', \overline{T})$

(1) $r \leftarrow \mathtt{ro}(\mathsf{vk}, \mathsf{U}, \mathsf{u}, \mathsf{U}', \overline{T})$

(2) Return $\begin{pmatrix} \left(\mathsf{U}.\overline{\mathbf{W}}, \mathsf{u}.\overline{\mathbf{W}}, \mathsf{U}'.\overline{\mathbf{W}}, r\right)_{\mapsto\mathbb{F}_r} \\ \left(\mathsf{U}.\overline{\mathbf{E}}, \mathsf{u}.\overline{\mathbf{E}}, \mathsf{U}'.\overline{\mathbf{E}}, r\right)_{\mapsto\mathbb{F}_r} \\ \left(\mathsf{U}.\overline{\mathbf{W}^{(\rho)}}, \mathsf{u}.\overline{\mathbf{W}^{(\rho)}}, \mathsf{U}'.\overline{\mathbf{W}^{(\rho)}}, r\right)_{\mapsto\mathbb{F}_r} \end{pmatrix}$.

$\mathbf{2}$ $\left(\mathsf{x}^{(\mathtt{main})}, \mathsf{x}^{(\mathtt{fold})}, \mathsf{x}^{(\mathtt{pair})}\right) \leftarrow$
$\mathsf{F}'_j\left(\mathsf{vk}, \left(V^{(\mathtt{main})}, V^{(\mathtt{fold})}, V^{(\mathtt{pair})}\right)\right)$

(1) Parse
$V^{(\mathtt{main})} = \left(\mathsf{U}_i, \mathsf{u}_i, \mathsf{U}_{i+1}, \mathsf{pc}_i, \mathsf{pc}_{i+1}, (i, \mathsf{z}_0, \mathsf{z}_i), \omega_i, \mathsf{O}_i, \overline{T}\right)$.
$V^{(\mathtt{fold})}$
$= \left(\begin{array}{c} \mathsf{U}_i^{(\mathtt{fold})}, \mathsf{U}_{i+1}^{(\mathtt{fold})}, \mathsf{u}^{(\mathtt{fold},1)}, \mathsf{u}^{(\mathtt{fold},2)}, \mathsf{u}^{(\mathtt{fold},3)}, \\ \overline{T^{(\mathtt{fold},1)}}, \overline{T^{(\mathtt{fold},2)}}, \overline{T^{(\mathtt{fold},3)}} \end{array}\right)$.
$V^{(\mathtt{pair})} = \left(\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{U}_{i+1}^{(\mathtt{pair})}, \mathsf{u}_i^{(\mathtt{pair})}, \overline{T^{(\mathtt{pair})}}, \mathbf{x}^{(\mathtt{pair})}\right)$

(2) Run $\phi(\mathsf{pc}_i, \mathsf{pc}_{i+1})$ to assert validty of pc_{i+1}.

(3) $\mathsf{z}_{i+1} \leftarrow \mathsf{F}_j(\mathsf{z}_i, \omega_i)$.

(4) Assert $\mathsf{z}_i = \mathsf{hash}(\mathsf{O}_i)$.

(5) Assert $\mathsf{O}_i.\mathtt{CyclePairInput} = \mathbf{x}^{(\mathtt{pair})}$.

(6) If $i = 0$: assert $\mathsf{z}_i = \mathsf{z}_0$, and
$\left(\mathsf{U}_{i+1}, \mathsf{U}_{i+1}^{(\mathtt{fold})}, \mathsf{U}_{i+1}^{(\mathtt{pair})}\right) \leftarrow \left(\left(\mathsf{u}_{\perp,\mathsf{i}}\right)_{i=1}^{\ell}, \mathsf{u}_{\perp}^{(\mathtt{fold})}, \mathsf{u}_{\perp}^{(\mathtt{pair})}\right)$

(7) If $i > 0$:

(7.1) Check u_i is the commitment to a standard R1CS, i.e., $\mathsf{u}_i.x = \mathsf{hash}(\mathsf{vk}, i, \mathsf{pc}_i, \mathsf{z}_0, \mathsf{z}_i, \mathsf{U}_i)$, and $\mathsf{u}_i.\overline{E} = \mathsf{u}_{\perp,\mathsf{i}}.\overline{E}$, and $\mathsf{u}_i.u = 1$.

(7.2) Check for $k \neq \mathsf{pc}_i$: $\mathsf{U}_{i+1}[k] = \mathsf{U}_i[k]$ and

(7.3) Perform folding of $\mathbf{R}_{\mathtt{fold}}$ and $\mathbf{R}_{\mathtt{pair}}$ natively over $\mathbb{F}_r$:

$$\mathsf{U}^{(\mathtt{fold})} \leftarrow \mathsf{NIFS.V}\left(\mathsf{vk}, \mathsf{U}_{\mathsf{i}}^{(\mathtt{fold})}, \mathsf{u}^{(\mathtt{fold},1)}, \overline{T^{(1)}}\right)$$
$$\mathsf{U}^{(\mathtt{fold})} \leftarrow \mathsf{NIFS.V}\left(\mathsf{vk}, \mathsf{U}^{(\mathtt{fold})}, \mathsf{u}^{(\mathtt{fold},2)}, \overline{T^{(2)}}\right)$$
$$\mathsf{U}_{\mathsf{i}+1}^{(\mathtt{fold})} \leftarrow \mathsf{NIFS.V}\left(\mathsf{vk}, \mathsf{U}^{(\mathtt{fold})}, \mathsf{u}^{(\mathtt{fold},3)}, \overline{T^{(3)}}\right)$$
$$\mathsf{U}_{\mathsf{i}+1}^{(\mathtt{pair})} \leftarrow \mathsf{NIFS.V}\left(\mathsf{vk}, \mathsf{U}_{\mathsf{i}}^{(\mathtt{pair})}, \mathsf{u}_{\mathsf{i}}^{(\mathtt{pair})}, \overline{T^{\mathtt{pair},(1)}}\right)$$

(7.4) Assert Inputs:

$$\left(\mathsf{u}^{(\mathtt{fold},\mathsf{i})}.\mathbf{x}\right)_{i=1}^{3} = \mathsf{gen\_cf\_x}\begin{pmatrix} \mathsf{vk}, \mathsf{U}_i[\mathsf{pc}_i], \mathsf{u}_i, \\ \mathsf{U}_{i+1}[\mathsf{pc}_i], \overline{T} \end{pmatrix}$$
$$\mathsf{u}_{\mathsf{i}}^{(\mathtt{pair})}.\mathbf{x} = \mathbf{x}^{(\mathtt{pair})}$$

(8) Return $\begin{pmatrix} \mathsf{hash}\left(\mathsf{vk}, (i, \mathsf{z}_0, \mathsf{z}_{i+1}), \mathsf{U}_{i+1}\right), \\ \mathsf{hash}\left(\mathsf{U}_{\mathsf{i}+1}^{(\mathtt{fold})}\right), \\ \mathsf{hash}\left(\mathsf{U}_{\mathsf{i}+1}^{(\mathtt{pair})}\right) \end{pmatrix}$.

Figure 10: Modified Augmented Function from SuperNova [64] and CycleFold [65]

To minimize the public I/O size, each z_i is generated from a fixed structure O_i by $\mathsf{hash}(\mathsf{O}_i)$. In O_i there is a field of $\mathtt{CyclePairInput}$ of form $([t_1]_T, [a]_1, [b]_2, [t_2]_T)$ which serves as the input for $\mathbf{R}_{\mathtt{pair}}$. For arithmetic function F_j we use $\mathsf{F}_j^{(\mathtt{Raw})}(\mathsf{pk}, \mathbf{x}, \omega_i)$ to output the O_{i+1} instance, thus F_j can be written as $\mathsf{hash}\left(\mathsf{F}_j^{(\mathtt{Raw})}(\mathsf{pk}, \mathsf{O}_i, \omega_i)\right)$.

THEOREM A.4. *The construction in Figure 11 is a complete and knowledge sound IVC scheme.*

1 $(\mathsf{U}_{i+1}, \mathsf{W}_{i+1}, \mathsf{u}_i, \mathsf{w}_i, \overline{T}) \leftarrow \mathsf{NIFS.P'}(\mathsf{pk}, \mathsf{U}_i, \mathsf{W}_i, \mathbf{x}, R)$
(1) Given $\mathbf{x}$ and R, compute $(\mathbf{x}; \mathbf{w}) \in R$.
(2) Let u_i be committed R1CS instance of $(\mathbf{x}; \mathbf{w})$ for R.
(3) $(\mathsf{U}_{i+1}, \mathsf{W}_{i+1}, \overline{T}) \leftarrow \mathsf{NIFS.P}(\mathsf{pk}, (\mathsf{U}_i, \mathsf{W}_i), (\mathsf{u}_i, \mathsf{w}_i))$.
(4) Return $(\mathsf{U}_{i+1}, \mathsf{W}_{i+1}, \mathsf{u}_i, \mathsf{w}_i, \overline{T})$.

2 $((\mathsf{pk}_i, \mathsf{vk}_i))_{i=1}^{\ell} \leftarrow \mathsf{SetupIVC}\left(\lambda, \{F_i\}_{i=1}^{\ell}\right)$. [64, pp. 17]
(1) $\mathsf{pp}_i \leftarrow \mathcal{G}(1^\lambda)$ for each $i \in [\ell]$.
(2) Let $(\mathsf{pk}_i, \mathsf{vk}_i) \leftarrow \mathsf{NIF.K}(\mathsf{pp}_i, S(\mathsf{F}'_i))$ for each $i \in [\ell]$.
(3) Output $((\mathsf{pk}_i, F_i))_{i=1}^{\ell}$ as pk and $((\mathsf{vk}_i, F_i))_{i=1}^{\ell}$ as vk.

3 $\Pi_{i+1} \leftarrow \mathsf{ProveIVC}(\mathsf{pk}, \mathsf{vk}, (i, z_0, z_i), \omega_i, \mathsf{pc}_{i+1}, \Pi_i)$.
(1) Parse Π_i as $\left((\mathsf{U}_i, \mathsf{W}_i, \mathsf{u}_i, \mathsf{w}_i), \left(\mathsf{U}_i^{(\mathtt{fold})}, \mathsf{W}_i^{(\mathtt{fold})}\right), \left(\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{W}_i^{(\mathtt{pair})}\right), \mathsf{pc}_i, \mathsf{O}_i\right)$.
(2) $\left(\mathbf{x}^{(\mathtt{fold},1)}, \mathbf{x}^{(\mathtt{fold},2)}, \mathbf{x}^{(\mathtt{fold},3)}\right) \leftarrow \mathsf{gen\_cf\_x}(\mathsf{vk}, \mathsf{U}_i, \mathsf{u}_i)$
(3) $\mathbf{x}^{(\mathtt{pair})} \leftarrow \mathsf{O}_i.\mathtt{CyclePairInput}$.
(5) $\mathsf{O}_{i+1} \leftarrow \mathsf{F}_{\mathsf{pc}_{i+1}}^{(\mathtt{Raw})}(\mathsf{O}_i, \omega_i)$ and $\mathsf{z}_{i+1} \leftarrow \mathsf{hash}(O_{i+1})$.
(6) Compute:
$\left(\left(\mathsf{U}^{(\mathtt{fold},\mathtt{j})}, \mathsf{W}^{(\mathtt{fold},\mathtt{j})}, \mathsf{u}^{(\mathtt{fold},\mathtt{j})}, \overline{T^{(\mathtt{fold},\mathtt{j})}}\right)\right)_{j=1}^{3} \leftarrow$
$\left(\mathsf{NIFS.P'}(\mathsf{pk}, \mathsf{U}_i^{(\mathtt{fold})}, \mathsf{W}_i^{(\mathtt{fold})}, \mathbf{x}^{(\mathtt{fold},\mathtt{j})}, \mathbf{R}_{\mathtt{fold}})\right)_{j=1}^{3}$.
$\left(\mathsf{U}_{i+1}^{(\mathtt{pair})}, \mathsf{W}_{i+1}^{(\mathtt{pair})}, \mathsf{u}_i^{(\mathtt{pair})}, \overline{T^{(\mathtt{pair})}}\right) \leftarrow$
$\mathsf{NIFS.P'}(\mathsf{pk}, \mathsf{U}_i^{(\mathtt{pair})}, \mathsf{W}_i^{(\mathtt{fold})}, \mathbf{x}^{(\mathtt{pair})}, \mathbf{R}_{\mathtt{pair}})$.
(7) Let $V = \left(V^{(\mathtt{main})}, V^{(\mathtt{fold})}, V^{(\mathtt{pair})}\right)$ where
$V^{(\mathtt{main})} = \left(\mathsf{U}_i, \mathsf{u}_i, \mathsf{U}_{i+1}, \mathsf{pc}_i, \mathsf{pc}_{i+1}, (i, \mathsf{z}_0, \mathsf{z}_i), \omega_i, \mathsf{O}_i, \overline{T}\right)$.
$V^{(\mathtt{fold})} = \left(\begin{array}{c} \mathsf{U}_i^{(\mathtt{fold})}, \mathsf{U}_{i+1}^{(\mathtt{fold})}, \mathsf{u}^{(\mathtt{fold},1)}, \mathsf{u}^{(\mathtt{fold},2)}, \mathsf{u}^{(\mathtt{fold},3)}, \\ \overline{T^{(\mathtt{fold},1)}}, \overline{T^{(\mathtt{fold},2)}}, \overline{T^{(\mathtt{fold},3)}} \end{array}\right)$.
$V^{(\mathtt{pair})} = \left(\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{U}_{i+1}^{(\mathtt{pair})}, \mathsf{u}_i^{(\mathtt{pair})}, \overline{T^{(\mathtt{pair})}}, \mathbf{x}^{(\mathtt{pair})}\right)$
(8) $(\mathsf{u}_{i+1}, \mathsf{w}_{i+1}) \leftarrow \mathsf{trace}\left(\mathsf{F}'_{\mathsf{pc}_{i+1}}(\mathsf{vk}, V)\right)$
(9) Output Π_{i+1} as
$\left(\begin{array}{c} (\mathsf{U}_{i+1}, \mathsf{W}_{i+1}, \mathsf{u}_{i+1}, \mathsf{w}_{i+1}), \\ \left(\mathsf{U}_{i+1}^{(\mathtt{fold})}, \mathsf{W}_{i+1}^{(\mathtt{fold})}\right), \left(\mathsf{U}_{i+1}^{(\mathtt{pair})}, \mathsf{W}_{i+1}^{(\mathtt{pair})}\right), \mathsf{pc}_{i+1} \end{array}\right)$.

4 $(0, 1) \leftarrow \mathsf{VerIVC}(\mathsf{vk}, (i, z_0, z_i), \Pi_i)$.
If $i = 0$ check $z_i = z_0$ Otherwise Parse Π_i as
$\left((\mathsf{U}_i, \mathsf{W}_i, \mathsf{u}_i, \mathsf{w}_i), \left(\mathsf{U}_i^{(\mathtt{fold})}, \mathsf{W}_i^{(\mathtt{fold})}\right), \left(\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{W}_i^{(\mathtt{pair})}\right), \mathsf{pc}_i\right)$.
check the following:
(1) $1 \le \mathsf{pc}_i \le \ell$.
(2) Assert $\mathsf{u}_i.\mathbf{x} = \left(\begin{array}{l} \mathsf{hash}(\mathsf{vk}, (i, \mathsf{z}_0, \mathsf{z}_i), \mathsf{U}_i), \\ \mathsf{hash}\left(\mathsf{U}_i^{(\mathtt{fold})}\right), \\ \mathsf{hash}\left(\mathsf{U}_i^{(\mathtt{pair})}\right) \end{array}\right)$.
(3) $\forall i \in [\ell]$: u_i, $\mathsf{u}_i^{(\mathtt{fold})}$, $\mathsf{u}_i^{(\mathtt{pair})}$ are standard, i.e., their $\overline{\mathbf{E}}$ is $\mathsf{u}_{\perp,\overline{\mathbf{E}}}$ and $u = 1$.
(4) Assert: $\begin{cases} (\mathsf{u}_i, \mathsf{w}_i) \in \mathsf{F}'_{\mathsf{pc}_i} \\ (\mathsf{U}_i^{(\mathtt{fold})}, \mathsf{W}_i^{(\mathtt{fold})}) \in \mathbf{R}_{\mathtt{fold}} \\ (\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{W}_i^{(\mathtt{pair})}) \in \mathbf{R}_{\mathtt{pair}} \end{cases}$
(5) Assert: $\forall j \in [\ell]$, $(\mathsf{U}_i[j], \mathsf{W}_i[j]) \in \mathsf{F}'_j$.

Figure 11: Main IVC Framework

The completeness of the construction is apparent. As the foldpot scheme is an integration of SuperNova (extension of Nova) and Cyclefold, its soundness proof originates from Lemma 10 of Nova [68], Lemma 3 of SuperNova [64], and Lemma 2 of CycleFold [65], with adaptation from CCS to standard R1CS. For the soundness proof, the key observation is that given IVC proof $\Pi_i = \left((\mathsf{U}_i, \mathsf{W}_i, \mathsf{u}_i, \mathsf{w}_i), \left(\mathsf{U}_i^{(\mathtt{fold})}, \mathsf{W}_i^{(\mathtt{fold})}\right), \left(\mathsf{U}_i^{(\mathtt{pair})}, \mathsf{W}_i^{(\mathtt{pair})}\right), \mathsf{pc}_i, \mathsf{O}_i\right)$, one can extract U_{i-1} and u_{i-1} which fold into U_i. Then based on the extractor properties for folding relations, one can extract the W_{i-1} and w_{i-1} for U_{i-1} and u_{i-1}. The same observation applies to the cyclefold and cycle-pair relations. Thus, it allows the knowledge extraction to proceed recursively. The cyclefold instance, according to Lemma 2 of CycleFold [65] guarantees the entire scheme's knowledge extraction, by asserting the correctness of the out-sourced linear combination of curve points. The correctness of cyclepair is done through the check of $\mathsf{z}_i = \mathsf{hash}(\mathsf{O}_i)$ in the F'_j circuit.

A.2 Circuit

In this section we provide further details about the circuits. Given a set of words $\mathbf{s} = \mathbf{s}^{(1)}||...||\mathbf{s}^{(n)}$, our goal is to prove that a certain predicate $\Phi_{\mathbf{s}}$ are satisfied by these words. As an example, the predicate may assert that each word does not match any of the regex in a malware signature set. As another example, the predicate asserts that the words correctly encode the verifier function of a QA-NIZK relation. We accomplish the "proof" of $\Phi_{\mathbf{s}}$ via a sequential execution of a collection of circuits $\{\mathsf{C}_1, ...\mathsf{C}_k\}$ via N steps s.t. $\forall i \in [N] : z_i = \mathsf{C}^i_{\mathsf{pc}_i}(z_{i-1})$, and in the last output z_N there is a boolean flag indicating if $\Phi_{\mathbf{s}}$ is true. We use IVC here for the folding speed is faster than general purpose SNARK provers. However, the computing model is closer to the Repeated Computation with Global State (RCG) [45] in the way that the prover witnesses of all the steps are already known before folding is started, and so is zk-rollup [25].

A.3 Circuit and Function View of Σ-IR1CS

It is convenient to view a Σ-IR1CS in the point of view of functions and also as an arimetic circuit. We mainly consider functions (circuits, i.e., Σ-IR1CS) that can compute values given input words. To simplify the presentation, we assume that these functions process fixed size words, e.g., a circuit that discharge a word of size 2^{10} of a given set of malware signatures.[27] Note that however, the word problem itself, is not fixed size. We thus distinguish these two cases in the following definition.

Definition A.5. A *word processing function* has the form $z \leftarrow f(\mathbf{w}, \mathbf{p})$, with linear time and space copmlexity. Here $z \in \mathbb{F}$, $\mathbf{w} \in \mathbb{F}^n$ and $\mathbf{p} \in \mathbb{F}^m$ represent the output, the input word, and any additional information. If the function needs to return multiple elements, we let a fixed structure $[\![z]\!]$ encode them s.t. $z = \mathsf{hash}([\![z]\!])$. We call $[\![z]\!]$ its *output structure*. f is called of *fixed size (n)* if n is a fixed constant.

[27] In implementation, we allow circuit to process a word with a prior-set max length, so that we can reduce the number of circuits needed to handle words of arbitrary length, i.e., we do not have to provide circuits which handles word length 2^0, 2^1, etc which are not "economical" given the IVC overhead.

Since our goal is RCG (IVC), we would like to split an expensive function into many steps. We say that a word function $f(\mathbf{w}, \mathbf{p})$ is *streaming*, if it can process the word "character by character". A streaming function is a stronger notion than recompusively computable by IVC, in fact, it implies that the function can be computed chunk by chunk (more efficient than character by character). Streaming functions enjoy a nice property: combination of any two streaming functions is still streaming.[28]

Definition A.6. A word function f is called *streaming*, if there exists a function g of fixed-size 1, whose output structure contains an element v s.t. for any $\mathbf{w}, \mathbf{p}$, letting $n = |\mathbf{w}|$, there exists a sequence of $z_0, ..., \mathtt{z}_n$ satisfying:

(1) $\forall i in [1, n] : z_i = g(\mathbf{w}_i, \mathbf{p})$.
(2) $[\![z_n]\!].v = f(\mathbf{w}, \mathbf{p})$.

We call g the atomic function of f.

Lemma A.7. *Given any two streaming functions $f(\mathbf{w}, \mathbf{p})$ and $g(\mathbf{w}, \mathbf{p})$, their* composition *(denoted as $f \circ g$) is defined as a function $z \leftarrow h(\mathbf{w}, \mathbf{p})$ s.t. $[\![z]\!]$ is a vector $\big(f(\mathbf{w}, \mathbf{p}), g(\mathbf{w}, \mathbf{p})\big)$. h is streaming.*

All the approximated regex discharging algorithms are streaming functions. Streaming implies recursive computation using IVC. When processing a word problem, there is a need to pass information between circuits, e.g., the remaining signatures that are not handled by `CP` will be passed to `BS`, and then `SDE` circuits. [29] We thus further (synatically) restrict Σ-IR1CS relations as special arithmetic circuit with defined structures.

Definition A.8. Given a fixed-size streaming function f, we say a $\Sigma-$ I-R1CS instance $(N, n, \mathbb{F}; \mathbf{T} \in \mathbb{F}^{N\times 2}, \ell_x, \ell_{m_1}, \ell_{m_2}, \ell_{m_3} \in \mathbb{N}; A, B, C \in \mathbb{F}^{n\times m}; \chi)$ is the arithmetic circuit for f, denoted as $\mathtt{C}_f$, if $\ell_x = \ell_w + \ell_i + \ell_o + \ell_d + 2$, i.e., its statement $\mathbf{x}$ is structured as the concatenation of 4 (fixed size) segements $\mathbf{x}^{(w)}$ (the word), $\mathbf{x}^{(i)}$ (the input), $\mathbf{x}^{(o)}$ (the output), $\mathbf{x}^{(d)}$ (the data), and two additional field elements z_{i-1} and z_i, and for any satisfying instance for $\mathtt{C}_f$, its $z_i = f(z_{i-1}, \mathbf{x}^{(w)}||\mathbf{x}^{(i)}||\mathbf{x}^{(o)}||\mathbf{x}^{(d)})$.

Intuitively, given a streaming function f, and its atomic function g. Let $g^{(i)}$ be a function by chaining g for i times. We have a collection of circuits $\left\{\mathtt{C}_{g^{(2^i)}}\right\}_{i=1}^{k}$ that can compute f for any input word. A larger k permits less overhead caused by IVC in practice.

A.4 Batch and Individual Proof

We recall the idea of batch and individual proof and formally define the proof framework in this section. There is a public regex set R, whose approximated discharging algorithms can be encoded as a streaming function which utilizes a lookup table with two columns $\mathbf{k} = (\mathbf{lk}_{(1)}, \mathbf{lk}_{(2)})$, denoted as $f_{\mathbf{k}}$. The prover holds a *secret* set of words $\left\{\mathbf{w}^{(i)}\right\}_{i=1}^{n}$, and alternatively we write it as $(\mathbf{w}, \ell)$ where $\mathbf{w} = \mathbf{w}^{(1)}||...||\mathbf{w}^{(n)}$ and $\forall i \in [n] : \ell_i = |\mathbf{w}^{(i)}|$. The proof proceeds in two phases: (1) *batch proof:* where the prover produces one single proof for $\mathbf{w}$ that each of its $\mathbf{w}^{(i)} \notin R$. (2) *individual proof:* where given $\overline{\mathbf{w}^{(i)}}$, the prover convinces the verifier that it indeed encodes is the i'th word in $\overline{\mathbf{w}}$. This scheme is accomplished using a $O(1)$ size SNARK proof (decider proof for an IVC scheme) and several KZG proofs.

Now consider the individual proof for i. The verifier sees: $\overline{\mathbf{w}^{(i)}}$, $\overline{\mathbf{w}}$, $\overline{\ell}$. The prover can produce a non-interactive proof as below: let $r_i = \mathtt{ro}\left(i, \overline{\mathbf{w}^{(i)}}, \overline{\mathbf{w}}, \overline{\ell}\right)$. let $v_i = \sum_{j=0}^{|\mathbf{w}^{(i)}|-1} r^i \mathbf{w}_j^{(i)}$. The prover can provide a kzg proof to show that $\overline{\mathbf{w}^{(i)}}$ evaluates to v_i at r_i. Now it remains to show that according to ℓ, the corresponding i'th segment of $\mathbf{w}$ is indeed $\mathbf{w}^{(i)}$. This is accomplished combined with the SNARK of the batch proof. In the batch proof, there is an IVC sequence of circuits (whose concatenation of $\mathbf{x}^{(w)}$ is $\mathbf{w}$). We perform in circuit evaluation of r_i for each $\mathbf{w}^{(i)}$. Thus, the verification of individual proof relies on the batch proof, which we elaborate the details below.

We first formally define the statement of a batch proof. We say a streaming predicate is a streaming function which returns a booleaning value. Fix a streaming predicate $\Phi_{\mathbf{k}}$, given any word problem $(\mathbf{w}, \ell)$, we define the public batch proof claim $\mathbb{X}_{(\mathbf{w},\ell,f_{\mathbf{k}})}$ (shortened by $\mathbb{X}$ when the context is clear) as a tuple: $(\overline{\mathbf{w}}, \overline{\ell}, \overline{\mathbf{k}}, \overline{\mathbf{r}}, \overline{\mathbf{v}})$, where $\mathbf{r}$ and $\mathbf{v}$ encodes the r_i and v_i for the individual proof of each word. $\mathbb{X}$ is visible to verifier and all commitments are completely hiding KZG commitments. The relation $R_{\mathbb{X}}$ is defined as below:

$$R_{\mathbb{X}_{\mathbf{w},\ell,\Phi_{\mathbf{k}}}} = \left\{ (\overline{\mathbf{w}}\overline{\ell}, \overline{\mathbf{k}}, \overline{\mathbf{r}}\overline{\mathbf{v}}) \;\middle|\; \begin{array}{l} \mathbf{w} = \mathbf{w}^{(1)}||...||\mathbf{w}^{((|\ell|))} \\ \overline{\mathbf{w}} = \mathtt{kzgcom}(\mathsf{pk}, \mathbf{w}) \\ \overline{\ell} = \mathtt{kzgcom}(\mathsf{pk}, \ell) \\ \overline{\mathbf{r}} = \mathtt{kzgcom}(\mathsf{pk}, \mathbf{r}) \\ \overline{\mathbf{v}} = \mathtt{kzgcom}(\mathsf{pk}, \mathbf{v}) \\ \forall i \in [|\ell|] : \ell_i = |\mathbf{w}^{(i)}| \\ \forall i \in [|\ell|] : \mathbf{v_i} = \sum_{j=0}^{|\ell_i|} (\mathbf{r}_i)^j \cdot \mathbf{w}_j^{(i)} \\ \forall i \in [|\ell|] : \exists \mathbf{p} : \Phi_{\mathbf{k}}(\mathbf{w}^{(i)}, \mathbf{p}) = 1 \end{array} \right\} \tag{7}$$

Intuitively, $R_{\mathbb{X}}$ states that each word satisfies the predicate, the vector lengths match the words, and given a non-deterministic vector $\mathbf{r}$, the weighted sum of each $\mathbf{w}^{(i)}$ using $\mathbf{r}_i$ generates $\mathbf{v}_i$.[30] It is not hard to see that enforcing the $(\mathbf{r}, \mathbf{v})$ semantics itself is a streaming function (essentially computing weighted sum). Combining all required checks (including $\Phi_{\mathbb{X}}$) results in a streaming function, which can be recursively computed through IVC. This immediately applies to the approxmiated Regex discharging problem.

Theorem A.9. *Given any streaming predicate $\Phi_{\mathbf{k}}$, there exists a set of circuits $\mathtt{C}$ and an IVC scheme, such that for any word problem $(\mathbf{w}, \ell)$, the IVC scheme using $\mathtt{C}$ can produce a*

[28] Note that this property does not apply to any recursively computing functions. If two functions each has special requirement on the chunk size, then they cannot be safely composed.

[29] Notice that, these input/output information belong to the *fixed meomry* part of the non-deterministic advice to the circuit, i.e., they do not depend on verifier challenges.

[30] We do not have to specify that r_i is the result of hashing individual claim. This can be checked in the individual proof using vector commitments.

zk-SNARK proof for $R_{\mathbb{X}_{\mathbf{w},\ell,\Phi_{\mathbf{k}}}}$. The the prover work is $O(|\mathbf{w}|)$ group operations, the proof size and verifier time are $O(1)$.

We now introduce the an IVC scheme that realizes Theorem A.9, and present the scheme in Figure 12. It is a concrete scheme that realizes Theorem A.9. It has four stages.

In Stage 1: given a collection of streaming circuits $\{\mathtt{C}_i\}_{i=1}^k$ which computes $f_{R_{\mathbb{X}}}$, we run the standard RCG scheme. At step 4, we first run a heurstics, depending on the `CP`, `SDE`, `DFA` proofs for each word and the capacity of circuits, determine the pc_i for each step to minimize resource consumption. These discharging proofs are encoded as fixed witness $\mathbf{w}^{(\rho,i)}$, and at Step 6 we compute `hc` as the hashchain of the fixed segments first for Fiat-Shamir randoms. Note that we also embed into the fixed segments: (1) each word $\mathbf{w}^{(i)}$, (2) its length ℓ_i, and (3) the randoms and evals of $\mathbf{r}_i$ and $\mathbf{v}_i$ used for individual proof. Then the random challenge r and a combination factor c is generated over `hc`. We run the IVC scheme, and after N steps, at step (8), the ivc proof Π_N asserts the validity of a variety of attributes about the words such as $\ell i = |\mathbf{w}^{(i)}|$, and $\mathbf{v}_i = \sum_{j=0}^{|wordi|-1} {\mathbf{r_i}}^{j-1} \cdot \mathbf{w}^{(i)}[j]$. Note $[\![z_N]\!]$ also includes attribute e as the evaluation of a combination of vectors (polynomials), which we later correlate with KZG proof.

Now at the end of Stage 1, we have a SuperNova IVC proof, which has n instances of committed R1CS, where each has a $\overline{\mathbf{W}^{(i)}}$, $\overline{\mathbf{E}^{(i)}}$, $\overline{\mathbf{W}^{(\rho,i)}}$. We need to demonstrate the knowledge of the correpsonding witness for these commitments. In Sonobe [1], this is handled by providing one KZG commitment for each witness, and do a matching evaluation in circuit. This works when k is small, but will result in a large proof size (e.g., 60 KZG proofs for 20 circuits!)

Stage 2: We take an alternative approach using QA-NIZK proof. The basic idea is that we collect the concentation of all witnesses and call it $\mathbf{W}||\mathbf{E}$ (see Step 11). We then provide a matrix based QA-NIZK proof which shows that the corresponding subsegments of $\mathbf{W}||\mathbf{E}$ generates all the $3k$ commitments. The verification equation takes $3k+1$ pairing operations (see Step 12).

We would like to further compress the proof and verification cost of Step 12. This is done through Stage 2 (steps 11 to 12) where we run another $3k+1$ steps of IVC, using one single chip for $\mathbf{R}_{\mathtt{pair}}$, to prove the QA-NIZK equations. Essentially it is to out-source one pairing operation to Grumpkin curve in each step to minimize folding cost. Apparently, when k is large, the overall prover cost is much smaller compared with encoding all $3k+1$ non-natively over EC1 (e.g., BN254).

Stage 3: We now have two IVC proofs: Π_N for stage 1 (main circuits), and Π_k for the cyclepair circuit. We then generate a SNARK proof π_{snark} (Lines 15-18), which proves the knowledge of these two pairs of IVC proofs, that satisfy a certain properties: (1) $[\![z_N]\!].e$ matches the given e, which is the combined evaluation of polynomials behind $\mathbb{X}$, and (2) $[\![z_N]\!].\Phi_{\mathbb{X}}$ is true.

Recall that the decider of IVC mainly performs two tasks: (1) to prove that the witness satisfy the circuit relations, and (2) to prove that the witness does generate the commitments in the committed SuperNova instances. Here we let $\mathbf{R}_{\mathsf{half}}$ to represent the logic of (1), and use the Sonobe technique to outsource (2) to KZG commitment. The idea is: let $\overline{\mathbf{W}||\mathbf{E}}$ be the KZG commitment to all witness and error terms of all circuits. We provide a KZG proof for showing that it evaluates to a certain value e_2 at a point r_2. Then in circuit, we evaluate the polynomials and prove that all witness/error terms do evaluate to e_2 at r_2. We include such KZG proof in π_{snark} at line (14), and similarly do this for the cyclepair circuit using (r', e').

Stage 4: we build the batch proof as planned, which centeres around the π_{snark}, and provide the corresponding KZG proof. As the 2nd stage SuperNova has only one circuit, we disclose its hiding commitments, and use an extra QA-NIZK proof to prove the knowledge of witness behind these commitments with $\overline{\mathbf{W}'||\mathbf{E}'}$. For individual proof, we mainly re-generating the public randoms, $\mathbf{r}_i$. Show that the committed word $\overline{\mathbf{w}^{(i)}}$ does generate $\mathbf{v}_i$, and show that they are the i'th pair in the vaues embedded in the circuit, which links $\mathbf{w}^{(i)}$ with the i'th word in the circuit.

The batch proof size is: 13 $\mathbb{G}_1$, 1 $\mathbb{G}_2$ and 5 $\mathbb{F}$ elements. Verifier cost is mainly 10 pairings. Individual proof size is 2 $\mathbb{G}_1$, and 1 $\mathbb{F}$. Verifier cost is: 4 pairings.

1 $\left((\mathbb{X}, \pi^{(\mathsf{batch})}), \left(\mathbb{X}_i^{(\mathsf{ind})}, \pi_i^{(\mathsf{ind})}\right)_{i=1}^n\right) \leftarrow$ **prove_all** $\left(\mathsf{pk}, \{\mathbf{w}^{(i)}\}_{i=1}^n, \mathbf{k}, (\mathtt{C}_i)_{i=1}^k\right)$

(1) Compute $\ell \leftarrow \left(|\mathbf{w}^{(i)}|\right)_{i=1}^n$, and $\mathbf{w} \leftarrow \mathbf{w}^{(1)}||...||\mathbf{w}^{(n)}$.

(2) Generate KZG commitments: $\left(\overline{\mathbf{w}^{(i)}}\right)_{i=1}^n$, $\overline{\ell}$, $\overline{\mathbf{w}}$, $\overline{\mathbf{k}}$. Compute $\mathbf{r} \leftarrow \left(\mathtt{ro}(i, \overline{\mathbf{w}^{(i)}}, \overline{\mathbf{w}}, \overline{\ell})\right)_{i=1}^n$, and $\mathbf{v} \leftarrow \left(\sum_{j=0}^{\ell_i - 1} \mathbf{r}_i^j \mathbf{w}_j^{(i)}\right)_{i=1}^n$.

(3) Compute KZG for $\mathbf{r}$ and $\mathbf{v}$. Define $\mathbb{X} = (\overline{\mathbf{w}}, \overline{\ell}, \overline{\mathbf{k}}, \overline{\mathbf{r}}, \overline{\mathbf{v}})$, and for $i \in [n] : \mathbb{X}_i = \left(\overline{\mathbf{w}}, \overline{\ell}, \overline{\mathbf{w}^{(i)}}\right)$.

 # Stage 1: main proof

(4) Based on capacity of $(\mathtt{C}_i)_{i=1}^k$, run heurstics to determine N, the steps needed for computing $f_{R_{\mathbb{X}}}$, and $(\mathsf{pc}_i)_{i=1}^N$, the circuit IDs of all steps

(5) For $i \in [n]$: Run CP, SDE, DFA, ISDE for each $\mathbf{w}^{(i)}$, and generate non-deterministic advice $\omega^{(i)}$ for each $f_{\mathtt{C}_{\mathsf{pc}_i}}$, and based on it, generate the partial witness $\mathbf{W}^{(\rho,i)}$, includes $\mathbf{w}^{(i)}$, $\mathbf{r}_i$, and $\mathbf{v}_i$, and $\mathbf{k}^{(i)}$ (the i'th piece of lookup table distributed into $\mathtt{C}_{\mathsf{pc}_i}$), $\mathbf{q}^{(i)}$ (the query table in circuit $\mathtt{C}_{\mathsf{pc}_i}$). We write $\mathbf{q} = \mathbf{q}^{(1)}||...||\mathbf{q}^{(N)}$, and note $\mathbf{q} \hat{\subset} \mathbf{k}$.

(6) Compute $\mathtt{hc} \leftarrow \mathtt{hashchain}(\{\mathbf{W}^{(\rho,\mathbf{i})}\}_{i=1^N})$. Then $(r, c, \beta) \leftarrow \mathtt{ro}(\mathtt{hc})$.

(7) Compute $z_0 \leftarrow \mathsf{hash}([\![\mathbf{z}_0]\!])$, where $[\![\mathbf{z}_0]\!]$ contains constants $(\mathtt{hc}, r, c, \beta)$ that are passed to each $[\![z_i]\!]$ and variables for computing accumulated sum such as $\sum_{i=1}^N \mathbf{w}_i r^{i-1}$. Let Π_0 be trivially satisfying IVC proof for initialization.

(8) For $i \in [N]$: compute the full non-deterministic advice $\omega^{(i)}$ with $(\mathtt{hc}, r, c, \beta)$, and $z_{i+1} \leftarrow f_{R_{\mathbb{X}}}\left(z_i, \omega^{(i)}\right)$. Run $\Pi_{i+1} \leftarrow \mathsf{ProveIVC}\left(\mathsf{pk}, (i, z_0, z_i), \omega^{(i)}, \mathsf{pc}_{i+1}, \Pi_i\right)$. A valid Π_N proves that via N steps if IVC, $z_N \leftarrow f_{R_{\mathbb{X}}}(\mathbf{w}, \ell)$, where $b_{\mathbb{X}}$ in $[\![z_N]\!]$ asserts

$$\begin{pmatrix} \forall i \in [n] : \ell_i = |\mathbf{w}^{(i)}| \\ \forall i \in [n] : \mathbf{v}_i = \sum_{j=1}^{|\mathbf{w}^{(i)}|} \mathbf{w}_j^{(i)} (\mathbf{r}_i)^{j-1} \\ \sum_{i=1}^{|\mathbf{q}|} \frac{1}{\beta + \mathbf{q}_i^{(1)} + \alpha \cdot \mathbf{q}_i^{(2)}} = \sum_{i=1}^{|\mathbf{k}|} \frac{\mathbf{m}_i}{\beta + \mathbf{k}_i^{(1)} + \alpha \cdot \mathbf{k}_i^{(2)}} \\ \forall i \in [n] : \mathtt{CP}(\mathbf{w}^{(i)}) \subseteq \mathtt{SDE}(\mathbf{w}^{(i)}) \cup \mathtt{DFA}(\mathbf{w}^{(i)}) \cup \mathtt{ISDE}(\mathbf{w}^{(i)}) \end{pmatrix} \text{ and } v \text{ in } [\![z_N]\!] \text{ has value } \begin{pmatrix} c^0 \cdot \sum_{i=1}^{|\mathbf{k}|} \mathbf{k}_i^{(1)} \cdot r^{i-1} \\ +c^1 \cdot \sum_{i=1}^{|\mathbf{k}|} \mathbf{k}_i^{(2)} \cdot r^{i-1} \\ +c^2 \cdot \sum_{i=1}^{|\mathbf{w}|} \mathbf{w}_i \cdot r^{i-1} \\ +c^3 \cdot \sum_{i=1}^{|\overline{\ell}|} \ell_i \cdot r^{i-1} \\ +c^4 \cdot \sum_{i=1}^{|\overline{\ell}|} \mathbf{r}_i \cdot r^{i-1} \\ +c^5 \cdot \sum_{i=1}^{|\overline{\ell}|} \mathbf{v}_i \cdot r^{i-1} \end{pmatrix}$$

 # Stage 2: CyclePair Proof For Saving Verifier Cost

(10) Parse Π_N as $\left((\mathsf{U}_N, \mathsf{W}_N, \mathsf{u}_N, \mathsf{w}_N), \left(\mathsf{U}_i^{(\mathsf{fold})}, \mathsf{W}_i^{(\mathsf{fold})}\right), \left(\mathsf{U}_i^{(\mathsf{pair})}, \mathsf{W}_i^{(\mathsf{pair})}\right), \mathsf{pc}_N, \mathsf{O}_N\right)$. Compute $(\mathsf{U}_{N+1}, \mathsf{W}_{N+1})$ by folding $(\mathsf{U}_N, \mathsf{W}_N)$ and $(\mathsf{u}_N, \mathsf{w}_N)$. For $j \in [k]$: extract the comitments: $\left(\overline{\mathbf{W}^{(j)}}, \overline{\mathbf{E}^{(j)}}, \overline{\mathbf{W}^{(\rho,j)}}\right)$ from $\mathsf{U}_{N+1}[j]$ and witness and error terms $\left(\mathbf{W}^{(j)}, \mathbf{E}^{(j)}\right)$ from $\mathsf{W}_{N+1}[j]$. Let $\mathbf{W} = \mathbf{W}^{(1)}||...||\mathbf{W}^{(k)}$, i.e., concatenated witness. Note that $\mathbf{W}^{(\rho,i)}$ is a subsegment of $\mathbf{W}^{(i)}$ located at fixed position. Similarly let $\mathbf{E} = \mathbf{E}^{(1)}||...||\mathbf{E}^{(k)}$. Compute $\overline{\mathbf{W}||\mathbf{E}}$ as a hiding KZG commitment of $\mathbf{W}||\mathbf{E}$.

(11) Construct QA-NIZK Proof π_{qa} for the following relation that asserts the KZG commitment $\overline{\mathbf{W}||\mathbf{E}}$ with all commitments in supernova commmitted instance U_{N+1}: $\left\{\left((\mathbf{W}^{(j)}, \mathbf{E}^{(j)})_{j=1}^k\right) : \forall j \in [k] : \begin{pmatrix} \overline{\mathbf{W}^{(j)}} = \mathtt{kzgcom}(\mathbf{W}^{(j)}). \\ \overline{\mathbf{W}^{(\rho,j)}} = \mathtt{kzgcom}(\mathbf{W}^{(\rho,j)}). \\ \overline{\mathbf{E}^{(j)}} = \mathtt{kzgcom}(\mathbf{E}^{(j)}), \end{pmatrix}, \text{ and } \overline{\mathbf{W}||\mathbf{E}} = \mathtt{kzgcom}(\mathbf{W}^{(1)}||...||\mathbf{W}^{(k)}||\mathbf{E}^{(1)}||...||\mathbf{E}^{(k)}).\right\}$, where its public input $\mathbf{x}_{\mathsf{qa}}$ is: $\left(\overline{\mathbf{w}||\mathbf{E}}, \left(\overline{\mathbf{W}^{(j)}}, \overline{\mathbf{E}^{(j)}}, \overline{\mathbf{W}^{(\rho,j)}}\right)_{j=1}^k\right)$. Note that its verifier key $\mathsf{vk}_{\mathsf{qa}} = (\mathbf{C}, a)$ is a $3k+2$ vector of $\mathbb{G}_2$ elements, with $|\mathbf{C}| = 3k+1$. We now construct another IVC to prove the QA-NIZK equation $\sum_{j=1}^{3k+1} e(\mathbf{x}_{\mathsf{qa}_j}, \mathbf{C}_j) = e(\pi_{\mathsf{qa}}, a)$.

(12) Repeat step (4) to (8) for a single circuit system where the circuit encodes $\mathbf{R}_{\mathsf{pair}}$. Running k steps of IVC, results in the assertion for public statement: $(\mathtt{hc}_1, \mathtt{hc}_2)$, where there exists $\mathbf{x}_{\mathsf{qa}} \in \mathbb{G}_1^{3k+1}$ and $\mathbf{C} \in \mathbb{G}_2^{3k+1}$ s.t. $\sum_{j=1}^{3k+1} e(\mathbf{x}_{\mathsf{qa}_j}, \mathbf{C}_j) = e(\pi_{\mathsf{qa}}, a)$, i.e., the QA-NIZK verification equation.

 # Stage 3. Generate SNARK Proof for

(13) Given a supernova IVC proof Π for a scheme with k instances, let predicate $\mathbf{R}_{\mathsf{half}}(\Pi_i)$ defines the "half" of the SuperNova decider check on Π_i by skipping the check on witness commitments. Concretely it is define as as below: Parse $\Pi_i = \left((\mathsf{U}_i, \mathsf{W}_i, \mathsf{u}_i, \mathsf{w}_i), \left(\mathsf{U}_i^{(\mathsf{fold})}, \mathsf{W}_i^{(\mathsf{fold})}\right), \left(\mathsf{U}_i^{(\mathsf{pair})}, \mathsf{W}_i^{(\mathsf{pair})}\right), \mathsf{pc}_i, \mathsf{O}_i\right)$. Compute (non-natively for group operations in circuit) $(\mathsf{U}_{i+1}, \mathsf{W}_{i+1}, \mathsf{u}_{i+1}, \mathsf{w}_{i+1})$ by folding $(\mathsf{U}_i, \mathsf{W}_i, \mathsf{u}_i, \mathsf{w}_i)$. Use non-deterministic advice $[\![z_i]\!]$ to show to show: (1) $(\mathsf{u}_i.\overline{E}, \mathsf{u}_i.u) = (\mathsf{u}_\perp.\overline{E}, 1)$. (2) $\mathsf{u}_{i+1}.x = \left(\mathsf{hash}(\mathsf{vk}, i, z_0, z_i, U_{i+1}), \mathsf{hash}(\mathsf{vk}, \mathsf{U}_i^{(\mathsf{fold})}), \mathsf{hash}(\mathsf{vk}, \mathsf{U}_i^{(\mathsf{pair})})\right)$. (3) $\mathsf{hash}([\![z_i]\!]) = z_i$. (4) $\forall j \in [k] : \mathsf{U}_{i+1}[j]$ and $\mathsf{W}_{i+1}[j]$ is a relaxed R1CS instance for circuit $\mathtt{C}_j$. (4) $\mathsf{U}_i^{(\mathsf{fold})}$ and $\mathsf{W}_i^{(\mathsf{fold})}$ is a relaxed cyclefold instance. (5) $\mathsf{U}_i^{(\mathsf{pair})}$ and $\mathsf{W}_i^{(\mathsf{pair})}$ is a relaxed cyclepair instance.

Now given Π_N (the IVC proof of stage 1), and given $\mathbf{W}||\mathbf{E}$ in Step 10. Let $[\![z_N]\!]$ be the non-deterministic advice for $z_N = \mathsf{hash}([\![z_N]\!])$. Compute $r_2 \leftarrow \mathtt{ro}([\![z_N]\!].\overline{\mathbf{W}^{(\rho)}}, \overline{\mathbf{W}||\mathbf{E}})$. Compute $e_2 \leftarrow \sum_{i=1}^{|\mathbf{W}||\mathbf{E}|} (\mathbf{W}||\mathbf{E})_i \cdot r_2^{i-1}$. Compute the correpsonding KZG proof $\pi_{\mathtt{kzg\_WE}}$ for e_2. Similarly for Π'_k (the IVC proof for stage 2), given $[\![z'_k]\!]$, $\mathbf{W}'||\mathbf{E}'$ extracted from Π'_k, compute $r' \leftarrow \mathtt{ro}([\![z_k]\!]'.\overline{\mathbf{W}'^{(\rho)}}, \overline{\mathbf{W}'||\mathbf{E}'})$, and $e' \leftarrow \sum_{i=1}^{(} \mathbf{W}'||\mathbf{E}')_i \cdot r'^{i-1}$. Similarly compute the $\pi_{\mathtt{kzg\_WE'}}$ for e'. Let $(\mathsf{U}'_{i+1}, \mathsf{W}'_{i+1})$ be the folded instance for Π'_k as shown in computing $\mathbf{R}_{\mathsf{half}}$. Since stage (2) has one circuit, collect $(\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}})$ from its own $\mathsf{W}'_{i+1}[0]$. Similarly to step (11), construct $\pi_{\mathtt{qa\_WE'}}$ for showing the relation between $\overline{\mathbf{W}'||\mathbf{E}'}$, and $\overline{\mathbf{W}'}$, $\overline{\mathbf{E}'}$, $\overline{\mathbf{W}'^{(\rho)}}$. Now construct the public statement for the SNARK proof as below: $\mathbb{X}_{\mathsf{snark}} = \left(\left((r, c, [\![z_N]\!].v), (r_2, e_2), [\![]\!]z_N.\overline{\mathbf{W}^{(\rho)}}\right), \left((r', e'), (\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}})\right)\right)$

(14) Produce π_{snark} for $\mathbb{X}_{\mathsf{snark}} = \mathbb{X}_{\mathsf{snark}} = \left(\left((r, c, e), (r_2, e_2), \overline{\mathbf{W}^{(\rho)}}\right), \left((r', e'), (\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}})\right)\right)$, i.e., there exists Π_N and Π'_k which satisfies the following: (1) $\mathbf{R}_{\mathsf{half}}(\Pi_N)$ for stage 1, (2) $\mathbf{R}_{\mathsf{half}}(\Pi'_k)$ for stage 2, (3) $(r, c, e, \overline{\mathbf{W}^{(\rho)}}) \in [\![z_N]\!]$, (4) Given r_2, and $\mathbf{W}||\mathbf{E}$ collected from u_N, verify e_2 is equal to the value computed in Step (13). (5) Similarly verify e' using formula given in Step (13). (6) Assert $\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}} = \mathsf{u}'_k[0]$

 # Stage 4: Build Batch and Individual Proof

(15) Compute $\overline{\mathbf{r}}^{(\mathsf{VEC})} \leftarrow \mathtt{veccom}(\mathbf{r})$, $\overline{\mathbf{v}}^{(\mathsf{VEC})} \leftarrow \mathtt{veccom}(\mathbf{v})$. Compute QA-NIZK proof $\pi_{\mathtt{qa\_r\_v}} \in \mathbb{G}_1$ for showing their equivalence to KZG, i.e., there exist $\mathbf{r}$ which generates both $\overline{\mathbf{r}}^{(\mathsf{VEC})}$ and $\overline{\mathbf{r}}$ (KZG), and also $\mathbf{v}$ which generates both for $\overline{\mathbf{v}}^{(\mathsf{VEC})}$ and $\overline{\mathbf{v}}$.

(16) Comute batched KZG proof $\pi_{\mathtt{kzg\_agg}}$, which asserts that given $\mathbb{X} = \left(\overline{\mathbf{k}^{(1)}}, \overline{\mathbf{k}^{(2)}}, \overline{\mathbf{w}}, \overline{\ell}, \overline{\mathbf{r}}, \overline{\mathbf{v}}\right)$, combining the evaluation at r using combinning factor c, results in e, Formally, let $\mathbf{p}$ be the corresponding polynoaims for $\mathbb{X}$, $e = \sum_{i=1} c^{i-1} \cdot \mathbf{p}_i(r)$,i.e., it matches the $[\![z_N]\!].v$ in Step (9).

(17) Output $\pi^{(\mathsf{batch})} = \left(\mathtt{hc}(\{\overline{\mathbf{W}^{(\rho,i)}}\}_{i=1}^N), \left(\overline{\mathbf{r}}^{(\mathsf{VEC})}, \overline{\mathbf{v}}^{(\mathsf{VEC})}, \pi_{\mathtt{qa\_r\_v}}\right), (r, c, e, \pi_{\mathtt{kzg\_agg}}), \left(\overline{\mathbf{W}||\mathbf{E}}, r_2, e_2, \pi_{\mathtt{kzg\_WE}}\right), \left(\overline{\mathbf{W}'||\mathbf{E}'}, r', e', \pi_{\mathtt{kzg\_WE'}}\right), \left(\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}}, \pi_{\mathtt{qa\_WE'}}\right), \pi_{\mathsf{snark}}\right)$.

(18) For each $i \in [n]$: Compute $c_i \leftarrow \mathtt{ro}(\mathbf{r}_i, \mathbf{v}_i)$, ${\pi_{\mathtt{veccom}}}^{(i)} \leftarrow \mathtt{veccom.prove}(\mathsf{pk}, \mathbf{r} + c_i \cdot \mathbf{v}, i, \mathbf{r}_i + c_i \cdot \mathbf{v}_i)$. $\pi_{\mathtt{kzg}}^{(\mathbf{w}_i)} \leftarrow \mathtt{kzgcom.prove}(\mathsf{pk}, \mathbf{w}^{(i)}, \mathbf{r}_i, \mathbf{v}_i)$. Output $\pi_i^{(\mathsf{ind})} = \left({\pi_{\mathtt{veccom}}}^{(i)}, \mathbf{v}_i, \pi_{\mathtt{kzg}}^{(\mathbf{w}_i)}\right)$.

2 $1/0 \leftarrow$ **verify_batch** $\left(\mathsf{vk}, \mathbb{X}, \pi^{(\mathsf{batch})}\right)$

(1) Parse $\mathbb{X} = \left(\overline{\mathbf{k}^{(1)}}, \overline{\mathbf{k}^{(2)}}, \overline{\mathbf{w}}, \overline{\ell}, \overline{\mathbf{r}}, \overline{\mathbf{v}}\right)$, and $\pi^{(\mathsf{batch})} = \left(\mathtt{hc}, \left(\overline{\mathbf{r}}^{(\mathsf{VEC})}, \overline{\mathbf{v}}^{(\mathsf{VEC})}, \pi_{\mathtt{qa\_r\_v}}\right), (e, \pi_{\mathtt{kzg\_agg}}), \left(\overline{\mathbf{W}||\mathbf{E}}, e_2, \pi_{\mathtt{kzg\_WE}}\right), \left(\overline{\mathbf{W}'||\mathbf{E}'}, e', \pi_{\mathtt{kzg\_WE'}}\right), \left(\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}}, \pi_{\mathtt{qa\_WE'}}\right), \pi_{\mathsf{snark}}\right)$.

(2) Compute $(r, c) \leftarrow \mathtt{ro}(\mathbb{X}, \mathtt{hc})$, and $r_2 \leftarrow \mathtt{ro}\left(\mathtt{hc}, \overline{\mathbf{W}||\mathbf{E}}\right)$, and $r' \leftarrow \mathtt{ro}\left(\overline{\mathbf{W}'^{(\rho)}}, \overline{\mathbf{W}'||\mathbf{E}'}\right)$.

(3) Assert $\mathtt{qa\_nizk.verify}\left(\mathsf{vk}, \left(\overline{\mathbf{r}}^{(\mathsf{VEC})}, \overline{\mathbf{v}}^{(\mathsf{VEC})}, \overline{\mathbf{r}}, \overline{\mathbf{v}}\right), \pi_{\mathtt{qa\_r\_v}}\right)$. and $\mathtt{qa\_nizk.verify}\left(\mathsf{vk}, \left(\overline{\mathbf{W}||\mathbf{E}}, \overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}}\right), \pi_{\mathtt{qa\_WE'}}\right)$.

(4) Assert $\mathtt{kzgcom.verify}(\mathsf{vk}, \sum_{i=1}^{6} \mathbb{X}_i \cdot c^{i-1}, r, e, \pi_{\mathtt{kzg\_agg}})$, and $\mathtt{kzgcom.verify}(\mathsf{vk}, \overline{\mathbf{W}||\mathbf{E}}, r_2, e_2, \pi_{\mathtt{kzg\_WE}})$, and $\mathtt{kzgcom.verify}(\mathsf{vk}, \overline{\mathbf{W}'||\mathbf{E}'}, r', e', \pi_{\mathtt{kzg\_WE'}})$.

(5) Construct $\mathbb{X}_{\mathsf{snark}} = \left(\left((r, c, e), (r_2, e_2), \overline{\mathbf{W}^{(\rho)}}\right), \left((r', e'), (\overline{\mathbf{W}'}, \overline{\mathbf{E}'}, \overline{\mathbf{W}'^{(\rho)}})\right)\right)$, Assert $\mathtt{groth16.verify}(\mathsf{vk}, \mathbb{X}_{\mathsf{snark}}, \pi_{\mathsf{snark}})$.

3 $1/0 \leftarrow$ **verify_individual** $\left(\mathsf{vk}, \mathbb{X}, \mathbb{X}_i^{(\mathsf{ind})}, \pi_i^{(\mathsf{ind})}, \right)$

(1) Parse $\mathbb{X}_i^{(\mathsf{ind})} = \left(i, \overline{\mathbf{w}^{(i)}}, \overline{\mathbf{w}}, \overline{\ell}\right)$, and $\pi_i^{(\mathsf{ind})} = \left({\pi_{\mathtt{veccom}}}^{(i)}, \mathbf{v}_i, \pi_{\mathtt{kzg}}^{(\mathbf{w}_i)}\right)$.

(2) Compute $\mathbf{r}_i \leftarrow \mathtt{ro}(\mathbb{X}_i^{(\mathsf{ind})})$. Assert $\mathtt{kzgcom.verify}(\mathsf{vk}, \overline{\mathbf{w}^{(i)}}, \mathbf{r}_i, \mathbf{v}_i, \pi_{\mathtt{kzg}}^{(\mathbf{w}_i)})$.

(3) Compute $c_i \leftarrow \mathtt{ro}(\mathbf{r}_i, \mathbf{v}_i)$. Retrieve $\overline{r}^{(\mathsf{VEC})}$ and $\overline{v}^{(\mathsf{VEC})}$ from $\mathbb{X}$. Assert $\mathtt{veccom.verify}(\mathsf{vk}, \overline{\mathbf{r}}^{(\mathsf{VEC})} + c_i \cdot \overline{\mathbf{v}}^{(\mathsf{VEC})}, i, \mathbf{r}_i + c_i \cdot \mathbf{v}_i)$.